



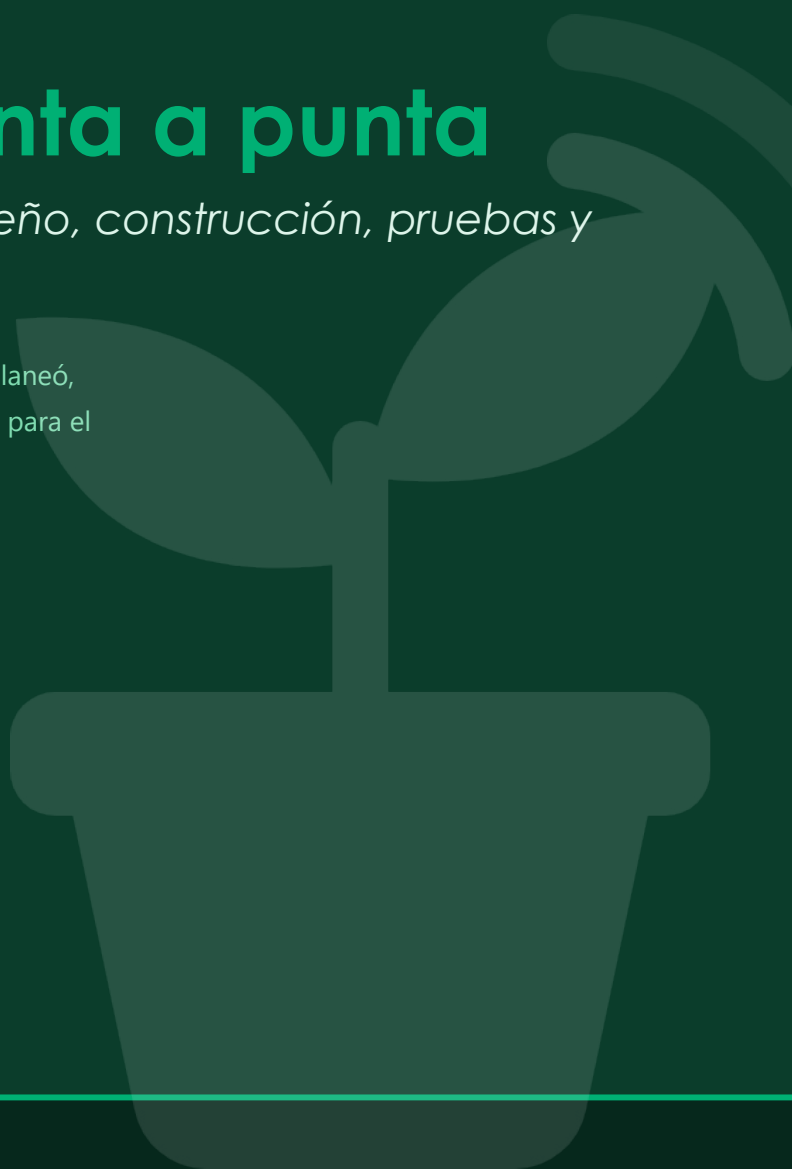
SmartPot

CICLO DE VIDA DEL SOFTWARE

SmartPot de **punta a punta**

Formulación, gestión, análisis, diseño, construcción, pruebas y mejora continua

Una lectura crítica de cada etapa del proyecto: qué se planeó, qué evidencia dejó, qué no cuadraba y qué se aprendió para el siguiente ciclo.



DOCUMENTO

Ciclo de vida del software

VERSIÓN

1.1 · octubre 2026

EQUIPO

SmartPotTech

PROYECTO

smartpot.app



DOCUMENTO

Ficha del documento

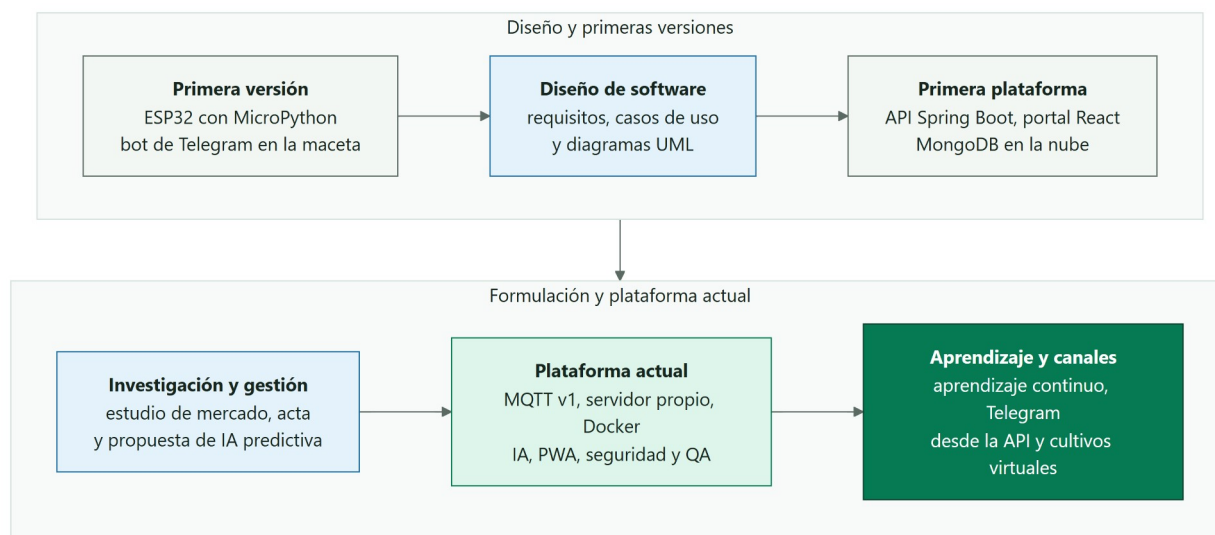
CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Organización	SmartPotTech
Documento	Ciclo de vida del software: formulación y evaluación, gestión, análisis, diseño, construcción, pruebas y mejora continua
Versión	1.1 · octubre 2026
Periodo	Septiembre de 2024 a octubre de 2026
Fuentes	Formulación del proyecto (idea, problema, árboles, 5 porqués, PESTEL, involucrados, marco lógico), estudio de mercado, acta de constitución, informe de gestión del cronograma y los recursos, informe de tareas críticas, propuesta de investigación, requisitos, casos de uso, diagramas UML del diseño y base de conocimiento; todo contrastado con el historial de los repositorios y la plataforma en producción
Documentos hermanos	Documentación técnica : la plataforma tal como funciona hoy. Recorrido del proyecto : requisito por requisito y diagrama por diagrama, qué se planeó y qué quedó construido
Cómo leerlo	Cada parte corresponde a una disciplina del ciclo de vida y tiene tres capas: lo que se hizo, con su diagrama; la lectura crítica , con lo que estuvo bien y lo que no cuadraba; y la evidencia hoy , con lo que queda en el código, las pruebas o la operación

SECCIÓN 01

El proyecto en el tiempo

En palabras simples

SmartPot pasó por todas las etapas de un proyecto de software, aunque no en el orden de un libro. Primero se diseñó y se construyó una primera versión; después se formuló como proyecto (problema, objetivos, mercado y finanzas); luego se planeó con las herramientas de la gerencia de proyectos; y al final la plataforma se reestructuró en ciclos cortos de mejora continua hasta quedar en producción. Este documento recorre esas etapas como disciplinas y, en cada una, separa lo que se dijo de lo que se puede comprobar.

**Figura 1** · Etapas del proyecto

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph antes["Diseño y primeras versiones"]
        direction LR
        f1["<b>Primera versión</b><br/>ESP32 con MicroPython<br/>bot de Telegram en la maceta"]
        f1 --> f2["<b>Diseño de software</b><br/>requisitos, casos de uso<br/>y diagramas UML"]
        f2 --> f3["<b>Primera plataforma</b><br/>API Spring Boot, portal React<br/>MongoDB en la nube"]
    end
    subgraph despues["Formulación y plataforma actual"]
        direction LR
        f4["<b>Investigación y gestión</b><br/>estudio de mercado, acta<br/>y propuesta de IA predictiva"] --> f5["<b>Plataforma actual</b><br/>MQTT v1, servidor propio, Docker<br/>IA, PWA, seguridad y QA"]
        f5 --> f6["<b>Aprendizaje y canales</b><br/>aprendizaje continuo, Telegram<br/>desde la API y cultivos virtuales"]
    end
    antes --> despues
    classDef past fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    classDef design fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef now fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef latest fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class f1,f3 past
    class f2,f4 design
    class f5 now
    class f6 latest
```

SEMESTRE	DISCIPLINAS DOMINANTES	QUÉ PRODUJO	COMMITTS HUMANOS
2024-2	Análisis y diseño	Primera matriz de requisitos (agosto de 2024), casos de uso, diagramas UML, primera API, primer portal, firmware del ESP32, simulador y primeros análisis de datos	379
2025-1	Construcción	Ajustes y mantenimiento de la primera plataforma	141
2025-2	Formulación y evaluación	Idea, problema, árboles, marco lógico, estudio de mercado (noviembre de 2025) y crecimiento de la PWA	553
2026-1	Gestión de proyectos	Acta de constitución, EDT, cronograma con ruta crítica, recursos y nivelación; propuesta de investigación	145



SEMESTRE	DISCIPLINAS DOMINANTES	QUÉ PRODUJO	COMMITTS HUMANOS
2026-2	Construcción, pruebas y mejora continua	Reestructuración completa, cinco ciclos de mejora y la plataforma en producción	714

Al cierre de septiembre de 2026, los repositorios sumaban 2217 commits, de los cuales 1932 eran de personas y el resto de bots de automatización, sobre todo actualizaciones de dependencias. La tabla ya deja ver la primera lección del proyecto: los semestres de planeación formal (2026-1) fueron los de menos construcción, y el mayor avance llegó cuando el trabajo se organizó en ciclos cortos con pruebas automáticas.

DISCIPLINA	PARTE	DIAGRAMAS
Formulación y evaluación de proyectos TI	I	Árbol de problemas, causa y efecto, árbol de objetivos, poder e interés, fases y cronograma de la investigación
Gestión de proyectos TI	II	EDT, red de precedencias, ruta crítica, ciclo Scrum, riesgos
Análisis	III	Casos de uso
Diseño	IV	Contexto, contenedores, paquetes, componentes, clases, objetos, artefactos, actividades, secuencia, estados, datos, mapa de la PWA, redes
Construcción	V	Despliegue, asistente de IA, aprendizaje, Telegram, cultivos reales y virtuales, ilustración de cada cultivo
Pruebas	VI	Pirámide de pruebas, workflow de QA
Mejora continua	VII	Ciclo PDCA, ciclos de mejora

CONTENIDO

Contenido

1. El proyecto en el tiempo

2. Identificación del problema

- 2.1 La idea
- 2.2 Árbol de problemas
- 2.3 Causa y efecto
- 2.4 Los 5 porqués

3. Objetivos y alternativa elegida

- 3.1 Árbol de objetivos
- 3.2 Objetivo SMART
- 3.3 Cómo evolucionaron los objetivos

4. Entorno e involucrados



4.1 PESTEL

4.2 Poder e interés

5. Marco lógico

6. Estudio de mercado

6.1 Demanda y oferta

6.2 Coherencia de las cifras

6.3 Qué quedó del estudio

6.4 Precio: las 5 C

6.5 Comercialización

6.6 Fases de penetración

7. Estudio técnico

7.1 Base temática

8. Estudio financiero y evaluación

8.1 Presupuestos

8.2 Modelo de evaluación

9. Propuesta de investigación

9.1 Marco teórico

9.2 Cronograma de la investigación

10. Acta de constitución: lo que fijó y lo que pasó

11. Alcance

11.1 Estructura de desglose del trabajo

11.2 Priorización y MVP

12. Cronograma

12.1 Red de precedencias

12.2 Ruta crítica

13. Recursos, asignación y nivelación

14. Riesgos

15. Metodología, comunicaciones y control de cambios

16. De la necesidad al requisito

17. Casos de uso

17.1 Especificación de los casos del diseño

18. Trazabilidad

19. Reglas de negocio y base de conocimiento

20. Arquitectura

20.1 Contexto

20.2 Contenedores

20.3 Decisiones de arquitectura

21. Del diseño original al actual



- 21.1 Clases
- 21.2 Componentes
- 21.3 Comportamiento: enviar un comando
- 21.4 Comportamiento: datos históricos y una lectura
- 21.5 Actividad: control automático
- 21.6 Datos
- 21.7 Objetos y artefactos
- 21.8 Paquetes

22. Interfaz

23. Seguridad por diseño

24. Organización del trabajo

25. Infraestructura y despliegue

26. Asistente de IA

27. Integraciones

- 27.1 Telegram
- 27.2 Cultivos reales y virtuales
- 27.3 Lugar, switches y avisos por cultivo

28. Aprendizaje continuo

29. Estrategia

30. Criterios de aprobación del acta

31. Calidad de los modelos

32. El ciclo de mejora

33. Ciclos de mejora

34. Mejora que ocurre sola

35. Indicadores

36. Backlog del próximo ciclo

37. Lecciones aprendidas



PARTE I

Formulación y evaluación de proyectos TI

SECCIÓN 02

Identificación del problema

2.1 La idea

SmartPot fue la idea elegida entre las propuestas de proyecto de un curso de tecnologías de la información. Nació de una observación simple: el riego y la nutrición de un cultivo hidropónico dependen de variables que cambian todo el tiempo (pH, sólidos disueltos, luz, temperatura y humedad), y la mayoría de pequeños productores y aficionados las controla a mano. La idea citaba tres datos de la FAO: la agricultura usa cerca del 70 % del agua dulce, el riego tradicional pierde hasta el 40 % por ineficiencia y la hidroponía puede reducir el consumo hasta en un 90 %.

La idea se formuló como un proyecto de datos y no como un aparato:

ALCANCE DE LA IDEA	CÓMO QUEDÓ CONSTRUIDO
Captura en tiempo real desde sensores	Telemetría MQTT v1 con credencial por cultivo
Integración y estandarización de las mediciones	Contrato JSON único y validadores <code>\$jsonSchema</code> en MongoDB
Automatización del riego según lo sensado	Agente de automatización con modo automático y enfriamiento por actuador
Visualizaciones, tabulados y alertas	PWA con panel general, historial, comparador y alertas por Telegram
Diccionario de variables y metadatos	Contrato MQTT, esquema de datos y base de conocimiento documentados
Documentación reproducible	Tres documentos, diagramas en Mermaid y una demo de un solo comando

LECTURA CRÍTICA

Plantear SmartPot como un *pipeline* de datos (capturar, limpiar, integrar, decidir, mostrar) fue la mejor decisión de la formulación. Separó el valor del hardware: el dispositivo solo mide y obedece, y todo lo que aprende la plataforma sirve igual para una maceta física, una simulada en Wokwi o una virtual.

2.2 Árbol de problemas

El problema central se definió como la **gestión ineficiente de cultivos hidropónicos por falta de control preciso y continuo de sus parámetros**.

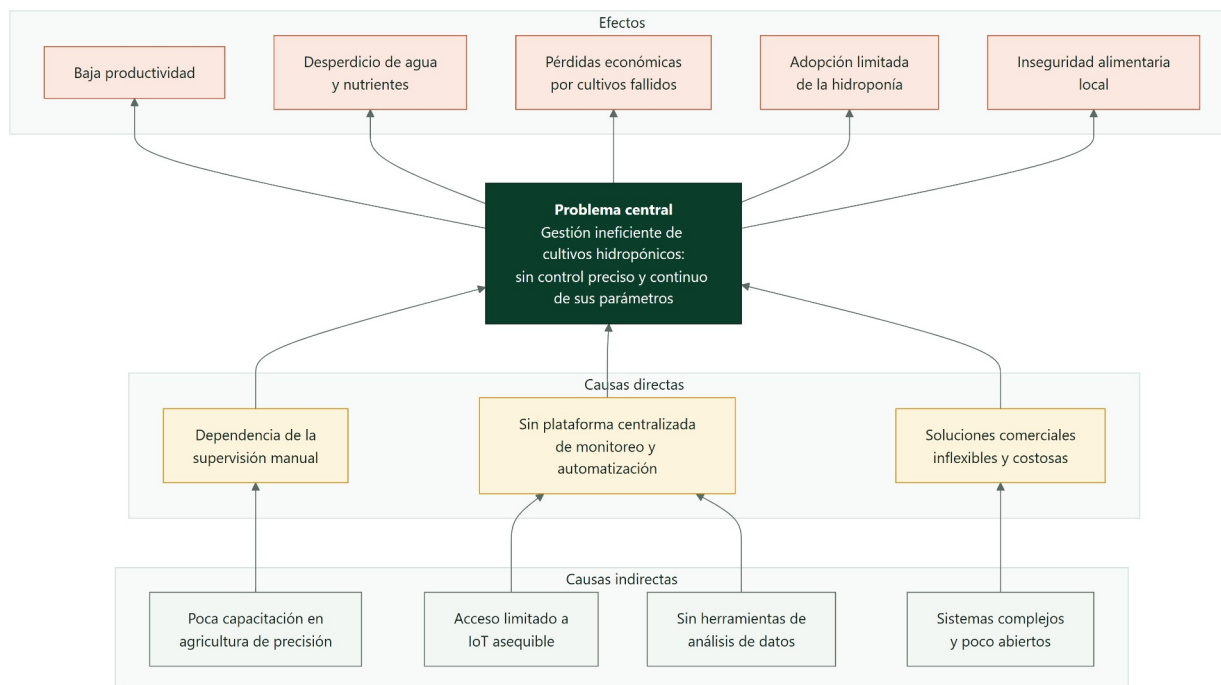


Figura 2 · Árbol de problemas



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart BT
    subgraph efectos["Efectos"]
        e1["Baja productividad"]
        e2["Desperdicio de agua<br/>y nutrientes"]
        e3["Pérdidas económicas<br/>por cultivos fallidos"]
        e4["Adopción limitada<br/>de la hidroponía"]
        e5["Inseguridad alimentaria<br/>local"]
    end
    problema["<b>Problema central</b><br/>Gestión ineficiente de cultivos hidropónicos:<br/>sin
control preciso y continuo de sus parámetros"]
    subgraph directas["Causas directas"]
        c1["Dependencia de la<br/>supervisión manual"]
        c2["Sin plataforma centralizada<br/>de monitoreo y automatización"]
        c3["Soluciones comerciales<br/>inflexibles y costosas"]
    end
    subgraph indirectas["Causas indirectas"]
        i1["Poca capacitación en<br/>agricultura de precisión"]
        i2["Acceso limitado a<br/>IoT asequible"]
        i3["Sin herramientas de<br/>análisis de datos"]
        i4["Sistemas complejos<br/>y poco abiertos"]
    end
    problema --> e1 & e2 & e3 & e4 & e5
    c1 & c2 & c3 --> problema
    i1 --> c1
    i2 --> c2
    i3 --> c2
    i4 --> c3
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class e1,e2,e3,e4,e5 clay
    class problema deep
    class c1,c2,c3 sun
    class i1,i2,i3,i4 muted
```



2.3 Causa y efecto

El diagrama de Ishikawa ordenó las causas con las categorías clásicas de métodos, máquinas, mediciones, mano de obra y medio ambiente.

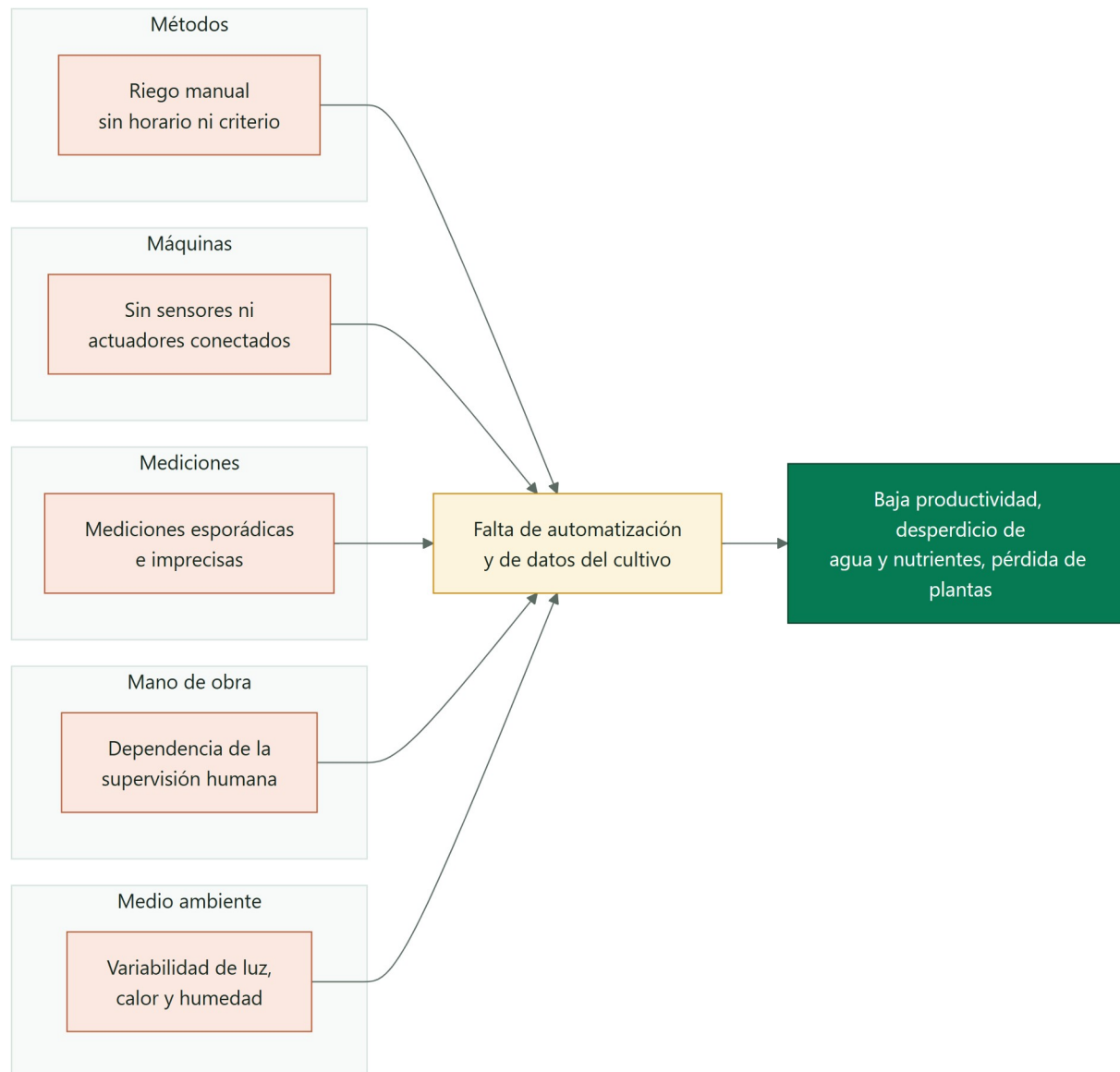


Figura 3 · Causa y efecto del problema



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph metodos["Métodos"]
        m1["Riego manual<br/>sin horario ni criterio"]
    end
    subgraph maquinas["Máquinas"]
        m2["Sin sensores ni<br/>actuadores conectados"]
    end
    subgraph mediciones["Mediciones"]
        m3["Mediciones esporádicas<br/>e imprecisas"]
    end
    subgraph mano["Mano de obra"]
        m4["Dependencia de la<br/>supervisión humana"]
    end
    subgraph ambiente["Medio ambiente"]
        m5["Variabilidad de luz,<br/>calor y humedad"]
    end
    m1 --> raiz["Falta de automatización<br/>y de datos del cultivo"]
    m2 --> raiz
    m3 --> raiz
    m4 --> raiz
    m5 --> raiz
    raiz --> efecto["Baja productividad, desperdicio de<br/>agua y nutrientes, pérdida de plantas"]
    classDef cause fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef root fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef effect fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class m1,m2,m3,m4,m5 cause
    class raiz root
    class efecto effect

```

2.4 Los 5 porqués

PREGUNTA	RESPUESTA DE LA FORMULACIÓN
¿Por qué la gestión es ineficiente?	Porque depende de la supervisión manual y hay poco control automatizado
¿Por qué la supervisión manual desperdicia recursos?	Porque una persona no puede seguir todas las variables todo el tiempo



PREGUNTA	RESPUESTA DE LA FORMULACIÓN
¿Por qué no se automatiza?	Porque las soluciones son costosas, poco accesibles o complejas para pequeños productores
¿Por qué el costo y la rigidez son un obstáculo?	Porque exigen una inversión alta y no se adaptan a cada cultivo
¿Por qué crear SmartPot?	Para ofrecer una herramienta asequible, flexible y educativa que use la simulación para bajar costos

LECTURA CRÍTICA

Las tres técnicas llegan a la misma raíz, falta de automatización y de datos del cultivo, y eso da confianza en el diagnóstico. Tienen tres debilidades que conviene corregir en una próxima formulación:

- El quinto porqué no es una causa sino la justificación de la solución; la cadena causal termina en el cuarto.
- El Ishikawa omite la categoría *materiales* (solución nutritiva, sustrato, calidad del agua), que en hidroponía

explica buena parte de las pérdidas.

NOTA

- Efectos como la inseguridad alimentaria local son reales pero el proyecto no puede medirlos; en el marco lógico

quedan como fin, no como indicador.

SECCIÓN 03

Objetivos y alternativa elegida

3.1 Árbol de objetivos

Al invertir el árbol de problemas, cada causa se convierte en un medio. El diagrama agrega, en cursiva, cómo construyó SmartPot cada medio.

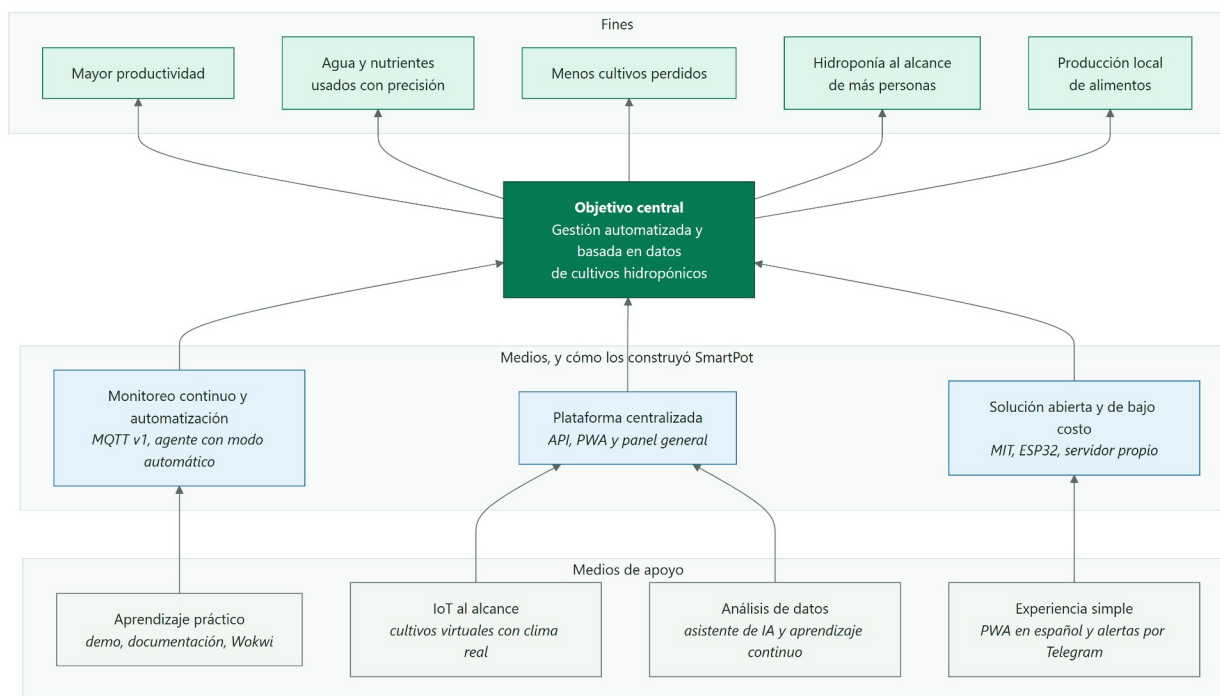


Figura 4 · Árbol de objetivos

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart BT
    subgraph fines["Fines"]
        f1["Mayor productividad"]
        f2["Agua y nutrientes<br/>usados con precisión"]
        f3["Menos cultivos perdidos"]
        f4["Hidroponía al alcance<br/>de más personas"]
        f5["Producción local<br/>de alimentos"]
    end
    objetivo["<b>Objetivo central</b><br/>Gestión automatizada y basada en datos<br/>de cultivos hidropónicos"]
    subgraph medios["Medios, y cómo los construyó SmartPot"]
        m1["Monitoreo continuo y automatización<br/><i>MQTT v1, agente con modo automático</i>"]
        m2["Plataforma centralizada<br/><i>API, PWA y panel general</i>"]
        m3["Solución abierta y de bajo costo<br/><i>MIT, ESP32, servidor propio</i>"]
    end
    subgraph apoyos["Medios de apoyo"]
        a1["Aprendizaje práctico<br/><i>demo, documentación, Wokwi</i>"]
        a2["IoT al alcance<br/><i>cultivos virtuales con clima real</i>"]
        a3["Análisis de datos<br/><i>asistente de IA y aprendizaje continuo</i>"]
        a4["Experiencia simple<br/><i>PWA en español y alertas por Telegram</i>"]
    end
    objetivo --> f1 & f2 & f3 & f4 & f5
    m1 & m2 & m3 --> objetivo
    a1 --> m1
    a2 --> m2
    a3 --> m2
    a4 --> m3
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class f1,f2,f3,f4,f5 leaf
    class objetivo core
    class m1,m2,m3 water
    class a1,a2,a3,a4 muted
```

3.2 Objetivo SMART

CRITERIO	FORMULACIÓN	LECTURA CRÍTICA
Específico	Monitorear y automatizar un jardín hidropónico simulado: humedad, temperatura, luz, pH y nutrientes	Bien acotado; la plataforma lo cumple y agrega humedad del sustrato y atmósfera
Medible	Simulación completa en Wokwi, API, interfaz, base de datos, reportes y riego e iluminación automáticos	Mide productos , no resultados: no dice cuánta agua se ahorra ni cuánto tiempo pasa el cultivo en rango
Alcanzable	Prototipo dentro del curso con un proceso iterativo	Cierto para el software; la parte física quedó fuera por diseño
Relevante	Valor académico y social	Se sostiene: código abierto, demo para instituciones y documentación completa
Temporal	Análisis y diseño de agosto a septiembre; desarrollo y pruebas de octubre a diciembre	La primera plataforma existía antes de esta formulación y siguió creciendo dos años más; el plazo describía un semestre, no el producto

3.3 Cómo evolucionaron los objetivos

DOCUMENTO	OBJETIVO GENERAL	QUÉ CAMBIÓ
Formulación	Automatizar la gestión de cultivos hidropónicos con monitoreo en tiempo real y análisis de datos	Cinco objetivos específicos por etapa: analizar, diseñar, desarrollar, probar y mantener
Proyecto integrador	El mismo	Siete objetivos por beneficio: monitoreo remoto, agua y nutrientes, errores humanos, pérdida de plantas, dependencia manual, espacios reducidos e historial
Acta de constitución	Sistema completo de monitoreo y automatización para lechuga y tomate	Objetivos por dimensión (alcance, tiempo, costo, calidad y relevancia) con indicadores de éxito
Propuesta de investigación	Integrar IoT y modelos de analítica para el control inteligente de la lechuga	Seis objetivos que incluyen hardware físico, parámetros agronómicos y validación experimental

LECTURA CRÍTICA

El objetivo general se mantuvo estable durante dos años, lo que muestra una visión clara. Los específicos, en cambio, se reescribieron en cada curso sin trazar cuáles reemplazaban a cuáles. La plataforma cumple los objetivos de software de los cuatro documentos; quedan abiertos los que exigen hardware y un ciclo de cultivo real (sección 9).

SECCIÓN 04

Entorno e involucrados

4.1 PESTEL

FACTOR	FORMULACIÓN	HOY
Político	Políticas de agricultura urbana y apoyo institucional	Sin cambios: el proyecto sigue dentro del marco académico
Económico	Herramientas educativas gratuitas y alojamiento económico	Se reemplazaron las capas gratuitas por un servidor propio pequeño para no depender de cuotas
Social	Seguridad alimentaria, aprendizaje práctico y comunidades <i>maker</i>	Demo de un solo comando y cultivos virtuales para aprender sin hardware
Tecnológico	Wokwi, Spring Boot, React y ESP32	Se sumaron MQTT con TLS, Docker, FastAPI con scikit-learn y la PWA instalable
Ecológico	Eficiencia hídrica y menos pesticidas	Riego preventivo y bloqueo de nutrientes; el ahorro real solo se podrá medir con un prototipo físico
Legal	Marco académico y software libre	Licencia MIT, datos personales mínimos, borrado de la cuenta y cultivos seudonimizados para el aprendizaje

4.2 Poder e interés

El análisis identificó actores primarios (docente y equipo de desarrollo), secundarios (institución, coordinación, proveedores de nube, horticultores, microproductores y comunidad *maker*) y terciarios (usuarios potenciales, proveedores alternativos y competencia). La matriz de interesados calificó a nueve de ellos de 1 a 10 en poder e interés.

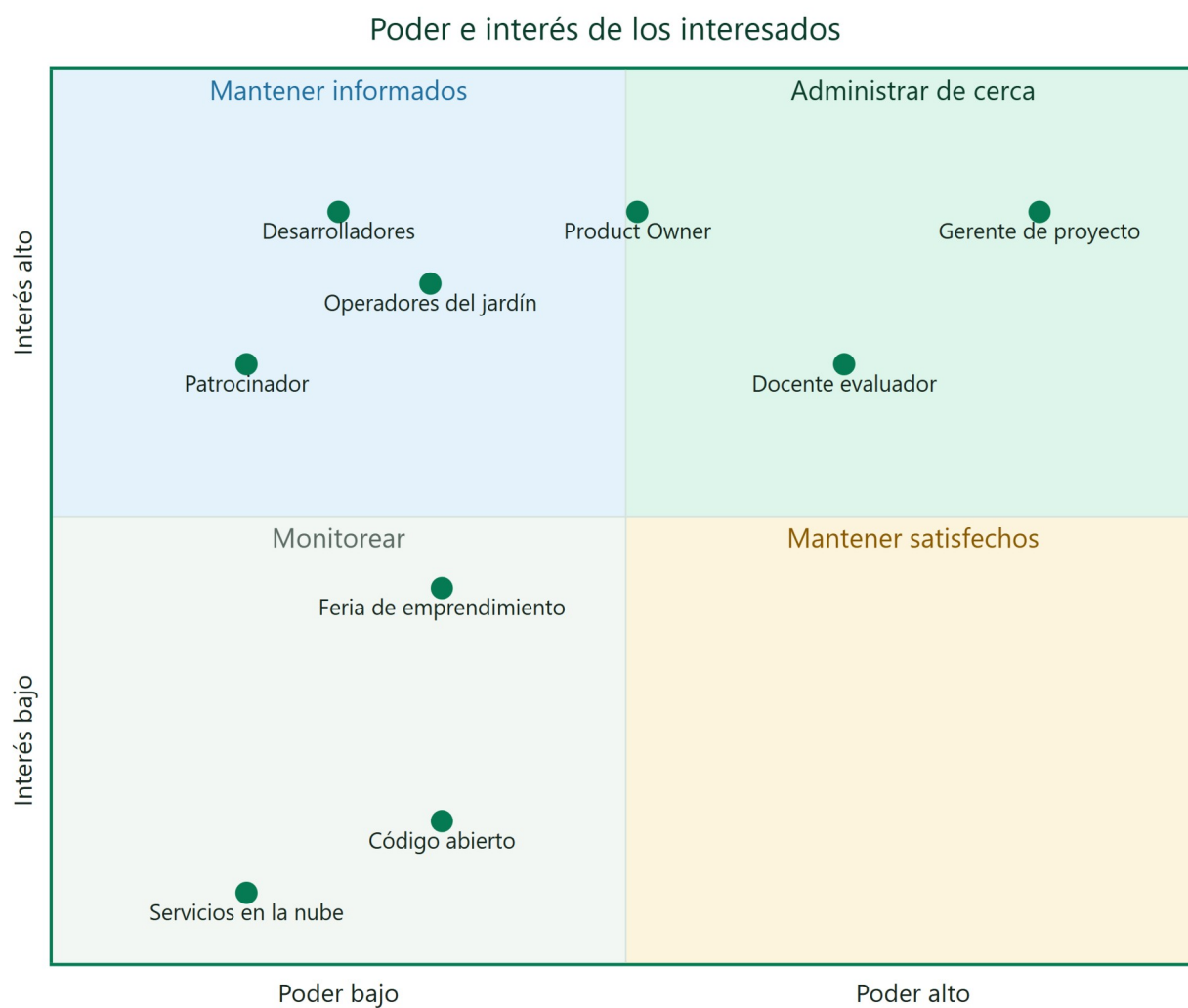


Figura 5 · Poder e interés de los interesados



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4", "quadrant1Fill": "#DDF5EA", "quadrant2Fill": "#E3F2FB", "quadrant3Fill":
"#F2F7F4", "quadrant4Fill": "#FDF4DD", "quadrant1TextFill": "#0B3D2B", "quadrant2TextFill":
"#1F6FA0", "quadrant3TextFill": "#5B6B63", "quadrant4TextFill": "#8A5A00",
"quadrantPointFill": "#067A52", "quadrantPointTextFill": "#17261F", "quadrantXAxisTextFill":
"#17261F", "quadrantYAxisTextFill": "#17261F", "quadrantTitleFill": "#0B3D2B",
"quadrantInternalBorderStrokeFill": "#D5E3DC", "quadrantExternalBorderStrokeFill":
"#067A52"}, "quadrantChart": {"chartWidth": 760, "chartHeight": 640, "pointLabelFontSize":
14, "quadrantLabelFontSize": 17, "titleFontSize": 20, "pointRadius": 7}}}%
quadrantChart
  title Poder e interés de los interesados
  x-axis Poder bajo → Poder alto
  y-axis Interés bajo → Interés alto
  quadrant-1 Administrar de cerca
  quadrant-2 Mantener informados
  quadrant-3 Monitorear
  quadrant-4 Mantener satisfechos
  Gerente de proyecto: [0.86, 0.84]
  Docente evaluador: [0.69, 0.67]
  Product Owner: [0.51, 0.84]
  Desarrolladores: [0.25, 0.84]
  Operadores del jardín: [0.33, 0.76]
  Patrocinador: [0.17, 0.67]
  Feria de emprendimiento: [0.34, 0.42]
  Código abierto: [0.34, 0.16]
  Servicios en la nube: [0.17, 0.08]
```

INTERESADO	PODER	INTERÉS	IMPACTO (PODER × INTERÉS)	MANEJO SEGÚN SU POSICIÓN	MANEJO EN LA MATRIZ ORIGINAL
Gerente de proyecto	10	10	100	Administrar de cerca	Administrar de cerca
Docente evaluador	8	8	64	Administrar de cerca	Administrar de cerca
Product Owner	6	10	60	Administrar de cerca	Administrar de cerca
Desarrolladores	3	10	30	Mantener informados	Mantener satisfechos
Operadores del jardín	3	9	27	Mantener informados	Mantener satisfechos

INTERESADO	PODER	INTERÉS	IMPACTO (PODER × INTERÉS)	MANEJO SEGÚN SU POSICIÓN	MANEJO EN LA MATRIZ ORIGINAL
Institución patrocinadora	2	8	16	Mantener informados	Mantener satisfechos
Feria de emprendimiento	4	5	20	Monitorear (en el límite)	Monitorear
Comunidad de código abierto	4	2	8	Monitorear	Monitorear
Servicios en la nube	2	1	2	Monitorear	Monitorear

LECTURA CRÍTICA

Tres de los nueve interesados tenían un manejo que no correspondía a su posición: con poder bajo e interés alto, la estrategia es *mantener informados*, no *mantener satisfechos*. Además, la columna de conclusiones quedó desplazada una fila y trae frases de otro contexto (ventas, una aplicación móvil, un equipo interno de TI), y la exposición oral ubicó a los proveedores de nube con poder alto mientras la matriz les daba 2. La tabla anterior es la versión corregida y es la que usa el [recorrido del proyecto](#).

SECCIÓN 05

Marco lógico

NIVEL	RESUMEN NARRATIVO	INDICADOR PLANTEADO	EVIDENCIA HOY
Fin	Contribuir a la sostenibilidad y eficiencia de la agricultura urbana mediante la tecnología	Menor consumo de agua; más uso de tecnologías de monitoreo	No medible todavía: requiere un prototipo físico y un ciclo de cultivo comparado
Propósito	Optimizar la gestión de cultivos hidropónicos con monitoreo y control automatizados	95 % de precisión en la gestión de variables; menos tiempo de supervisión manual	El agente actúa solo con el modo automático; «precisión» nunca se definió, así que no hay con qué comparar
Resultado 1	Prototipo completo en simulación	Validación en Wokwi	Cumplido: firmware en Wokwi (cultivo real) y cultivos virtuales que publican por MQTT
Resultado 2	Plataforma web con backend y frontend	API con 80 % de cobertura de pruebas	Plataforma en producción con 260 pruebas; la cobertura no se mide formalmente
Resultado 3	Monitoreo y control en tiempo real	Sistema funcional	Cumplido: telemetría, comandos con confirmación y estado en línea

NIVEL	RESUMEN NARRATIVO	INDICADOR PLANTEADO	EVIDENCIA HOY
Actividades	Análisis y diseño; desarrollo; pruebas y validación; documentación y cierre	Cronograma y repositorios	Cumplidas en ciclos (parte VII)

LECTURA CRÍTICA

El marco lógico tenía la estructura correcta pero indicadores sin línea base, sin meta fechada y, sobre todo, sin supuestos: la columna que dice qué debe ser cierto fuera del proyecto para que funcione. Para el próximo ciclo se propone medir el propósito con un indicador que la plataforma ya puede calcular:

- **Tiempo en rango:** porcentaje de lecturas de cada cultivo dentro del rango ideal de su especie, por semana.
- **Intervenciones manuales:** comandos enviados por la persona frente a los ejecutados por el agente.
- **Supuestos explícitos:** el dispositivo tiene conexión estable, los sensores están calibrados y la persona activa

el modo automático.

SECCIÓN 06

Estudio de mercado

6.1 Demanda y oferta

SEGMENTO	DEMANDA INICIAL ESTIMADA
Horticultores urbanos y domésticos (Medellín, Bogotá, Cali)	500 a 2000 personas
Instituciones educativas y semilleros	50 a 150 instituciones
Makers y comunidad IoT	200 a 500 personas
Microproductores agrotecnológicos	100 a 300 personas

La oferta se describió como un oligopolio parcial con tres tipos de oferentes: sistemas comerciales de hidroponía inteligente (500 a 5000 USD, robustos pero caros), plataformas IoT genéricas (baratas pero sin el dominio hidropónico) y desarrollos caseros (gratuitos pero sin soporte). SmartPot se ubicó en el espacio vacío entre ellos: accesible, educativo y listo para usar.

6.2 Coherencia de las cifras

DATO	PRESENTACIÓN DEL ESTUDIO	DOCUMENTO DEL ESTUDIO	OBSERVACIÓN
Plan Profesional	9 a 12 USD al mes	20 USD al mes	Dos precios para el mismo plan



DATO	PRESENTACIÓN DEL ESTUDIO	DOCUMENTO DEL ESTUDIO	OBSERVACIÓN
Plan Empresarial	50 a 100 USD al mes	100 USD al mes	Coherente en el tope
Costo del MVP	~20 USD al año	81 000 a 120 000 COP al año	El mismo documento lista un servidor de 5 a 8 USD al mes, que ya suma 60 a 96 USD al año
Crecimiento	30 % a 40 % anual	De 800–2900 a 2500–5000 personas	Pasar de un rango al otro implica crecer entre 72 % y 212 %
Demanda	Consumo nacional aparente	Producción + importaciones – exportaciones	Fórmula de bienes físicos; no se calculó para un servicio digital
Aceptación	75 % a 85 %	Probabilidad «alta»	Sin método ni fuente primaria
Valor percibido	—	Ahorro de agua del 90 %	Es un beneficio de la hidroponía, no de SmartPot

LECTURA CRÍTICA

El estudio acertó en lo cualitativo: el nicho entre lo caro y lo casero existe, el canal académico es el natural y el costo de operación es bajo. Falló en lo cuantitativo: los rangos de demanda no salen de un cálculo, el precio cambió entre la presentación y el documento, y ninguna cifra tuvo fuente primaria. Su propia recomendación sigue vigente y es la prioridad comercial: **validar la demanda con un piloto académico y métricas de

adopción** antes de fijar precios.

6.3 Qué quedó del estudio

RECOMENDACIÓN DEL ESTUDIO	ESTADO
Piloto académico con instituciones	Demo de un solo comando en GHCR y Docker Hub, pensada para aulas y semilleros
Métricas de adopción	Pendiente: la plataforma no registra analítica de uso; se propone un conteo agregado y anónimo de cuentas y cultivos activos
Canal directo, comunidad y academia	Portal en smartpot.app, organización pública en GitHub y documentación en español
Modelo <i>freemium</i>	No implementado: hoy todo es gratuito y de código abierto

6.4 Precio: las 5 C

C	LO QUE PLANTEÓ EL ESTUDIO	LECTURA CRÍTICA
Costos	MVP de 81 000 COP al año (servidor de 5 a 8 USD al mes y dominio de unos 7 USD al año; herramientas en planes gratuitos) y, para el prototipo físico, sensores de pH y conductividad de 25 a 40 USD, ESP32 de 20 a 30 USD y bombas de 15 a 50 USD	El servidor por sí solo ya suma de 60 a 96 USD al año, más que la cifra del MVP. Hoy corre en un servidor propio compartido con otros servicios: el costo marginal de SmartPot es bajo, pero no nulo
Clientes	Horticultores dispuestos a pagar de 0 a 50 USD al mes e instituciones de 100 a 300 USD al trimestre; sensibilidad al precio alta en el MVP y moderada en la versión comercial	Ninguna cifra salió de una encuesta: el propio estudio recomienda hacerla. La disposición a pagar es la variable que decide el modelo financiero (sección 8)
Competidores	De 0 USD (proyectos caseros) a 5000 USD (sistemas comerciales); SmartPot entre 0 y 50 USD al mes, como opción de «valor por dinero»	Un rango tan amplio no orienta un precio. El comparable directo son las plataformas IoT genéricas, de 50 a 200 USD al año
Capacidad	Infraestructura escalable en una nube educativa y un equipo reducido	Se cumplió con otra infraestructura: un servidor propio con contenedores, límites por cuenta (20 cultivos, 5 de ellos virtuales) y despliegue automático
Canales	Un portal web (con un dominio de ejemplo), el canal académico y las comunidades <i>maker</i> en GitHub	El portal es smartpot.app; el canal académico y la organización en GitHub existen; la comunidad <i>maker</i> todavía no tiene una acción concreta

La estrategia propuesta tenía dos fases. Primero, un *freemium* académico: gratis con dos cultivos por persona, gratis para instituciones certificadas y una donación voluntaria de 5 a 10 USD. Después, un *freemium* comercial: un plan gratuito con tres cultivos y sin reportes, uno Profesional de 20 USD al mes con reportes y diez cultivos, y uno Empresarial de 100 USD al mes con API pública, análisis avanzado y disponibilidad pactada del 99 %.

LECTURA CRÍTICA

Los límites de los planes ya no coinciden con lo construido: la plataforma permite 20 cultivos por cuenta sin cobrar y ofrece gratis lo que el plan Profesional prometía como pago (resúmenes, análisis de todos los cultivos) y una API documentada. Si algún día se cobra, el valor diferencial no estará en la cantidad de cultivos, sino en el asistente que aprende, los avisos y el acompañamiento.

6.5 Comercialización

FACTOR	PLANTEADO	HOY
Transporte	Distribución digital en la nube, incluida en el hosting	Imágenes en GHCR y Docker Hub y una PWA instalable, sin tiendas de aplicaciones de por medio



FACTOR	PLANTEADO	HOY
Almacenamiento	Base de datos en la nube, de 10 a 20 USD al mes al escalar	MongoDB propio en el servidor, sin costo adicional mientras el volumen sea pequeño
Empaque	Interfaz moderna y documentación	PWA con la paleta SmartPot, documentación en md, DOCX y PDF y una demo de un solo comando
Precio	<i>Freemium</i> de 0 a 50 USD al mes	Gratis y de código abierto
Publicidad	Redes sociales, GitHub y comunidades educativas, de 0 a 100 USD al mes	Perfil de la organización en GitHub y una página de inicio indexable y verificada ante los buscadores; sin campañas
Servicio	Soporte comunitario, tutoriales y demostraciones	Guía de conexión dentro de la aplicación, documentación y plantillas para reportar problemas

El estudio proponía tres modalidades: venta directa en el sitio a horticultores y *makers*, con un margen del 70 % en la versión premium; acceso institucional para universidades y centros educativos, con acuerdos a la medida; y una comunidad de código abierto para desarrolladores. Solo la tercera está activa: el código es abierto y cualquiera puede levantar la demo.

6.6 Fases de penetración

FASE	PLAN	ESTADO
Lanzamiento (primer semestre de 2026)	Instituciones educativas, con demostración gratuita y documentación completa; meta de 50 a 100 instituciones	La demostración y la documentación existen; la adopción institucional no se registró
Crecimiento (segundo semestre de 2026)	Comunidades <i>maker</i> y horticultores urbanos, con redes sociales, tutoriales en video y hackatones; meta de 500 a 1000 personas activas	Sin iniciar: sin métricas de adopción no hay forma de saber si se cumple
Consolidación (2027 en adelante)	<i>Freemium</i> comercial, prototipo físico piloto y alianzas con productores	Depende del prototipo físico y de validar la disposición a pagar

LECTURA CRÍTICA

Las metas se fijaron en cantidades sin un mecanismo para contarlas. Por eso el backlog pone primero las métricas de adopción agregadas y anónimas: sin ellas, ninguna de las tres fases se puede evaluar.

SECCIÓN 07

Estudio técnico

ASPECTO	PLANTEADO EN LA FORMULACIÓN Y EL ACTA	RESUELTO EN LA PLATAFORMA
Localización	Capas gratuitas en la nube	Un servidor virtual propio con Nginx y TLS; dominios <code>smartpot.app</code> , <code>api</code> , <code>mqtt</code> y <code>mail</code>
Tamaño	Un MVP para dos especies	Ocho contenedores; seis especies; una lectura cada 5 s como máximo por cultivo; hasta 5 cultivos virtuales por cuenta; cuatro formas de cultivo
Proceso	ESP32 que envía datos a la API por HTTP	Telemetría y comandos por MQTT v1 con QoS 1, confirmación y última voluntad
Tecnología	Java 17, H2 o MySQL con JPA, React con Vite	Java 21 con Spring Boot 4.1, MongoDB 8, Redis, FastAPI con scikit-learn, React 19 como PWA
Seguridad	JWT, AES y límite de peticiones	Lo anterior más control de dueño, cuenta MQTT por cultivo, contenedores endurecidos, CodeQL y SBOM
Insumos para el prototipo físico	Cotización de sensores y actuadores	Sensores de pH y TDS de 25 a 40 USD cada uno, ESP32 de 20 a 30 USD y bombas de 15 a 50 USD; del orden de 85 a 160 USD sin la estructura hidráulica

LECTURA CRÍTICA

El estudio técnico original era una lista de tecnologías más que un estudio de capacidad. La plataforma respondió las preguntas que faltaban (cuánto aguanta, dónde corre, cómo se asegura) con decisiones medibles: límites por cultivo, validadores de datos, pruebas de humo de cada imagen y un endpoint `/health` por servicio.

7.1 Base temática

Antes de diseñar, el equipo reunió en una wiki lo que había que saber para empezar. Esta es su síntesis, con lo que la construcción confirmó o corrigió.

TEMA	LO QUE SE ESTABLECIÓ	CÓMO QUEDÓ
Jardín hidropónico	Cultivo sin tierra, en agua con nutrientes o en sustratos inertes (arena, grava, fibra de coco). Ahorra agua porque la recircula, acelera el crecimiento, ocupa poco espacio y permite controlar el entorno todo el año	Es la premisa del producto; las cuatro formas que dibuja la PWA (maceta, tubos NFT, torre y balsa) son variantes de este sistema



TEMA	LO QUE SE ESTABLECIÓ	CÓMO QUEDÓ
Partes del jardín	Soporte para las plantas, bomba de agua de 5 V, reservorio, tuberías o canales, sensores del agua y de la luz, y un ESP32 que automatiza riego y luz; la wiki dejaba abierto qué más medir y controlar	Seis variables (temperatura y humedad del aire, luz, pH, TDS y humedad del sustrato) y seis actuadores (bomba, luz ultravioleta, ventilador, humidificador y dos dosificadores)
ESP32	Doble núcleo hasta 240 MHz, WiFi y Bluetooth, hasta 34 pines con ADC, PWM, I2C, SPI y UART, bajo consumo y programable en MicroPython	MicroPython 1.23: el ADC lee los sensores analógicos, I2C maneja la pantalla y el WiFi lleva MQTT con TLS
¿Uno o varios ESP32?	Uno basta para un jardín pequeño o uniforme; varios, uno por zona, dan escala y tolerancia a fallas a cambio de más costo y de coordinarlos	La plataforma resolvió la coordinación: cada cultivo es un dispositivo con su propia cuenta MQTT, así que un jardín con zonas se modela como varios cultivos y el panel general los reúne
Wokwi	Simula el ESP32 con sensores de valores ajustables, pantallas y protocolos, y admite MicroPython	Es la forma de probar el firmware real sin hardware; los cultivos virtuales cubren la demo y el QA, que Wokwi no puede atender porque se ejecuta a mano
Métricas de la lechuga	Aire de 18 a 24 °C, humedad de 50 a 70 %, pH de 5,5 a 6,5, TDS de 800 a 1700 ppm y luz de 20 000 a 40 000 lux	La base de conocimiento usa 15 a 22 °C, 560 a 840 ppm y de 300 a 1400 en la escala del sensor de luz (0 a 2000)
Métricas del tomate	pH de 5,5 a 6,5, conductividad de 2,0 a 3,5 mS/cm, agua de 18 a 22 °C, humedad de 60 a 70 % y aire de 22 a 28 °C de día	La base usa 20 a 28 °C, humedad de 60 a 80 % y 1400 a 2800 ppm de TDS, del orden de la conductividad que pedía la wiki; la temperatura del agua no se mide
Clima de afuera	La wiki guardaba la consulta de un pronóstico de Medellín (temperatura, nubosidad, lluvia, radiación UV, presión y horas de sol) como referencia	La idea volvió dos años después: el clima del lugar ilustra cada cultivo, mueve la simulación y le sirve al asistente para no regar con lluvia

LECTURA CRÍTICA

La wiki mezclaba escalas: lux reales para la lechuga y conductividad para el tomate. La construcción normalizó todo a lo que mide el dispositivo (la luz va de 0 a 2000 porque en Wokwi es un potenciómetro) y a TDS en ppm. Por eso los rangos de la base de conocimiento no coinciden con los de la wiki: no es otra agronomía, es otra escala. Quedó sin medir la temperatura del agua, que la wiki señalaba para el tomate.



SECCIÓN 08

Estudio financiero y evaluación

8.1 Presupuestos

DOCUMENTO	MONTO	COMPOSICIÓN
Acta de constitución	1 000 000 COP	40 % infraestructura, 0 % licencias, 50 % tiempo del equipo, 10 % documentación
Restricción del mismo acta	0 a 50 USD	Solo herramientas abiertas y capas gratuitas
Propuesta de investigación	108 606 000 COP	45 000 000 COP de flujo institucional y 63 606 000 COP de recursos propios; 67,2 millones en equipos

LECTURA CRÍTICA

El acta se contradice: asigna un millón de pesos y a la vez restringe el gasto a 50 USD, unos 200 000 COP. La propuesta de investigación multiplica ese monto por cien y su lista de equipos (cabina para microorganismos, autoclave, incubadora, microscopio) viene de una plantilla de otra disciplina: no corresponde a un sistema hidropónico con ESP32. Un presupuesto de investigación coherente con SmartPot se arma con el prototipo físico (sección 7), el tiempo del equipo y la socialización de resultados.

8.2 Modelo de evaluación

El proyecto nunca tuvo un flujo de caja. Este modelo lo construye con los datos del estudio de mercado y supuestos explícitos, para responder la pregunta que el estudio dejó abierta: ¿qué tendría que pasar para que SmartPot se sostenga?

SUPUESTO	VALOR	ORIGEN
Horizonte	Año 0 de inversión, año 1 académico y gratuito, años 2 y 3 comerciales	Fases de precios del estudio
Inversión	1 000 000 COP	Acta de constitución
Operación del MVP	120 000 COP al año	Tope del estudio de mercado
Operación en escalamiento	20 USD al mes	Tope del estudio de mercado
Tasa de cambio y de descuento	4000 COP por USD; 12 % anual	Supuestos de este análisis
Comisión de pago	5 % del ingreso	Supuesto de este análisis
Costo del equipo	1 300 000 COP al mes: una persona a medio tiempo desde el año 2	Rubro de personal de la propuesta de investigación



ESCENARIO	USUARIOS AÑO 2	CONVERSIÓN A PAGO	PRECIO	SUSCRIPCIONES AÑO 2
Conservador	2500 (+30 % al año 3)	1 %	9 USD	25
Base	3750 (+35 %)	3 %	10,5 USD	112
Optimista	5000 (+40 %)	5 %	12 USD	250

ESCENARIO	VPN SIN COSTO DEL EQUIPO	B/C	VPN CON UNA PERSONA A MEDIO TIEMPO	B/C
Conservador	14,9 millones COP	5,07	-8,6 millones COP	0,68
Base	92,0 millones COP	12,91	68,4 millones COP	3,19
Optimista	242,6 millones COP	16,50	219,1 millones COP	6,59

PRECIO DEL PLAN	PUNTO DE EQUILIBRIO SIN EQUIPO	CON UNA PERSONA A MEDIO TIEMPO
9 USD al mes	2,6 suscripciones	40,6 suscripciones
10,5 USD al mes	2,3 suscripciones	34,8 suscripciones
12 USD al mes	2,0 suscripciones	30,5 suscripciones

LECTURA CRÍTICA

La TIR no aporta aquí: con una inversión de un millón y flujos de decenas de millones da valores de varios cientos por ciento o no existe. Lo que decide la viabilidad es otra cosa:

- La infraestructura casi no pesa: dos o tres suscripciones la pagan.
- El tiempo de las personas sí: sostener a alguien a medio tiempo exige entre 31 y 41 suscripciones, y el escenario

conservador no lo logra.

NOTA

- La variable crítica es la **conversión a pago**, que nadie ha medido. Por eso la prioridad es el piloto con métricas

de adopción, no el modelo de precios.

**NOTA**

La evaluación social es favorable en cualquier escenario: el software es abierto, la demo no cuesta y el conocimiento queda documentado para otras instituciones.

SECCIÓN 09

Propuesta de investigación

La propuesta formuló SmartPot como investigación aplicada de doce meses: *sistema IoT asistido por IA para la automatización, supervisión remota y análisis predictivo de las condiciones ambientales en cultivos hidropónicos de lechuga*, alineada con los ODS 2, 12 y 13. Su objetivo era pasar de un control reactivo, que actúa cuando la variable ya salió de rango, a uno predictivo.

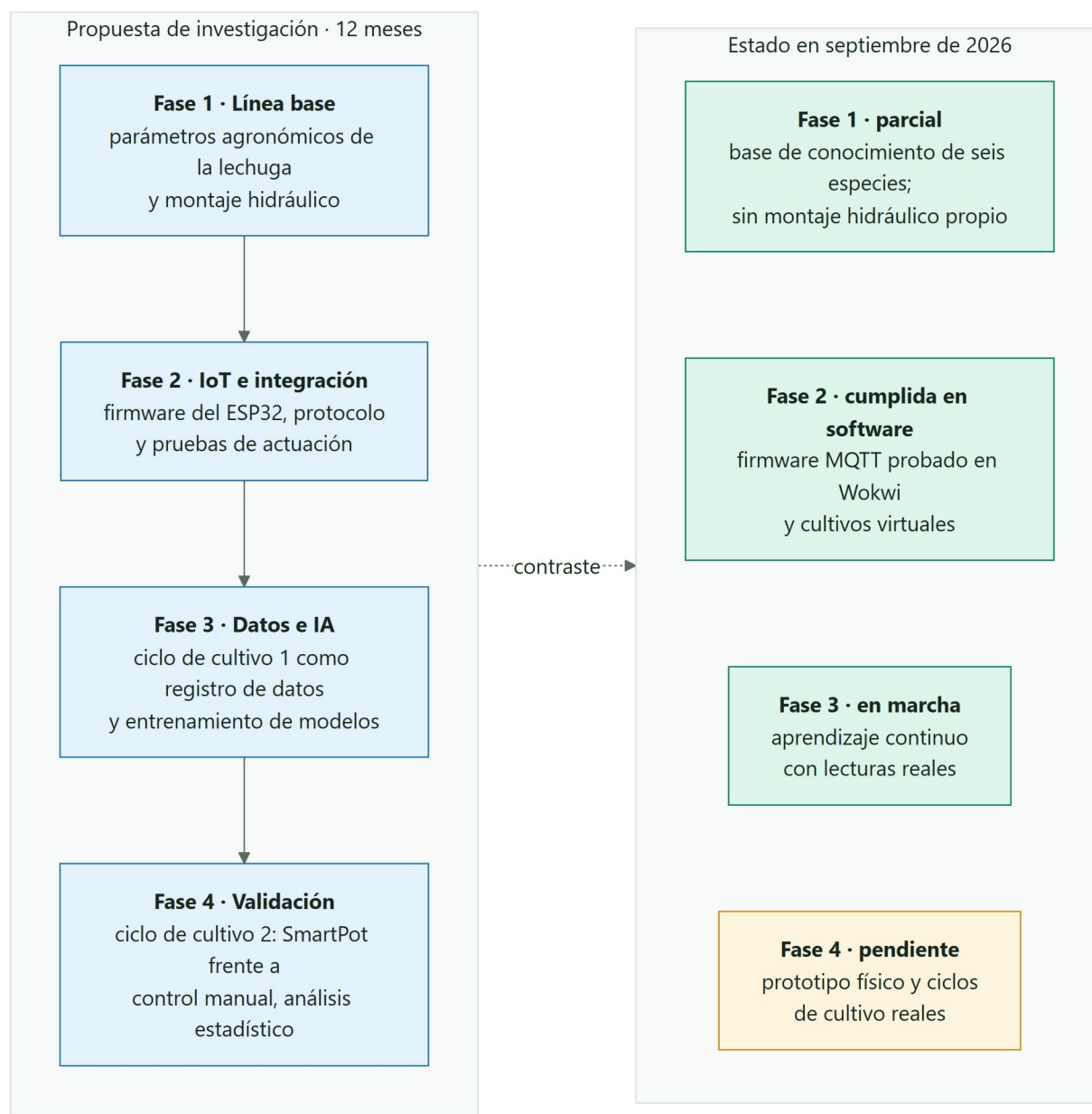


Figura 6 · Fases de la propuesta de investigación y su estado



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph propuesta["Propuesta de investigación · 12 meses"]
        direction TB
        f1["<b>Fase 1 · Línea base</b><br/>parámetros agronómicos de la lechuga<br/>y montaje
hidráulico"]
        f2["<b>Fase 2 · IoT e integración</b><br/>firmware del ESP32, protocolo<br/>y pruebas de
actuación"]
        f3["<b>Fase 3 · Datos e IA</b><br/>ciclo de cultivo 1 como registro de datos<br/>y
entrenamiento de modelos"]
        f4["<b>Fase 4 · Validación</b><br/>ciclo de cultivo 2: SmartPot frente a<br/>control
manual, análisis estadístico"]
        f1 --> f2 --> f3 --> f4
    end
    subgraph hoy["Estado en septiembre de 2026"]
        direction TB
        h1["<b>Fase 1 · parcial</b><br/>base de conocimiento de seis especies;<br/>sin montaje
hidráulico propio"]
        h2["<b>Fase 2 · cumplida en software</b><br/>firmware MQTT probado en Wokwi<br/>y
cultivos virtuales"]
        h3["<b>Fase 3 · en marcha</b><br/>aprendizaje continuo<br/>con lecturas reales"]
        h4["<b>Fase 4 · pendiente</b><br/>prototipo físico y ciclos<br/>de cultivo reales"]
        h1 ~~~ h2 ~~~ h3 ~~~ h4
    end
    propuesta -. "contraste" .-> hoy
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class f1,f2,f3,f4 water
    class h1,h2,h3 leaf
    class h4 sun
```

ELEMENTO	PROPUESTA	LECTURA CRÍTICA
Hipótesis	Implícita: el control asistido por IA mantiene mejor el cultivo que el manual	Conviene escribirla y ligarla a métricas: tiempo en rango, intervenciones, biomasa fresca y litros de agua y solución



ELEMENTO	PROPUESTA	LECTURA CRÍTICA
Modelos	Regresión y redes LSTM	Una LSTM necesita meses de datos etiquetados; la plataforma empezó con técnicas explicables y aprende con lo que llega (sección 28)
Datos	Modo <i>data logger</i> durante un ciclo de 30 a 45 días	Ya existe: la IA guarda las lecturas reales, seudonimizadas, y reentrena por especie
Validación	SmartPot frente a control manual en un segundo ciclo	Pendiente: exige el montaje físico
Productos	Artículo y socialización en un evento	Pendiente; esta documentación es la base del artículo
Bibliografía	Mínimo 20 referencias	Tiene 10; hay que completarla

Referencias de la propuesta, útiles para el trabajo futuro:

REFERENCIA	APORTE
Kurniasari et al. (2025), <i>IOP Conf. Series: Earth and Environmental Science</i>	Hidroponía con ESP32 y monitoreo en tiempo real
Kushawaha et al. (2024), <i>MethodsX</i>	Sistema hidropónico doméstico, pequeño e inteligente
Munaganuri, Yamarthi y Bolem (2025), <i>PeerJ Computer Science</i>	LSTM para agricultura inteligente
Rajaseger et al. (2023), <i>Bioinformation</i>	Tendencias de la hidroponía sostenible
Stevens et al. (2024), <i>PLoS ONE</i>	Manejo de nutrientes de bajo costo en microescala
<i>The Aquaponic Ecosystem Using IoT and IA Solutions</i> (2022), <i>IJWLTT</i>	IoT e IA en acuaponía
Ullah et al. (2019), <i>Intelligent Control and Automation</i>	Monitoreo y control hidropónico de bajo costo
Lowe, Qin y Mao (2022), <i>Water</i>	Aprendizaje automático en tratamiento y monitoreo de agua
Pandi et al. (2024), <i>Earth Science Informatics</i>	Monitoreo ambiental con IA para hidroponía
Mellit et al. (2021), <i>Energies</i>	Monitoreo remoto de invernaderos con IoT y redes profundas

9.1 Marco teórico

La propuesta apoyó el proyecto en tres ejes y lo inscribió en las líneas de investigación de ambiente y sostenibilidad, ciencias agrarias e ingenierías de la institución. Cada eje quedó en la plataforma de una forma concreta:



EJE	LO QUE SOSTIENE LA PROPUESTA	DÓNDE QUEDÓ	LECTURA CRÍTICA
Agricultura de ambiente controlado e hidroponía	La hidroponía es el pilar de la agricultura de ambiente controlado: sin pesticidas en el sustrato y con un consumo de agua de 80 % a 99 % menor que el cultivo tradicional; la lechuga es el cultivo modelo por estar bien caracterizado	Base de conocimiento de seis especies, con la lechuga como referencia, y el lugar del cultivo (bajo techo o al aire libre)	El ahorro de agua es un beneficio de la técnica, no de SmartPot; para atribuirle uno propio hay que medir litros con y sin el sistema (fase 4)
Monitoreo y automatización con IoT	El ESP32 supera a Arduino y al ESP8266 por su WiFi nativo y su doble núcleo; las variables críticas son la temperatura y la humedad del aire, el pH, los TDS y la radiación fotosintéticamente activa	Firmware MicroPython con MQTT y TLS, una cuenta por cultivo y seis variables	La luz se mide en una escala de 0 a 2000, no como radiación fotosintéticamente activa; un sensor de ese tipo queda para el prototipo físico
Analítica y modelos predictivos	Pasar del control reactivo con umbrales fijos a uno predictivo; las redes LSTM serían la arquitectura indicada para series de tiempo agrícolas	Sistema experto y lógica difusa explicables, pronóstico Theil-Sen y aprendizaje continuo con siete modelos comparados contra una línea base	La plataforma llegó a lo predictivo sin LSTM: con pocos datos ganan los modelos simples validados en el tiempo. La LSTM sigue en el backlog para cuando haya meses de lecturas

9.2 Cronograma de la investigación

La propuesta repartía doce meses en nueve actividades. En gris lo que ya está cumplido en software, en azul lo que está en marcha y en ocre lo que espera el montaje físico:

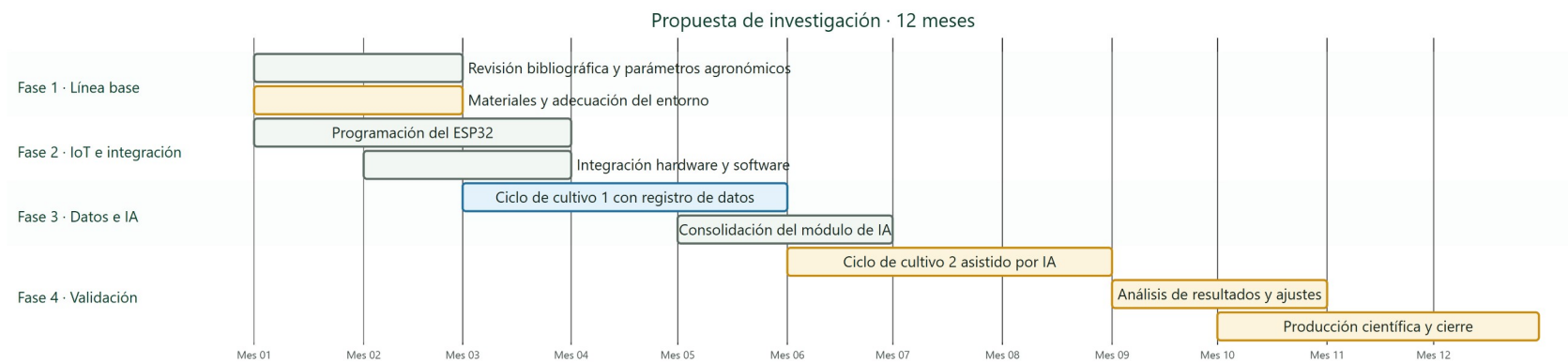


Figura 7 · Cronograma de la propuesta de investigación



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4", "taskBkgColor": "#DDF5EA", "taskBorderColor": "#067A52", "taskTextColor":
"#17261F", "taskTextDarkColor": "#17261F", "taskTextLightColor": "#17261F",
"taskTextOutsideColor": "#17261F", "activeTaskBkgColor": "#E3F2FB", "activeTaskBorderColor":
"#1F6FA0", "critBkgColor": "#FDF4DD", "critBorderColor": "#C98D12", "doneTaskBkgColor":
"#F2F7F4", "doneTaskBorderColor": "#5B6B63", "sectionBkgColor": "#F2F7F4",
"altSectionBkgColor": "#FFFFFF", "sectionBkgColor2": "#EEFAF4", "gridColor": "#D5E3DC",
"todayLineColor": "#D64545", "titleColor": "#0B3D2B"}}, "gantt": {"barHeight": 26, "fontSize":
14, "sectionFontSize": 14, "leftPadding": 230, "useWidth": 1500}}}%
gantt
    title Propuesta de investigación · 12 meses
    dateFormat YYYY-MM-DD
    axisFormat Mes %m
    tickInterval 1month
    section Fase 1 · Línea base
    Revisión bibliográfica y parámetros agronómicos :done, r1, 2027-01-01, 2027-03-01
    Materiales y adecuación del entorno :crit, r2, 2027-01-01, 2027-03-01
    section Fase 2 · IoT e integración
    Programación del ESP32 :done, r3, 2027-01-01, 2027-04-01
    Integración hardware y software :done, r4, 2027-02-01, 2027-04-01
    section Fase 3 · Datos e IA
    Ciclo de cultivo 1 con registro de datos :active, r5, 2027-03-01, 2027-06-01
    Consolidación del módulo de IA :done, r6, 2027-05-01, 2027-07-01
    section Fase 4 · Validación
    Ciclo de cultivo 2 asistido por IA :crit, r7, 2027-06-01, 2027-09-01
    Análisis de resultados y ajustes :crit, r8, 2027-09-01, 2027-11-01
    Producción científica y cierre :crit, r9, 2027-10-01, 2027-12-31
```

ACTIVIDAD	MESES	ESTADO
Revisión bibliográfica y parámetros agronómicos	1-2	Cumplida para seis especies; faltan referencias (10 de las 20 pedidas)
Materiales y adecuación del entorno	1-2	Pendiente: no hay montaje hidráulico propio
Programación del ESP32	1-3	Cumplida: firmware MQTT probado en Wokwi
Integración hardware y software	2-3	Cumplida en software: MQTT v1 con TLS y comandos confirmados, en lugar de HTTP



ACTIVIDAD	MESES	ESTADO
Ciclo de cultivo 1 con registro de datos	3–5	En marcha con lo disponible: la IA guarda las lecturas reales seudonimizadas, aunque no de un ciclo fenológico controlado
Consolidación del módulo de IA	5–6	Cumplida con otra técnica: reentrenamiento continuo con campeón y retador, en lugar de un entrenamiento único fuera de línea
Ciclo de cultivo 2 asistido por IA	6–8	Pendiente: exige el prototipo físico
Análisis de resultados y ajustes	9–10	Pendiente
Producción científica y cierre	10–12	Pendiente; esta documentación es la base del artículo

LECTURA CRÍTICA

El cronograma daba el software por terminado al empezar y concentraba el riesgo en el hardware. Ocurrió lo contrario: el software creció en cinco ciclos y el montaje físico no arrancó. La investigación sigue siendo viable, pero su ruta crítica ya no es el código, sino conseguir el prototipo y un ciclo de cultivo controlado.

PARTE II

Gestión de proyectos TI

SECCIÓN 10

Acta de constitución: lo que fijó y lo que pasó

DIMENSIÓN	OBJETIVO E INDICADOR DEL ACTA	RESULTADO
Alcance	90 % de las funcionalidades críticas entregadas	13 de 17 requisitos funcionales cumplidos (76 %), 3 parciales y 1 replanteado; 16 de 17 con avance (94 %)
Tiempo	Fase 1 de marzo a abril y fase 2 de abril a julio de 2026, con desviación máxima de una semana	Incumplido: de marzo a junio hubo poca construcción y el grueso llegó en septiembre de 2026
Costo	0 % de sobrecosto	Cumplido en licencias (todo es abierto); el servidor propio no estaba previsto en el acta
Calidad	Cobertura de pruebas de al menos 80 %; menos de 3 defectos críticos al cierre	Sin defectos críticos abiertos al cierre del tercer ciclo; la cobertura no se mide formalmente
Relevancia	Prototipo presentado y aprobado	Plataforma en producción, abierta y con demo pública

PREMISA DEL ACTA	QUÉ PASÓ
Equipo de cuatro personas: gerente, dos desarrolladores y un analista	Cerca de tres de cada cuatro commits humanos los hizo una sola persona
Capas gratuitas de nube como infraestructura	Se abandonaron por un servidor propio: las cuotas y la seguridad no se podían controlar
Hardware físico fuera de alcance	Se respetó; la simulación creció hasta tener cultivos virtuales siempre encendidos
Scrum con sprints de dos semanas	La línea base del cronograma usó sprints de tres a cinco días y la construcción se hizo en ciclos (sección 15)

**LECTURA CRÍTICA**

El acta es el documento de gestión más completo del proyecto: objetivos con indicador, riesgos con controles, criterios de aprobación y de cierre, y niveles de autoridad. Su debilidad fue la de muchos proyectos académicos: se escribió para aprobar la planeación y no se volvió a abrir. Dos de sus criterios de cierre se habrían disparado (el alcance creció más del 30 % y el calendario no se cumplió), y ninguno generó una solicitud de cambio.

SECCIÓN 11**Alcance**

11.1 Estructura de desglose del trabajo

La EDT se construyó en cuatro niveles; este es su primer nivel y los paquetes de trabajo.

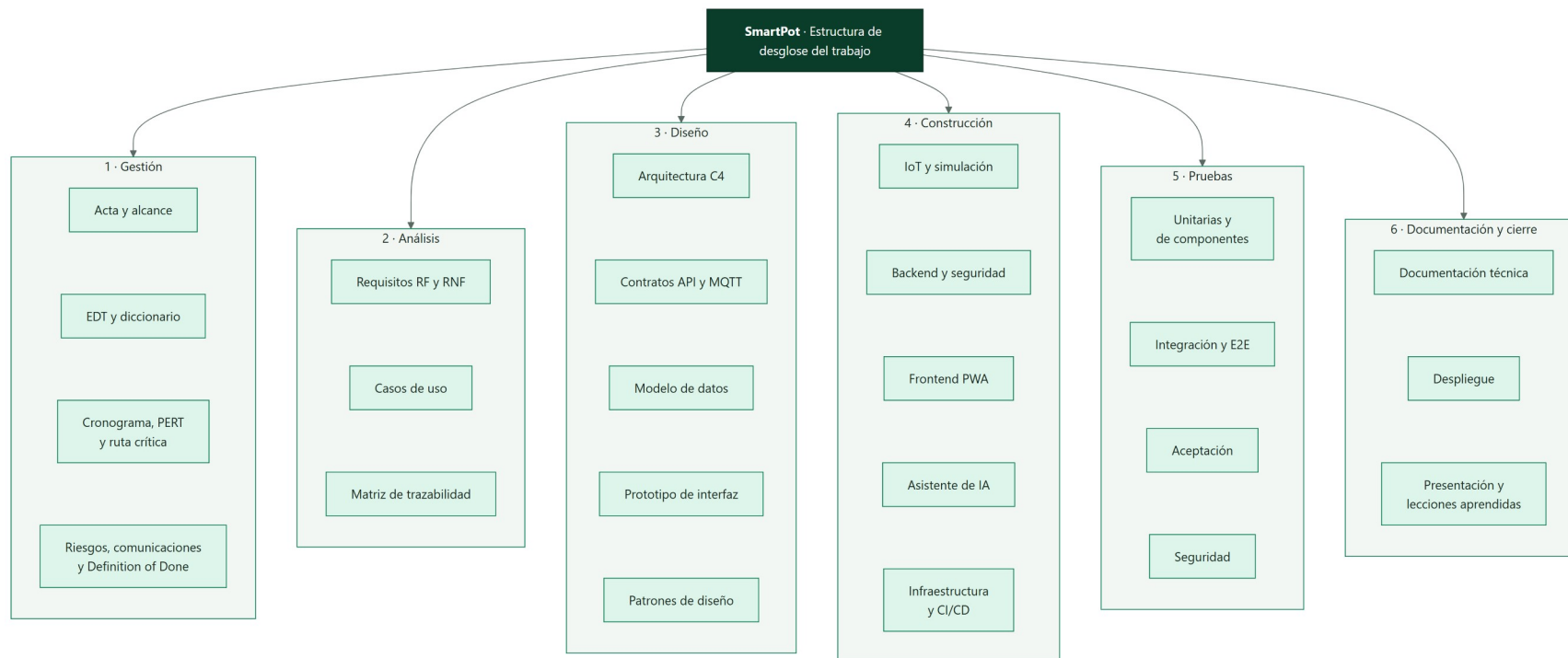


Figura 8 · Estructura de desglose del trabajo



MERMAID



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    root["<b>SmartPot</b> · Estructura de desglose del trabajo"]
    subgraph g["1 · Gestión"]
        direction TB
        g1["Acta y alcance"] ~~~ g2["EDT y diccionario"] ~~~ g3["Cronograma, PERT<br/>y ruta crítica"] ~~~ g4["Riesgos, comunicaciones<br/>y Definition of Done"]
    end
    subgraph a["2 · Análisis"]
        direction TB
        a1["Requisitos RF y RNF"] ~~~ a2["Casos de uso"] ~~~ a3["Matriz de trazabilidad"]
    end
    subgraph d["3 · Diseño"]
        direction TB
        d1["Arquitectura C4"] ~~~ d2["Contratos API y MQTT"] ~~~ d3["Modelo de datos"] ~~~ d4["Prototipo de interfaz"] ~~~ d5["Patrones de diseño"]
    end
    subgraph c["4 · Construcción"]
        direction TB
        c1["IoT y simulación"] ~~~ c2["Backend y seguridad"] ~~~ c3["Frontend PWA"] ~~~ c4["Asistente de IA"] ~~~ c5["Infraestructura<br/>y CI/CD"]
    end
    subgraph p["5 · Pruebas"]
        direction TB
        p1["Unitarias y<br/>de componentes"] ~~~ p2["Integración y E2E"] ~~~ p3["Aceptación"] ~~~ p4["Seguridad"]
    end
    subgraph x["6 · Documentación y cierre"]
        direction TB
        x1["Documentación técnica"] ~~~ x2["Despliegue"] ~~~ x3["Presentación y<br/>lecciones aprendidas"]
    end
    root --> g & a & d & c & p & x
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class root deep
    class g1,g2,g3,g4,a1,a2,a3,d1,d2,d3,d4,d5,c1,c2,c3,c4,c5,p1,p2,p3,p4,x1,x2,x3 leaf
    style g fill:#F2F7F4,stroke:#067A52

```



```
style a fill:#F2F7F4,stroke:#067A52
style d fill:#F2F7F4,stroke:#067A52
style c fill:#F2F7F4,stroke:#067A52
style p fill:#F2F7F4,stroke:#067A52
style x fill:#F2F7F4,stroke:#067A52
```

11.2 Priorización y MVP

La matriz de Eisenhower ordenó el trabajo en cuatro grupos y definió el producto mínimo viable.

CUADRANTE	TAREAS	HOY
Hacer	Arquitectura React, Spring Boot y base de datos; simulación básica en Wokwi; primer endpoint que recibe datos	Cumplido y superado
Planificar	Backend completo, frontend, control automático y pruebas de integración	Cumplido
Delegar	Investigación de mercado, documentación de usuario y diseño visual avanzado	Hechos por el equipo; el diseño visual se resolvió con la paleta SmartPot validada
Eliminar	Aplicación móvil, asistentes de voz y cultivos exóticos	La PWA instalable cubre el móvil; la voz sigue fuera; hay seis especies comunes

FUNCIÓN DEL MVP	ESTADO
Monitoreo en tiempo real de pH, temperatura y conductividad	Cumplido con siete variables
Comunicación completa: ESP32, backend, base de datos y frontend	Cumplido con MQTT v1
Control manual de actuadores desde la web	Cumplido, también en bloque

LECTURA CRÍTICA

Eliminar la aplicación móvil fue una buena decisión que la PWA volvió innecesaria de revisar: una sola base de código se instala en el teléfono y en el escritorio. El MVP estaba bien definido, con tres funciones verificables; es el artefacto de la formulación que mejor resistió el paso del tiempo.

SECCIÓN 12

Cronograma

12.1 Red de precedencias

La tabla de precedencias de la formulación codificó las actividades por fase (AN análisis, DI diseño, DE desarrollo, PR pruebas) con su duración y responsable.

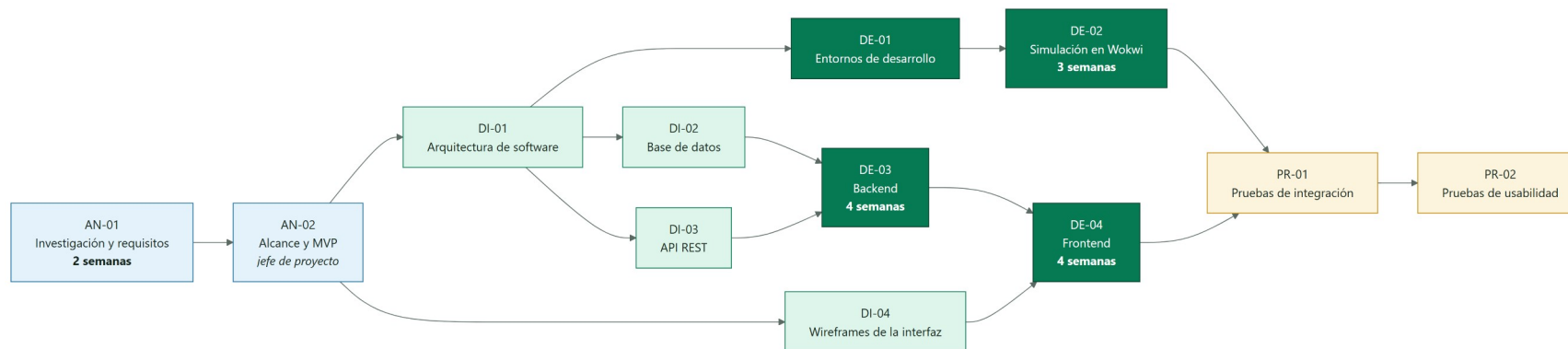


Figura 9 · Red de precedencias de la formulación



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%

```

```
flowchart LR

```

```
  an1["AN-01<br/>Investigación y requisitos<br/><b>2 semanas</b>"] --> an2["AN-02<br/>Alcance
  MVP<br/><i>jefe de proyecto</i>"]

```

```
  an2 --> di1["DI-01<br/>Arquitectura de software"]

```

```
  an2 --> di4["DI-04<br/>Wireframes de la interfaz"]

```

```
  di1 --> di2["DI-02<br/>Base de datos"]

```

```
  di1 --> di3["DI-03<br/>API REST"]

```

```
  di1 --> de1["DE-01<br/>Entornos de desarrollo"]

```

```
  de1 --> de2["DE-02<br/>Simulación en Wokwi<br/><b>3 semanas</b>"]

```

```
  di2 --> de3["DE-03<br/>Backend<br/><b>4 semanas</b>"]

```

```
  di3 --> de3

```

```
  de3 --> de4["DE-04<br/>Frontend<br/><b>4 semanas</b>"]

```

```
  di4 --> de4

```

```
  de2 --> pr1["PR-01<br/>Pruebas de integración"]

```

```
  de4 --> pr1

```

```
  pr1 --> pr2["PR-02<br/>Pruebas de usabilidad"]

```

```
classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F

```

```
classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F

```

```
classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F

```

```
classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F

```

```
classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF

```

```
classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF

```

```
classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F

```

```
class an1,an2 water

```

```
class di1,di2,di3,di4 leaf

```

```
class de1,de2,de3,de4 core

```

```
class pr1,pr2 sun

```

LECTURA CRÍTICA

La red hace depender el frontend (DE-04) de un backend terminado (DE-03), lo que alarga la ruta crítica en cuatro semanas. El acta proponía lo contrario en su plan de riesgos: documentar los contratos de la API desde el inicio y usar *mocks* para desacoplar capas. Con los contratos primero, backend y frontend pueden avanzar en paralelo; así se trabajó después con OpenAPI en [/docs](#) y el contrato MQTT v1.



12.2 Ruta crítica

En la gestión del proyecto el cronograma se llevó a MS Project con estimaciones PERT, una EDT de cuatro niveles y el método de la ruta crítica. El informe de tareas críticas lista 57 actividades sin holgura, desde la redacción del acta hasta el archivo del repositorio.

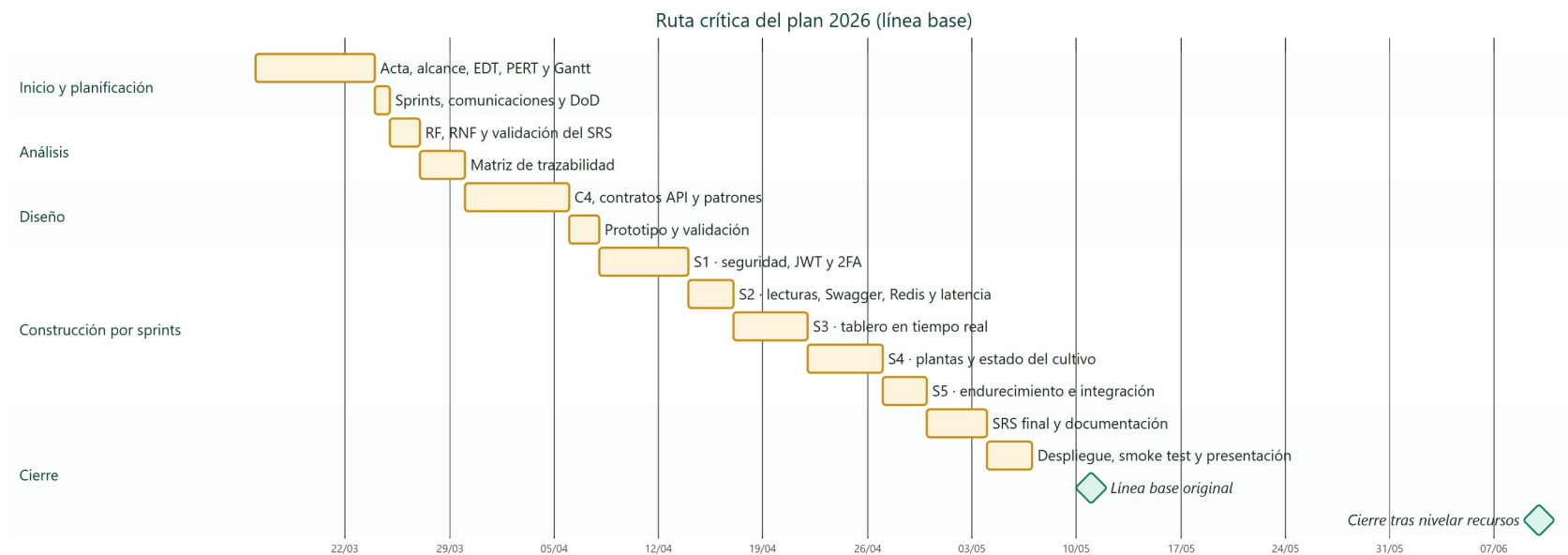


Figura 10 · Ruta crítica de la línea base



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4", "taskBkgColor": "#DDF5EA", "taskBorderColor": "#067A52", "taskTextColor":
"#17261F", "taskTextDarkColor": "#17261F", "taskTextLightColor": "#17261F",
"taskTextOutsideColor": "#17261F", "activeTaskBkgColor": "#E3F2FB", "activeTaskBorderColor":
"#1F6FA0", "critBkgColor": "#FDF4DD", "critBorderColor": "#C98D12", "doneTaskBkgColor":
"#F2F7F4", "doneTaskBorderColor": "#5B6B63", "sectionBkgColor": "#F2F7F4",
"altSectionBkgColor": "#FFFFFF", "sectionBkgColor2": "#EEFAF4", "gridColor": "#D5E3DC",
"todayLineColor": "#D64545", "titleColor": "#0B3D2B"}}, "gantt": {"barHeight": 26, "fontSize":
14, "sectionFontSize": 14, "leftPadding": 230, "useWidth": 1500}}}%
gantt
    title Ruta crítica del plan 2026 (línea base)
    dateFormat YYYY-MM-DD
    axisFormat %d/%m
    section Inicio y planificación
    Acta, alcance, EDT, PERT y Gantt :crit, p1, 2026-03-16, 2026-03-24
    Sprints, comunicaciones y DoD :crit, p2, 2026-03-24, 2026-03-25
    section Análisis
    RF, RNF y validación del SRS :crit, a1, 2026-03-25, 2026-03-27
    Matriz de trazabilidad :crit, a2, 2026-03-27, 2026-03-30
    section Diseño
    C4, contratos API y patrones :crit, d1, 2026-03-30, 2026-04-06
    Prototipo y validación :crit, d2, 2026-04-06, 2026-04-08
    section Construcción por sprints
    S1 · seguridad, JWT y 2FA :crit, s1, 2026-04-08, 2026-04-14
    S2 · lecturas, Swagger, Redis y latencia :crit, s2, 2026-04-14, 2026-04-17
    S3 · tablero en tiempo real :crit, s3, 2026-04-17, 2026-04-22
    S4 · plantas y estado del cultivo :crit, s4, 2026-04-22, 2026-04-27
    S5 · endurecimiento e integración :crit, s5, 2026-04-27, 2026-04-30
    section Cierre
    SRS final y documentación :crit, c1, 2026-04-30, 2026-05-04
    Despliegue, smoke test y presentación :crit, c2, 2026-05-04, 2026-05-07
    Línea base original :milestone, m1, 2026-05-11, 0d
    Cierre tras nivelar recursos :milestone, m2, 2026-06-10, 0d
```

VERSIÓN DEL PLAN	FIN DEL PROYECTO	OBSERVACIÓN
Formulación	Diciembre (un semestre)	Cuatro fases mensuales
Hitos del acta	24 de julio de 2026	Ocho hitos entre el 16 de marzo y el 24 de julio
Línea base en MS Project	11 de mayo de 2026	Tareas de uno a dos días y sprints de tres a cinco días



VERSIÓN DEL PLAN	FIN DEL PROYECTO	OBSERVACIÓN
Plan nivelado	10 de junio de 2026	56,06 días de duración tras eliminar sobreasignaciones

LECTURA CRÍTICA

El mismo alcance tuvo tres fechas de fin distintas. La línea base comprimió el plan del acta: terminaba dos meses y medio antes del último hito, con sprints de menos de una semana aunque el acta fijaba dos, y con actividades como «implementar la autenticación en dos pasos» en un día. Además, el informe de tareas críticas muestra todas las actividades al 0 % y en estado «retrasada»: el avance nunca se registró, así que el cronograma no podía avisar de ningún desvío. La lección práctica fue reemplazar el porcentaje declarado por evidencia automática: pruebas y QA en verde por cambio, y despliegues trazables.

SECCIÓN 13

Recursos, asignación y nivelación

TIPO	RECURSOS	CRITERIO DE COSTO
Personas (4)	Gerente de proyecto, desarrollador <i>full stack</i> 1, desarrollador <i>full stack</i> 2, analista y tester	Tasa estándar por hora
Equipos (4)	Un computador por persona	Costo cero: equipos propios
Plataformas (3)	IoT y simulación; backend y base de datos; frontend y despliegue web	Costo total; solo backend y base de datos con 20 USD de contingencia
Herramientas (2)	Repositorio e integración continua; desarrollo y pruebas	Costo cero: gratuitas

TIPO DE ACTIVIDAD	RECURSO BASE	RECURSOS ADICIONALES
Gestión, alcance, riesgos, comunicaciones y cierre	Gerente con su equipo	Repositorio o herramientas solo si la tarea las usa
Requisitos, trazabilidad, validación y pruebas	Analista con su equipo	Herramientas de pruebas y plataformas involucradas
Diseño técnico, backend, seguridad y API	Desarrollador 1	Backend y base de datos, simulación, repositorio
Frontend, tablero e interfaz	Desarrollador 2	Frontend y despliegue web
Simulación IoT e integración con el ESP32 virtual	Desarrollador 1	Simulación y backend

La nivelación fue manual: se movieron horas entre días cercanos hasta que ninguna persona superara 8 horas por jornada, sin sumar recursos que el equipo no tenía. El cierre pasó al 10 de junio de 2026.



TRABAJO DEL INFORME	PROCESO DEL PMBOK (6.ª EDICIÓN)
Ruta crítica	6.5 Desarrollar el cronograma
Hoja de recursos	9.3 Adquirir los recursos (el informe lo rotuló 9.2)
Asignación a actividades	9.2 Estimar los recursos de las actividades
Eliminación de sobreasignaciones	9.6 Controlar los recursos, 6.6 Controlar el cronograma y 4.6 Control integrado de cambios

LECTURA CRÍTICA

El informe de gestión es el mejor razonado de los documentos: explica qué se hizo, cómo y por qué, y acepta que alargar el plazo era más honesto que sostener cargas imposibles. Queda una deuda: el presupuesto del acta dedica la mitad al tiempo del equipo, pero el informe no reporta la línea base de costo que resultó de las tarifas. Cerrar ese círculo permite comparar el costo planeado con el valor ganado.



SECCIÓN 14

Riesgos

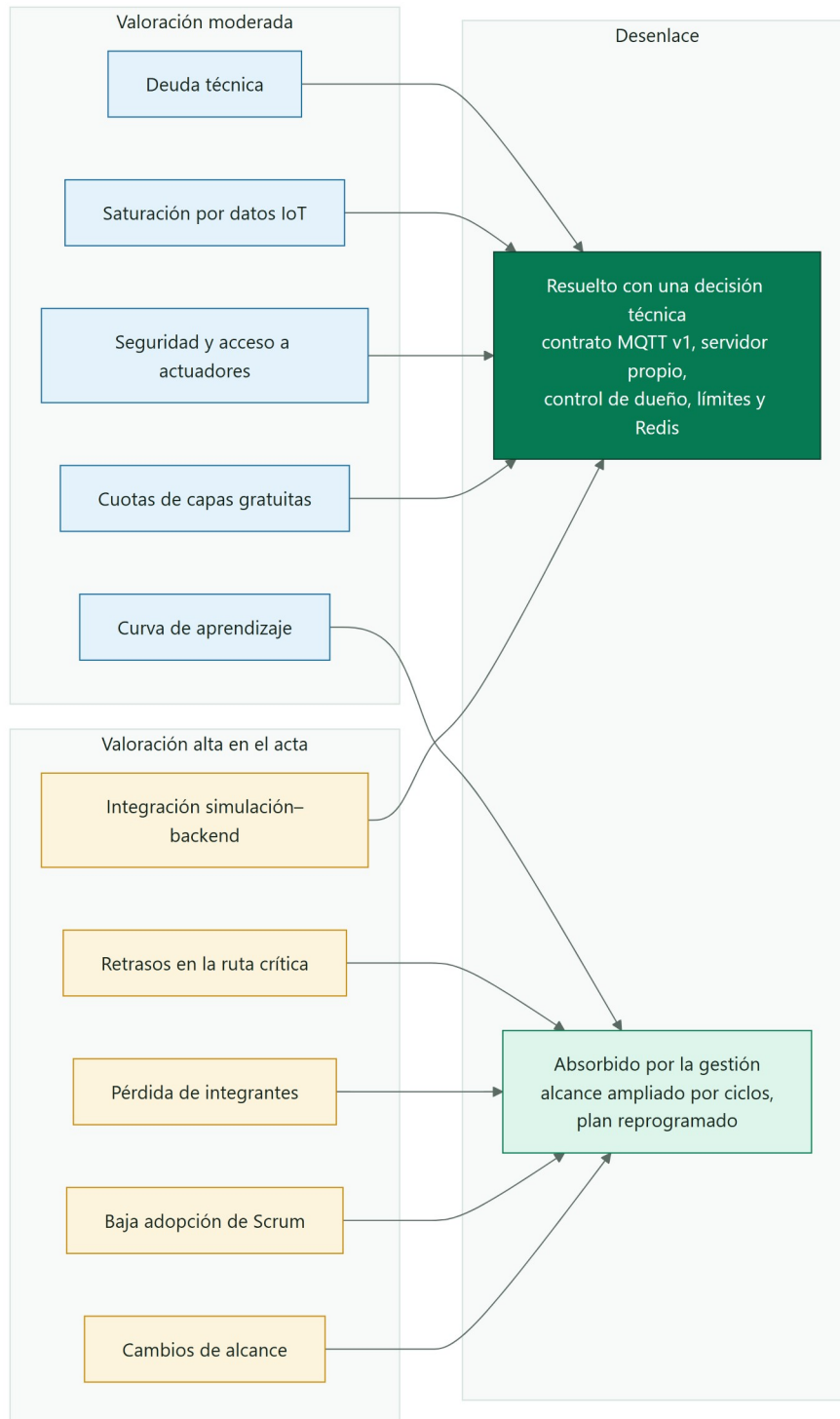


Figura 11 · Riesgos del acta y su desenlace



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph altos["Valoración alta en el acta"]
        r1["Integración simulación-backend"]
        r2["Cambios de alcance"]
        r3["Baja adopción de Scrum"]
        r4["Pérdida de integrantes"]
        r5["Retrasos en la ruta crítica"]
    end
    end
    subgraph moderados["Valoración moderada"]
        r6["Curva de aprendizaje"]
        r7["Deuda técnica"]
        r8["Cuotas de capas gratuitas"]
        r9["Seguridad y acceso a actuadores"]
        r10["Saturación por datos IoT"]
    end
    end
    subgraph desenlace["Desenlace"]
        ok["Resuelto con una decisión técnica<br/>contrato MQTT v1, servidor propio,<br/>control de dueño, límites y Redis"]
        parcial["Absorbido por la gestión<br/>alcance ampliado por ciclos,<br/>plan reprogramado"]
    end
    end
    r1 & r8 & r9 & r10 & r7 → ok
    r2 & r3 & r4 & r5 & r6 → parcial
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class r1,r2,r3,r4,r5 sun
    class r6,r7,r8,r9,r10 water
    class ok core
    class parcial leaf
```

El desenlace de cada riesgo del acta está en el [recorrido del proyecto](#). Aquí interesa lo que el registro **no** vio:



RIESGO NO IDENTIFICADO	CÓMO APARECIÓ	RESPUESTA
Configuración que rompe producción	Tras cambiar el prefijo a /api/v1, las rutas públicas quedaron con el prefijo viejo y todo respondía 401	Pruebas de controladores con la cadena de seguridad real y un E2E que se registra e ingresa en cada cambio
Servicios expuestos	Base de datos, caché, correo y API publicados en todas las interfaces	Redes internas de Docker, UFW cerrado y solo Nginx expuesto
Acceso a cultivos ajenos	Rutas sin control de dueño	Control de dueño en cada servicio y pruebas que lo verifican
Dependencia de una persona	Tres de cada cuatro commits humanos de una sola persona	Documentación completa, demo reproducible y convenciones iguales en todos los repositorios
Datos personales en documentos	Documentos de trabajo con datos de identificación	La documentación publicada no los incluye

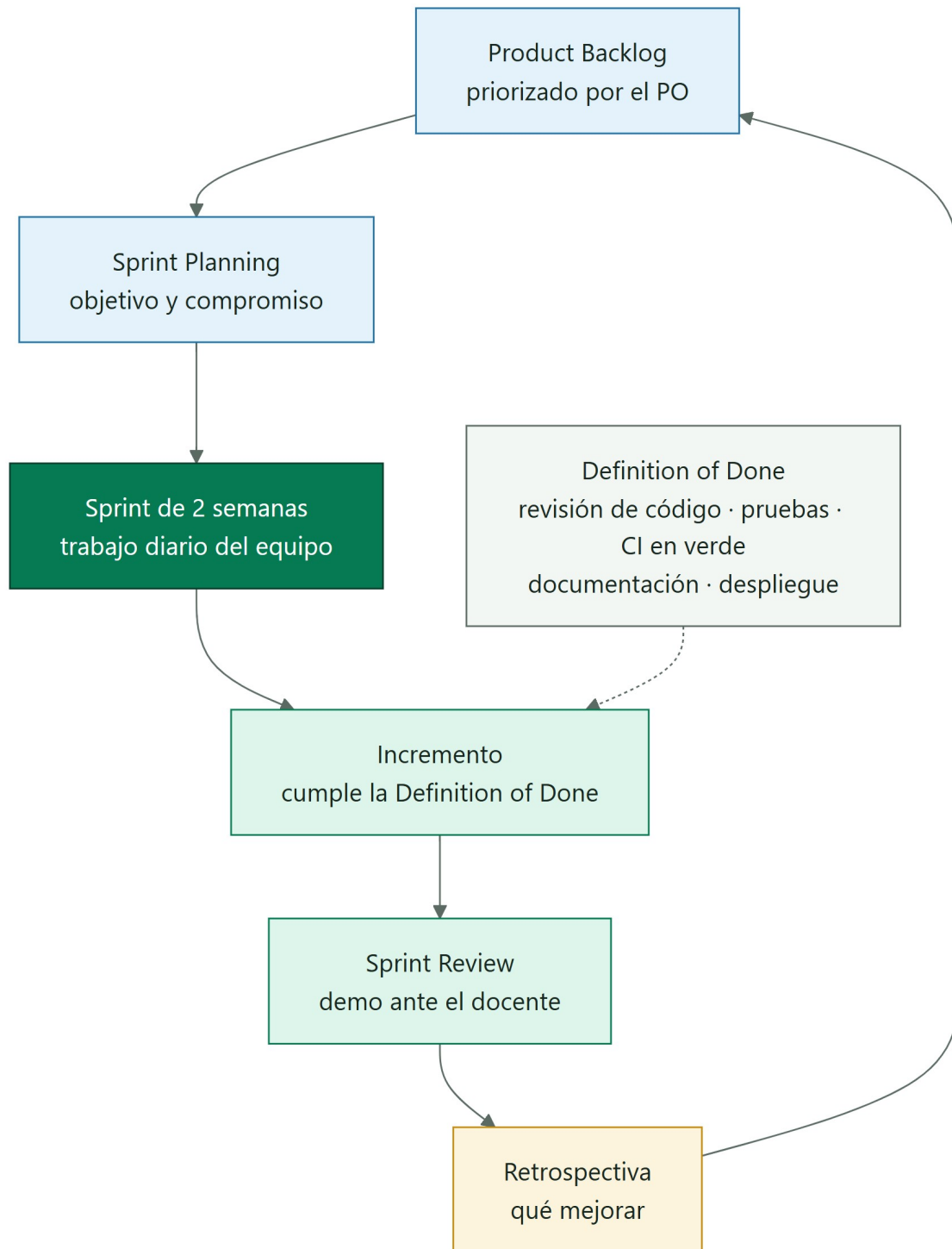
LECTURA CRÍTICA

El registro de riesgos miraba al equipo y al calendario; los riesgos que de verdad se materializaron fueron de configuración y seguridad. Un registro vivo, revisado al final de cada ciclo, los habría anticipado.

SECCIÓN 15

Metodología, comunicaciones y control de cambios

El acta adoptó Scrum con sprints de dos semanas, backlog priorizado por el Product Owner, demos ante el docente y retrospectivas.

**Figura 12** · Ciclo Scrum planeado



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7F8F8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    backlog["Product Backlog<br/>priorizado por el PO"] --> planning["Sprint
Planning<br/>objetivo y compromiso"]
    planning --> sprint["Sprint de 2 semanas<br/>trabajo diario del equipo"]
    sprint --> incremento["Incremento<br/>cumple la Definition of Done"]
    incremento --> review["Sprint Review<br/>demo ante el docente"]
    review --> retro["Retrospectiva<br/>qué mejorar"]
    retro --> backlog
    dod["Definition of Done<br/>revisión de código · pruebas · CI en verde<br/>documentación ·
despliegue"] -.→ incremento
classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
class backlog,planning water
class sprint core
class incremento,review leaf
class retro sun
class dod muted
```

PRÁCTICA	PLANEADA	CÓMO SE TRABAJÓ
Iteración	Cinco sprints de dos semanas	Ciclos de mejora con diagnóstico, backlog y criterios de aceptación (parte VII)
Definition of Done	Revisión de código, cobertura mínima y demo	Código con sus pruebas, CI del repositorio en verde, QA de extremo a extremo en verde, documentación al día y despliegue desde GHCR con <code>/health</code> en UP
Comunicaciones	Canales, frecuencia y demos	Commits pequeños en inglés con un solo propósito, READMEs por repositorio y estos documentos
Control de cambios	Proceso formal y backlog congelado por sprint	Cada ciclo abre un alcance nuevo; el contrato MQTT versionado (v1) protege a los dispositivos de los cambios
Métricas	Velocidad y <i>burndown</i>	Pruebas, estado del QA y despliegues por ciclo (sección 38)

**LECTURA CRÍTICA**

Scrum se planeó completo pero no se ejecutó como tal: con una persona haciendo la mayor parte del trabajo, las ceremonias pierden sentido. Lo que sí funcionó fue su esencia: iteraciones cortas, un incremento desplegable al final de cada una y una definición de terminado automatizada. Para un equipo pequeño, un tablero Kanban con límites de trabajo en curso y la misma definición de terminado es más honesto que un Scrum nominal.

PARTE III

Análisis

SECCIÓN 16

De la necesidad al requisito

Los requisitos tuvieron tres versiones:

VERSIÓN	FECHA	CONTENIDO
Matriz de requisitos	Agosto de 2024	Requisitos no funcionales RNF001 a RNF007 en versión 0.1, todos «sin empezar»; el contrato de la API era el RNF007
Acta de constitución	Marzo de 2026	RF-001 a RF-017 y RNF-001 a RNF-006; el contrato de la API pasó a ser el RF-015
Plataforma	Septiembre de 2026	13 requisitos funcionales cumplidos, 3 parciales y 1 replanteado; 4 no funcionales cumplidos y 2 pendientes

El estado de cada requisito está en el [recorrido del proyecto](#). Esta sección revisa su ****calidad**** con las características de un buen requisito (necesario, sin ambigüedad, verificable y completo).

REQUISITO	PROBLEMA	REESCRITURA VERIFICABLE
RF-002 Calibrar sensores	La descripción quedó cortada a mitad de frase	El sistema permite ajustar la escala de cada sensor de un cultivo y registra quién la cambió
RF-009, RF-012, RF-013 y RF-014	Usan «podrá» o «permitirá» junto a otros con «debe»: no queda claro si son obligatorios	Todos con «debe» y una prioridad MoSCoW explícita
RF-014 Gestión de nutrientes	«De manera efectiva» no se puede probar	El sistema bloquea la dosificación de nutrientes cuando el TDS supera el máximo de la especie
RF-017 Estado general	Tres niveles fijos	Índice de 0 a 100 con cinco niveles y el aporte de cada variable (implementado así)
RNF-001 Doble verificación	Se justificaba con ISO/IEC 25000, que es un marco de calidad y no exige 2FA	La cuenta admite un segundo factor por aplicación de autenticación; motivo: la característica de seguridad de ISO/IEC 25010
RNF-002 Paleta	«Un diseño llamativo que garantice la permanencia» no se puede verificar	Los textos cumplen contraste WCAG AA y las series de las gráficas son distinguibles con daltonismo (lo segundo ya se valida)



REQUISITO	PROBLEMA	REESCRITURA VERIFICABLE
RNF-003 Tiempo real	En la matriz de 2024 hablaba de «canciones guardadas»: un error de copia de otro proyecto	La PWA muestra una lectura nueva en menos de 30 s desde que el dispositivo la publica
RNF-005 Navegación clara	«Entendible para todas las edades» no es medible	Toda función principal está a dos toques o menos desde el panel

LECTURA CRÍTICA

Faltaron requisitos no funcionales que la plataforma tuvo que resolver igual: disponibilidad, rendimiento (la latencia de 500 ms solo aparecía en los criterios de aprobación), privacidad y retención de datos, observabilidad y mantenibilidad. Hoy existen como decisiones de construcción (retención de un año, límites de peticiones, `/health` por servicio, CI en todos los repositorios) y conviene elevarlos a requisitos para poder probarlos.



SECCIÓN 17

Casos de uso

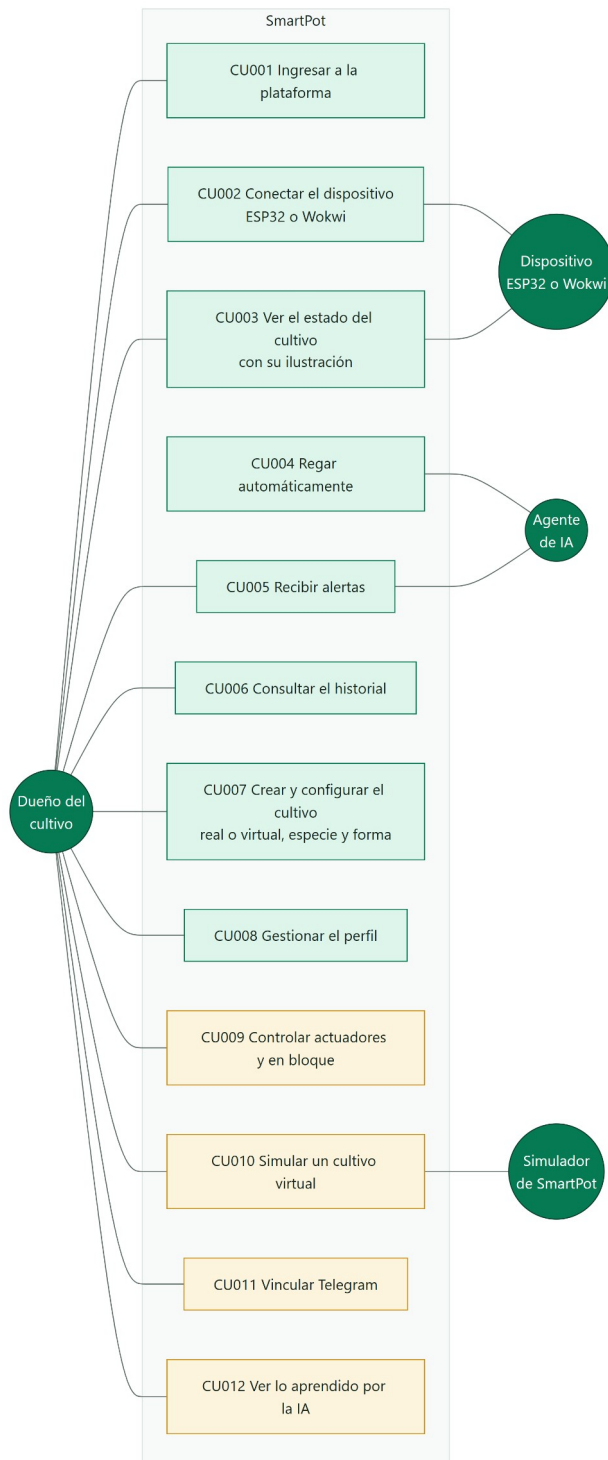


Figura 13 · Casos de uso de SmartPot



MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    persona(("Dueño del cultivo"))
    subgraph sistema["SmartPot"]
        direction TB
        cu1["CU001 Ingresar a la plataforma"]
        cu2["CU002 Conectar el dispositivo ESP32 o Wokwi"]
        cu3["CU003 Ver el estado del cultivo con su ilustración"]
        cu4["CU004 Regar automáticamente"]
        cu5["CU005 Recibir alertas"]
        cu6["CU006 Consultar el historial"]
        cu7["CU007 Crear y configurar el cultivo real o virtual, especie y forma"]
        cu8["CU008 Gestionar el perfil"]
        cu9["CU009 Controlar actuadores y en bloque"]
        cu10["CU010 Simular un cultivo virtual"]
        cu11["CU011 Vincular Telegram"]
        cu12["CU012 Ver lo aprendido por la IA"]
    end
    maceta(("Dispositivo ESP32 o Wokwi"))
    simulador(("Simulador de SmartPot"))
    agente(("Agente de IA"))
    persona --- cu1
    persona --- cu2
    persona --- cu3
    persona --- cu5
    persona --- cu6
    persona --- cu7
    persona --- cu8
    persona --- cu9
    persona --- cu10
    persona --- cu11
    persona --- cu12
    cu2 --- maceta
    cu3 --- maceta
    cu10 --- simulador
    cu4 --- agente
    cu5 --- agente
    classDef actor fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef base fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef nuevo fill:#FDF4DD,stroke:#C98D12,color:#17261F
    class persona,maceta,simulador,agente actor
    class cu1,cu2,cu3,cu4,cu5,cu6,cu7,cu8 base
    class cu9,cu10,cu11,cu12 nuevo
```

Los ocho casos de uso del diseño se conservan y la construcción sumó cuatro: controlar en bloque, simular un cultivo virtual, vincular Telegram y ver lo aprendido por la IA. El cambio de fondo es un actor nuevo: el **agente de IA** aparece como actor secundario de «regar automáticamente» y «recibir alertas». En el diseño la maceta decidía; en la plataforma decide un agente que la persona activa y puede auditar.

17.1 Especificación de los casos del diseño

La hoja de casos de uso del diseño especificó los ocho primeros con actor, prioridad, precondition, flujo normal, flujos alternos, excepciones y postcondición. Cada tabla pone al lado lo que hace hoy la plataforma: las diferencias son el registro de cómo cambió el análisis al construir.

CU001 · Ingresar a la plataforma de control

Basado en RF-001 · actor: usuario · prioridad media.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	La persona está registrada	Igual; el registro pide nombre, apellido, correo y una contraseña con mayúscula, minúscula y número
Flujo normal	1. Abre la página de ingreso. 2. Escribe usuario y contraseña. 3. El sistema valida y la deja entrar	Igual, con el correo como usuario; la sesión es un JWT de 7 días y «Mantener sesión iniciada» la conserva al cerrar el navegador
Flujo alternativo	Recuperar la contraseña y volver a entrar con las credenciales enviadas al correo	Recuperar la contraseña con un enlace de un solo uso que vence en 30 minutos: nunca se envía una contraseña por correo
Excepciones	No recuerda la contraseña; las credenciales son incorrectas	Un solo mensaje («Correo o contraseña incorrectos») que no revela si el correo existe, y un límite de 10 intentos por minuto
Postcondición	Entra a la plataforma	Entra al panel general

CU002 · Calibrar sensores

Basado en RF-002 · actor: usuario · prioridad media.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	Los sensores están instalados físicamente	El cultivo es real y su dispositivo tiene el firmware con la red, la clave y la CA del broker
Flujo normal	1. Entra a «Gestionar sensores». 2. Configura tipos, calibración e intervalo de lectura. 3. Guarda y el sistema se actualiza	No hay calibración remota: la escala de cada sensor vive en el firmware y el intervalo en <code>config.py</code> (30 s). La PWA administra los actuadores, la clave y la guía de conexión



ELEMENTO	DISEÑO ORIGINAL	HOY
Flujo alterno	Una notificación de error de sensor lleva a ajustar los sensores	La regla de falla de sensor avisa y bloquea las acciones del agente; si el sensor no coincide con el clima de afuera, el asistente sugiere revisarlo
Excepciones	La configuración ya estaba así: se avisa y no cambia	Sin equivalente: no hay configuración de sensores que repetir
Postcondición	El sistema calibra los sensores	Pendiente: la calibración remota sigue en el backlog (RF-002 parcial)

CU003 · Visualizar el estado de las plantas

Basado en RF-003 · actor: usuario · prioridad crítica.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	Hay plantas y los sensores funcionan	El cultivo existe y su dispositivo o su simulación está conectado
Flujo normal	1. Entra a la aplicación. 2. La página principal muestra el estado de las plantas	El panel general resume la salud, los cultivos en línea y la última lectura de cada uno; el detalle dibuja el cultivo en su lugar, con sus actuadores
Flujo alterno	Desde otra página vuelve al menú principal y ve el estado	La navegación lateral o inferior lleva al panel o a «Mis cultivos» desde cualquier pantalla
Excepciones	Sensores con fallas: se avisa y, solucionado, se muestra el estado. Sin plantas: se pide agregar una	Dispositivo desconectado: no se ilustra y se explica cómo conectarlo. Posible falla de sensor: alerta y acciones bloqueadas. Sin cultivos: estado vacío con «Nuevo cultivo»
Postcondición	Se muestra el estado de las plantas	Se muestra el estado con su índice de salud de 0 a 100

CU004 · Regar plantas automáticamente

Basado en RF-004 · actor: usuario · prioridad alta.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	Hay plantas y los sensores funcionan	El cultivo tiene bomba de agua y el modo automático activado
Flujo normal	1. El sistema detecta falta de agua. 2. Activa el riego	El agente riega 15 s (30 s si es crítico) cuando el sustrato está seco, y 10 s de forma preventiva si el pronóstico o lo aprendido anticipan que se secará en menos de una hora
Flujo alterno	La persona pulsa «Regar cultivo» en la página principal	El switch de la bomba en Control, con su duración, o «Regar 15 s» en Control general para varios cultivos



ELEMENTO	DISEÑO ORIGINAL	HOY
Excepciones	Sensores con fallas: no se riega y se avisa. Sin plantas: se inhabilita el riego	Posible falla de sensor: ninguna acción. Lluvia sobre un cultivo al aire libre: no riega. Otra orden en curso: espera. Sin confirmación en 2 minutos: el comando vence y se avisa
Postcondición	Las plantas quedan regadas	El comando queda ejecutado con la confirmación del dispositivo y la humedad del sustrato sube en las lecturas siguientes

CU005 · Gestionar alertas

Basado en RF-005 · actor: usuario · prioridad alta.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	Hay plantas y los sensores funcionan	El cultivo envía lecturas o deja de hacerlo
Flujo normal	1. Se detecta una anomalía. 2. Se notifica, por correo electrónico	Alertas en la PWA (campana y página Alertas) y por Telegram: cada cultivo elige qué avisa, al instante o en resúmenes, con un resumen diario y chats compartidos
Flujo alterno	El historial de notificaciones muestra todas y las nuevas en tiempo real	La página Alertas con las no leídas, marcar como leídas y borrar; la PWA consulta cada pocos segundos
Excepciones	Sensores con fallas o sin plantas: se inhabilitan las alertas del estado y se avisa	Posible falla de sensor: se avisa la falla en lugar del estado. Dispositivo desconectado: aviso de conexión. La misma alerta no se repite en una hora
Postcondición	Ve las alertas y puede cerrarlas	Igual, y en Telegram con un botón al cultivo

CU006 · Visualizar datos de control históricos

Basado en RF-006 · actor: usuario · prioridad media.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	Las plantas existen desde hace un tiempo y hay datos guardados	El cultivo tiene lecturas guardadas (se conservan un año)
Flujo normal	1. Entra a «Ver historial». 2. Ve el historial de todas las plantas	Historial de 6 h, 24 h o 7 días por variable, con la banda del rango ideal y exportación CSV, y comparación de una variable entre cultivos en el panel general
Flujo alterno	Notificaciones periódicas de que el historial se actualizó, con un botón para verlo	Sin notificaciones de historial: los resúmenes de Telegram traen la última lectura y lo que pasó en 24 horas
Excepciones	Sin datos: esperar a que haya. Sin plantas: agregar una	Sin lecturas: el gráfico lo indica. Sin cultivos: estado vacío



ELEMENTO	DISEÑO ORIGINAL	HOY
Postcondición	Ve todos los datos del historial	Ve y descarga el historial; la gráfica es la vista por defecto

CU007 · Configurar parámetros óptimos del cultivo

Basado en RF-008 · actor: usuario · prioridad media.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	La persona está registrada y eligió un cultivo	El cultivo existe
Flujo normal	1. Entra a «Configurar cultivo». 2. Ingresar los umbrales de humedad, temperatura, pH, luz y nutrientes	Ajustes del cultivo: nombre, especie, forma y lugar. Los umbrales vienen de la especie y el asistente los aplica solo
Flujo alterno	Sin especificar en el diseño	Cambiar la especie cambia los rangos ideales que usa el asistente
Excepciones	Parámetros mal ingresados	Validaciones en español por campo; el tipo real o virtual no se puede cambiar
Postcondición	Se actualizan los parámetros del cultivo	Se actualizan los datos y el lugar; los umbrales propios por cultivo siguen pendientes (RF-008 replanteado)

CU008 · Gestionar el perfil

Basado en RF-011 · actor: usuario · prioridad media.

ELEMENTO	DISEÑO ORIGINAL	HOY
Precondición	La persona está registrada	Igual
Flujo normal	1. Entra al menú > Perfil. 2. Ve su perfil. 3. Edita y guarda los cambios	Perfil: nombre y apellido, cambio de contraseña, Aplicaciones (Telegram), cierre de sesión y borrado de la cuenta
Flujo alterno	Desde la imagen de perfil del encabezado	Desde la tarjeta con el nombre en la barra lateral o desde el ícono de perfil en el teléfono
Excepciones	Datos mal ingresados al editar	Validaciones en español; la contraseña nueva cumple la política de la API
Postcondición	Ve y edita el perfil	Igual; borrar la cuenta borra sus cultivos, lecturas y vínculos

**LECTURA CRÍTICA**

Los casos se escribieron antes que la tecnología y envejecieron bien en lo funcional y mal en los detalles: enviar credenciales por correo, una pantalla de sensores que el firmware no necesita y umbrales que la persona no sabría elegir. Lo que más cambió fue el actor: en CU004 y CU005 el sistema dejó de ser un conjunto de umbrales para ser un agente que explica por qué actúa. Las dos excepciones que se repetían en casi todos los casos («no hay plantas en el huerto» y «sensores con problemas») se volvieron mecanismos generales: estados vacíos en la PWA y la regla de falla de sensor.

SECCIÓN 18

Trazabilidad

El acta pedía una matriz de trazabilidad de requisito a módulo y a caso de prueba. Esta es la matriz con la plataforma actual:

REQUISITO	CASO DE USO	DÓNDE VIVE	CÓMO SE PRUEBA
RF-001 Acceder	CU001	API <code>security</code> y <code>users</code> ; PWA de ingreso	E2E: registro, ingreso y rechazo sin sesión
RF-002 Calibrar sensores	CU002	Firmware <code>ADCSensor</code> ; PWA del dispositivo	Pruebas del firmware
RF-003 Estado de las plantas	CU003	API <code>overview</code> y <code>crops</code> ; IA de salud	E2E: panel general
RF-004 Riego automático	CU004	API <code>ai</code> (agente de automatización); IA del agente	Pruebas del agente y de sus reglas
RF-005 Alertas	CU005	API <code>notifications</code> y <code>channels</code>	Pruebas de notificaciones y del reenvío a Telegram
RF-006 Historial	CU006	API <code>readings</code> ; PWA de historial y CSV	E2E: series agregadas
RF-007 Registrar lecturas	CU003	API <code>mqtt</code> y <code>readings</code> , con retención de un año	E2E: lectura publicada por MQTT
RF-008 Configurar parámetros	CU007	Base de conocimiento de la IA	Pruebas de los perfiles por especie
RF-009 Control de luz	CU004	Reglas del agente con descanso nocturno	Pruebas de reglas
RF-010 Control manual	CU009	API <code>commands</code> y <code>actuators</code>	E2E: comando con ACK y orden en bloque
RF-011 Perfil	CU008	API <code>users</code> y <code>channels</code>	Pruebas de controladores
RF-012 Reportes	CU006, CU012	Resumen, CSV, análisis de flota y aprendizaje	Pruebas de la IA; E2E de aprendizaje

REQUISITO	CASO DE USO	DÓNDE VIVE	CÓMO SE PRUEBA
RF-013 Gestión de plantas	CU007	API <code>crops</code> y <code>actuators</code>	Pruebas de controladores
RF-014 Nutrientes	CU009	Dosificadores y regla de bloqueo de nutrientes	Pruebas de reglas
RF-015 Conectar con la API	CU002	Contrato MQTT v1 y OpenAPI en <code>/docs</code>	Pruebas de contrato y E2E
RF-016 Gráficos históricos	CU006	PWA: gráfico con banda ideal y comparador	Pruebas de componentes
RF-017 Estado general	CU003	IA: índice difuso de salud	Pruebas de la IA

SECCIÓN 19

Reglas de negocio y base de conocimiento

REGLA	VALOR	POR QUÉ
Frecuencia máxima de lecturas	Una cada 5 s por cultivo	Proteger la base de datos y la API de un dispositivo mal configurado
Retención de lecturas	Un año	Historial útil sin crecimiento indefinido
Evaluación del agente	Cada 5 minutos, con las últimas 48 lecturas	Suficiente para ver tendencias sin saturar la IA
Enfriamiento por actuador	10 minutos	Evitar que el agente encienda y apague en ciclo
Vencimiento de un comando	2 minutos sin confirmación	Avisar a la persona si el dispositivo no responde
Sensor sospechoso	El agente no actúa	Ante un valor imposible es más seguro no regar
Límite de peticiones	300 por minuto; 10 por minuto en autenticación	Frenar abuso y fuerza bruta
Recuperación de contraseña	Enlace de un solo uso que vence en 30 minutos	No enviar credenciales por correo
Cultivos virtuales	Hasta 5 por cuenta; real o virtual no cambia tras crearlo	Mantener acotado el simulador compartido
Descanso nocturno de la luz	De 22:00 a 6:00, hora de Colombia	Respetar el fotoperiodo

La base de conocimiento nació de la investigación temática del diseño (lechuga y tomate) y hoy cubre seis especies: lechuga, tomate, fresa, albahaca, espinaca y pimentón. El detalle de sus rangos está en el [recorrido del proyecto](#) y en la [documentación técnica](#).



PARTE IV

Diseño

SECCIÓN 20

Arquitectura

20.1 Contexto

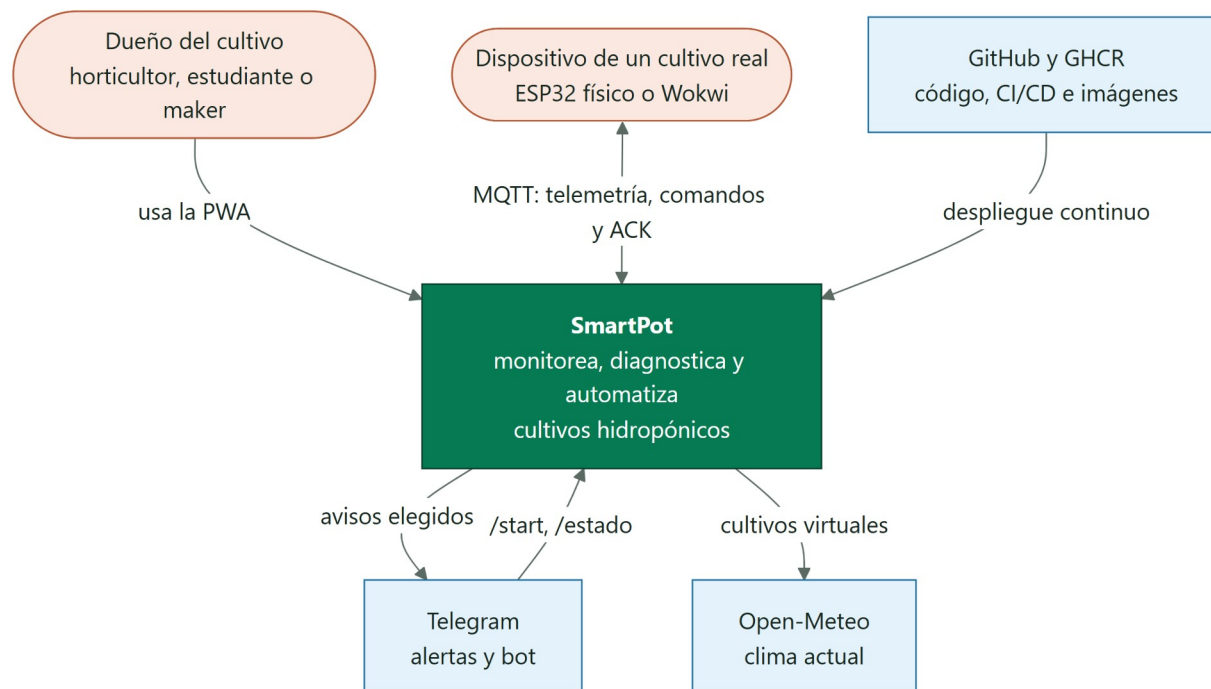


Figura 14 · Contexto del sistema

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    persona(["Dueño del cultivo<br/>horticultor, estudiante o maker"])
    smartpot["<b>SmartPot</b><br/>monitorea, diagnostica y automatiza<br/>cultivos
hidropónicos"]
    maceta(["Dispositivo de un cultivo real<br/>ESP32 físico o Wokwi"])
    telegram["Telegram<br/>alertas y bot"]
    clima["Open-Meteo<br/>clima actual"]
    github["GitHub y GHCR<br/>código, CI/CD e imágenes"]
    persona -->|"usa la PWA"| smartpot
    maceta <-->|"MQTT: telemetría, comandos y ACK"| smartpot
    smartpot -->|"avisos elegidos"| telegram
    telegram -->|"/start, /estado"| smartpot
    smartpot -->|"cultivos virtuales"| clima
    github -->|"despliegue continuo"| smartpot
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class persona,maceta clay
    class smartpot core
    class telegram,clima,github water
```



20.2 Contenedores

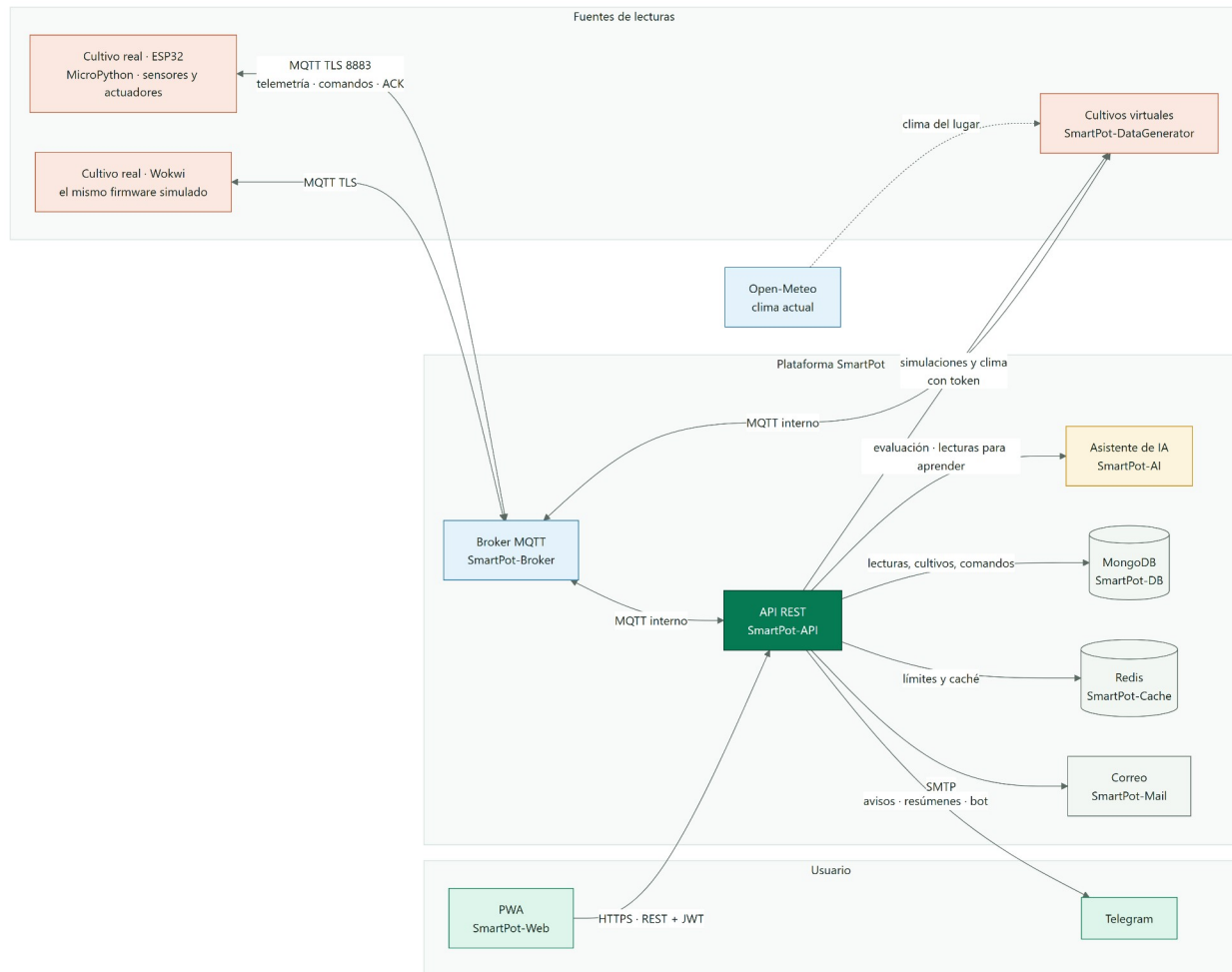


Figura 15 · Arquitectura general de SmartPot

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph campo["Fuentes de lecturas"]
        maceta["Cultivo real · ESP32<br/>MicroPython · sensores y actuadores"]
        wokwi["Cultivo real · Wokwi<br/>el mismo firmware simulado"]
        sim["Cultivos virtuales<br/>SmartPot-DataGenerator"]
    end
    subgraph usuario["Usuario"]
        pwa["PWA<br/>SmartPot-Web"]
        tg["Telegram"]
    end
    subgraph plataforma["Plataforma SmartPot"]
        broker["Broker MQTT<br/>SmartPot-Broker"]
        api["API REST<br/>SmartPot-API"]
        ai["Asistente de IA<br/>SmartPot-AI"]
        db["MongoDB<br/>SmartPot-DB"]
        cache["Redis<br/>SmartPot-Cache"]
        mail["Correo<br/>SmartPot-Mail"]
    end
    clima["Open-Meteo<br/>clima actual"]
    maceta <-->|"MQTT TLS 8883<br/>telemetría · comandos · ACK"| broker
    wokwi <-->|"MQTT TLS"| broker
    sim <-->|"MQTT interno"| broker
    clima -.→|"clima del lugar"| sim
    broker <-->|"MQTT interno"| api
    pwa →|"HTTPS · REST + JWT"| api
    api →|"simulaciones y clima<br/>con token"| sim
    api →|"evaluación · lecturas para aprender"| ai
    api →|"lecturas, cultivos, comandos"| db
    api →|"límites y caché"| cache
    api →|"SMTP"| mail
    api →|"avisos · resúmenes · bot"| tg
    classDef device fill:#F8E9E1,stroke:#B85A38,color:#17261F
    classDef edge fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef data fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    classDef brain fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef app fill:#DDF5EA,stroke:#067A52,color:#17261F
    class maceta,wokwi,sim device
    class broker,clima edge
    class api core
    class db,cache,mail data
    class ai brain
```



```
class pwa,tg app
```

20.3 Decisiones de arquitectura

DECISIÓN	ALTERNATIVA DESCARTADA	POR QUÉ	CONSECUENCIA
MQTT v1 entre dispositivo y plataforma	Consulta periódica por HTTP	Conexión liviana, bidireccional y con estado en línea (última voluntad)	Un broker más que operar y asegurar
MongoDB con validadores	H2 o MySQL con JPA	Lecturas con variables opcionales y consultas por cultivo y fecha	La integridad se protege con <code>\$jsonSchema</code> e índices
IA como servicio aparte en Python	Reglas dentro de la API en Java	El ecosistema de ciencia de datos está en Python	Contrato HTTP interno con token entre API e IA
El agente decide en la plataforma	El dispositivo decide solo	Un dispositivo barato se beneficia de todo lo aprendido	Sin conexión, el dispositivo solo mide y muestra en su pantalla
Simulador como servicio	Solo Wokwi	Demo y QA que no dependen de una pestaña abierta	Un contenedor más, interno y con token
Servidor propio con Docker	Capas gratuitas en la nube	Sin cuotas y con control total de la seguridad	La operación queda en manos del equipo
PWA	Aplicación móvil nativa	Un solo código para teléfono y escritorio	Sin acceso a funciones nativas avanzadas, que no se necesitan
Telegram en la API	Bot dentro del ESP32	Permisos y datos de cada cuenta en un solo lugar	El dispositivo no conoce ningún canal
Despliegue central	Cada repositorio despliega	Un solo lugar con acceso al servidor	Los servicios piden el despliegue y el central lo encola

SECCIÓN 21

Del diseño original al actual

Los diagramas de la fase de diseño describían otra aplicación. Ponerlos lado a lado muestra qué ideas sobrevivieron y cuáles se movieron de lugar.



21.1 Clases

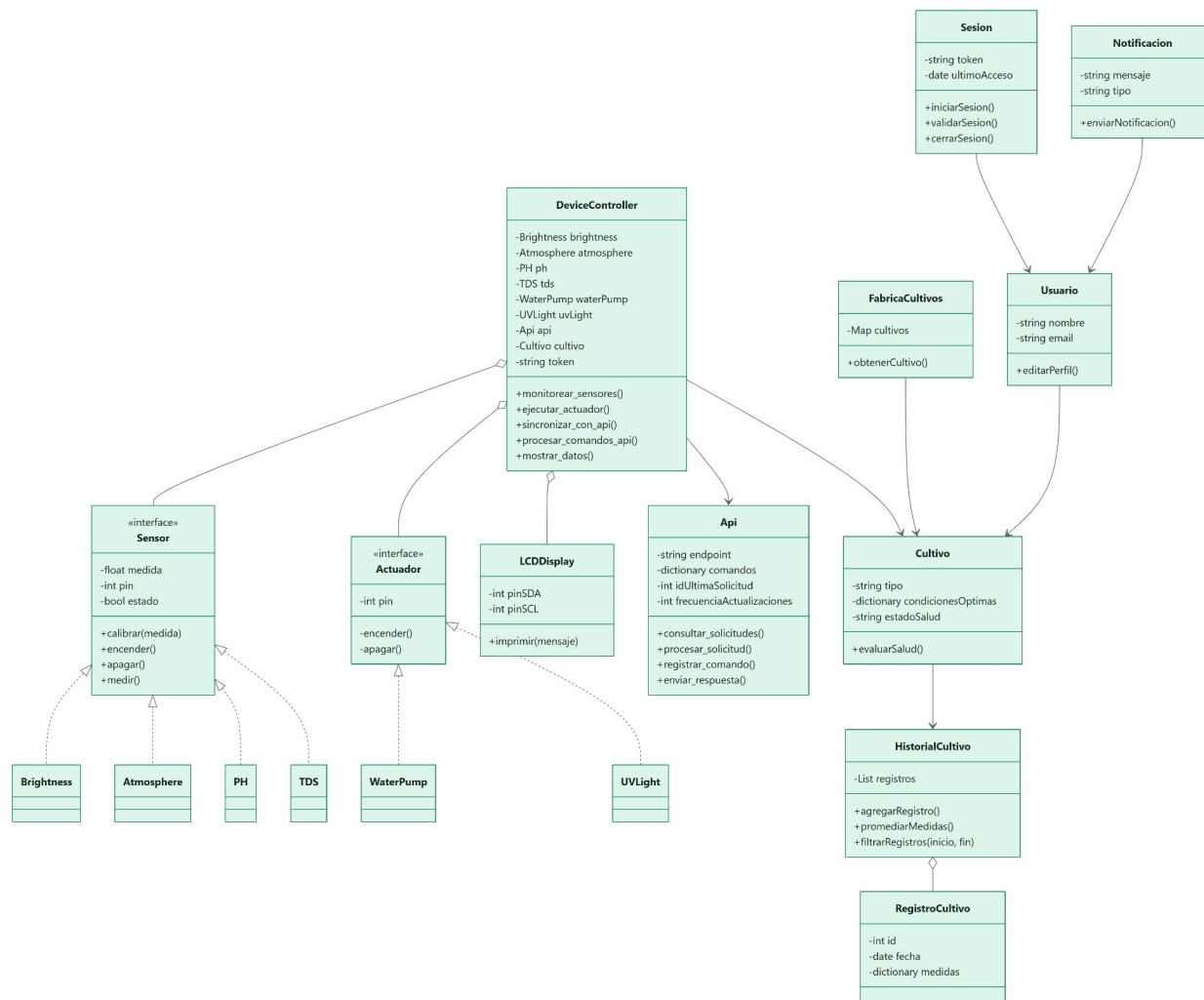


Figura 16 · Clases del diseño original



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
classDiagram
    direction TB
    class Sensor {
        <<interface>>
        -float medida
        -int pin
        -bool estado
        +calibrar(medida)
        +encender()
        +apagar()
        +medir()
    }
    class Actuador {
        <<interface>>
        -int pin
        -encender()
        -apagar()
    }
    class DeviceController {
        -Brightness brightness
        -Atmosphere atmosphere
        -PH ph
        -TDS tds
        -WaterPump waterPump
        -UVLight uvLight
        -Api api
        -Cultivo cultivo
        -string token
        +monitorear_sensores()
        +ejecutar_actuador()
        +sincronizar_con_api()
        +procesar_comandos_api()
        +mostrar_datos()
    }
    class Api {
        -string endpoint
        -dictionary comandos
        -int idUltimaSolicitud
        -int frecuenciaActualizaciones
        +consultar_solicitudes()
        +procesar_solicitud()
        +registrar_comando()
```



```
+enviar_respuesta()
}
class Cultivo {
    -string tipo
    -dictionary condicionesOptimas
    -string estadoSalud
    +evaluarSalud()
}
class HistorialCultivo {
    -List registros
    +agregarRegistro()
    +promediarMedidas()
    +filtrarRegistros(inicio, fin)
}
class RegistroCultivo {
    -int id
    -date fecha
    -dictionary medidas
}
class FabricaCultivos {
    -Map cultivos
    +obtenerCultivo()
}
class Usuario {
    -string nombre
    -string email
    +editarPerfil()
}
class Sesion {
    -string token
    -date ultimoAcceso
    +iniciarSesion()
    +validarSesion()
    +cerrarSesion()
}
class Notificacion {
    -string mensaje
    -string tipo
    +enviarNotificacion()
}
class LCDDisplay {
    -int pinSDA
    -int pinSCL
    +imprimir(mensaje)
}
Sensor <.. Brightness
Sensor <.. Atmosphere
Sensor <.. PH
Sensor <.. TDS
Actuador <.. WaterPump
Actuador <.. UVLight
DeviceController o-- Sensor
DeviceController o-- Actuador
DeviceController o-- LCDDisplay
DeviceController → Api
DeviceController → Cultivo
```



Cultivo → HistorialCultivo
HistorialCultivo o-- RegistroCultivo
FabricaCultivos → Cultivo
Usuario → Cultivo
Sesion → Usuario
Notificacion → Usuario

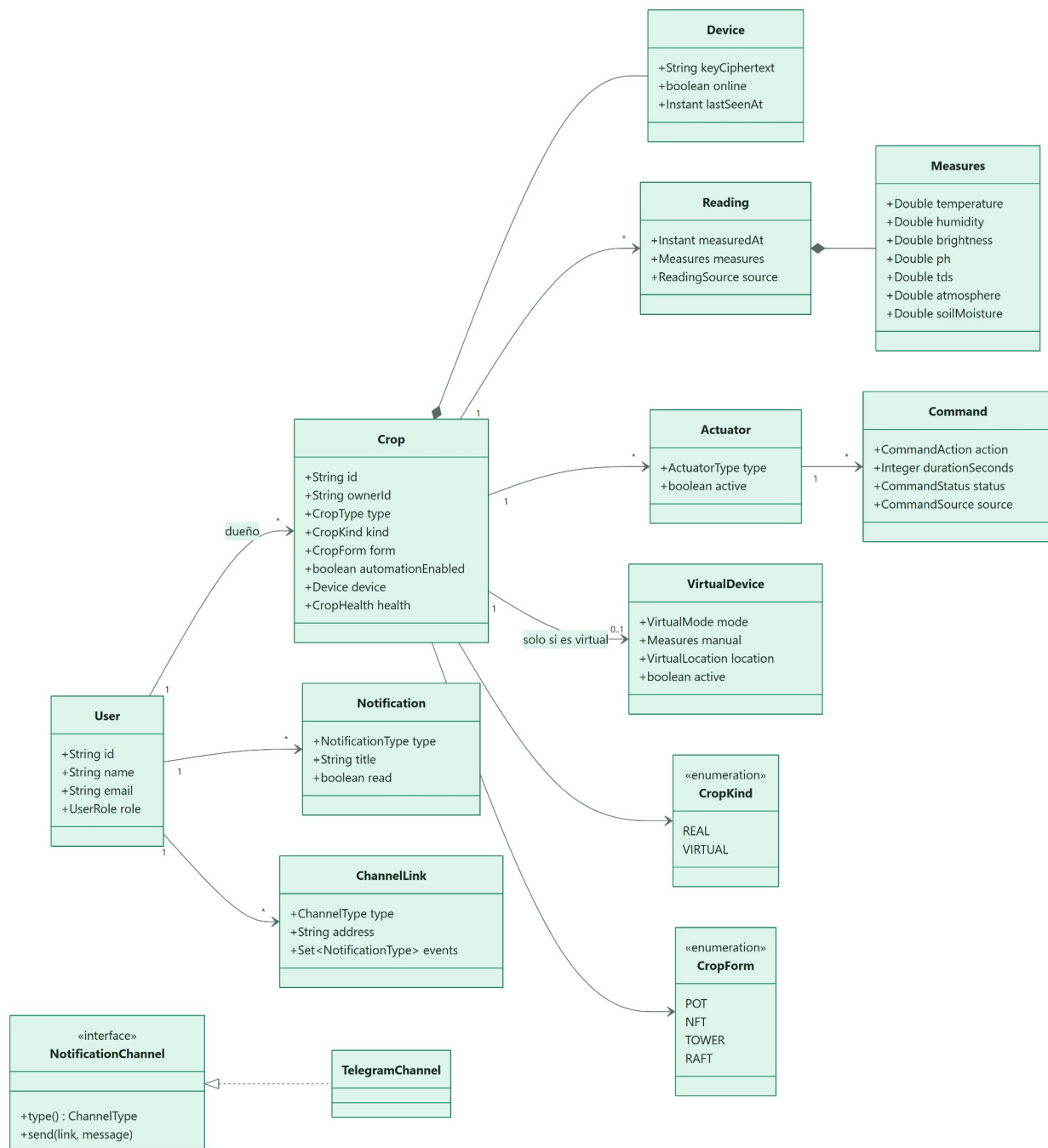


Figura 17 · Clases del dominio actual



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%%
classDiagram
    direction LR
    class User {
        +String id
        +String name
        +String email
        +UserRole role
    }
    class Crop {
        +String id
        +String ownerId
        +CropType type
        +CropKind kind
        +CropForm form
        +boolean automationEnabled
        +Device device
        +CropHealth health
    }
    class Device {
        +String keyCiphertext
        +boolean online
        +Instant lastSeenAt
    }
    class Reading {
        +Instant measuredAt
        +Measures measures
        +ReadingSource source
    }
    class Measures {
        +Double temperature
        +Double humidity
        +Double brightness
        +Double ph
        +Double tds
        +Double atmosphere
        +Double soilMoisture
    }
    class Actuator {
        +ActuatorType type
        +boolean active
    }
```



```
}
class Command {
  +CommandAction action
  +Integer durationSeconds
  +CommandStatus status
  +CommandSource source
}
class Notification {
  +NotificationType type
  +String title
  +boolean read
}
class ChannelLink {
  +ChannelType type
  +String address
  +Set~NotificationType~ events
}
class VirtualDevice {
  +VirtualMode mode
  +Measures manual
  +VirtualLocation location
  +boolean active
}
class CropKind {
  <<enumeration>>
  REAL
  VIRTUAL
}
class CropForm {
  <<enumeration>>
  POT
  NFT
  TOWER
  RAFT
}
class NotificationChannel {
  <<interface>>
  +type() ChannelType
  +send(link, message)
}
class TelegramChannel
User "1" → "*" Crop : dueño
Crop *-- Device
Crop "1" → "*" Reading
Reading *-- Measures
Crop "1" → "*" Actuator
Actuator "1" → "*" Command
User "1" → "*" Notification
User "1" → "*" ChannelLink
Crop "1" → "0..1" VirtualDevice : solo si es virtual
Crop → CropKind
Crop → CropForm
NotificationChannel <|.. TelegramChannel
```



DISEÑO ORIGINAL	ACTUAL	LECTURA
DeviceController con sensores, actuadores, API y cultivo	El dispositivo solo mide y obedece; el dominio vive en la API	La inteligencia se movió a la plataforma
Cultivo.evaluarSalud()	Índice de salud, diagnóstico y pronóstico en la IA	Un método se volvió un servicio
FabricaCultivos con un mapa de cultivos	Perfiles por especie compartidos	La fábrica era, en realidad, datos
Sesion con token y último acceso	JWT sin estado	No hace falta guardar sesiones
Api con consulta de solicitudes	Comandos con id, estados y confirmación	El «identificador de la última solicitud» se volvió una máquina de estados

21.2 Componentes

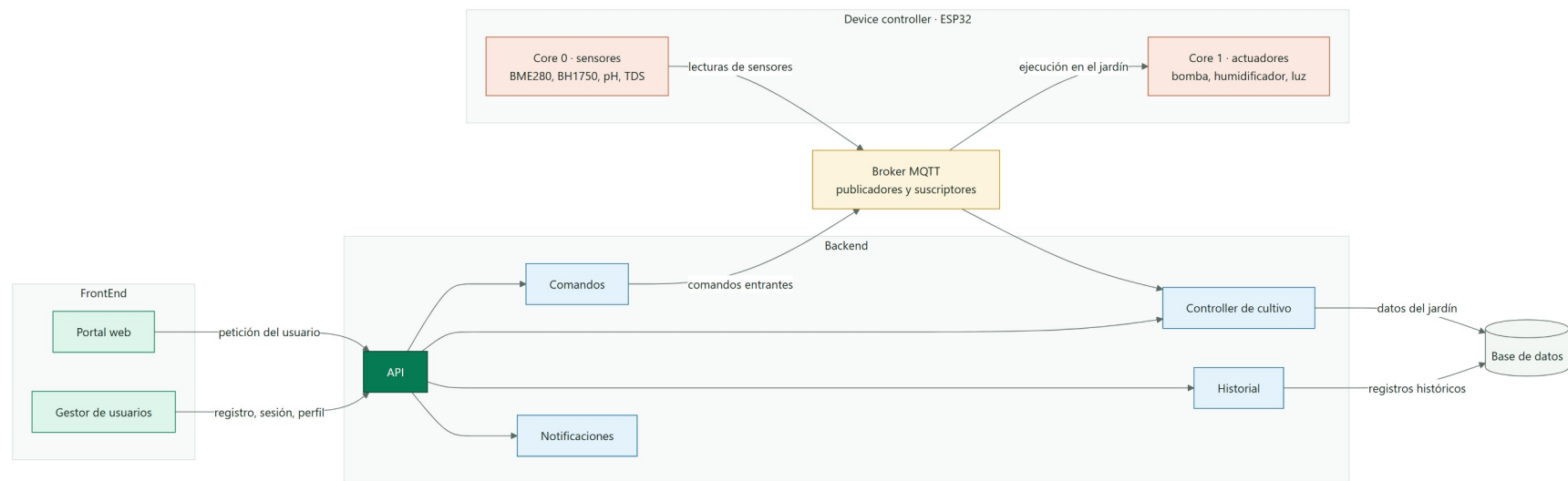


Figura 18 · Componentes del diseño original



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}%%
flowchart LR
    subgraph front["FrontEnd"]
        portal["Portal web"]
        gestor["Gestor de usuarios"]
    end
    subgraph back["Backend"]
        api["API"]
        comandos["Comandos"]
        controller["Controller de cultivo"]
        historial["Historial"]
        notificaciones["Notificaciones"]
    end
    db[("Base de datos")]
    broker["Broker MQTT<br/>publicadores y suscriptores"]
    subgraph esp["Device controller · ESP32"]
        core0["Core 0 · sensores<br/>BME280, BH1750, pH, TDS"]
        core1["Core 1 · actuadores<br/>bomba, humidificador, luz"]
    end
    portal -->|"petición del usuario"| api
    gestor -->|"registro, sesión, perfil"| api
    api --> comandos & controller & historial & notificaciones
    historial -->|"registros históricos"| db
    controller -->|"datos del jardín"| db
    comandos -->|"comandos entrantes"| broker
    broker -->|"ejecución en el jardín"| core1
    core0 -->|"lecturas de sensores"| broker
    broker --> controller
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class portal,gestor leaf
    class api core
    class comandos,controller,historial,notificaciones water
    class db muted
    class broker sun
    class core0,core1 clay
```



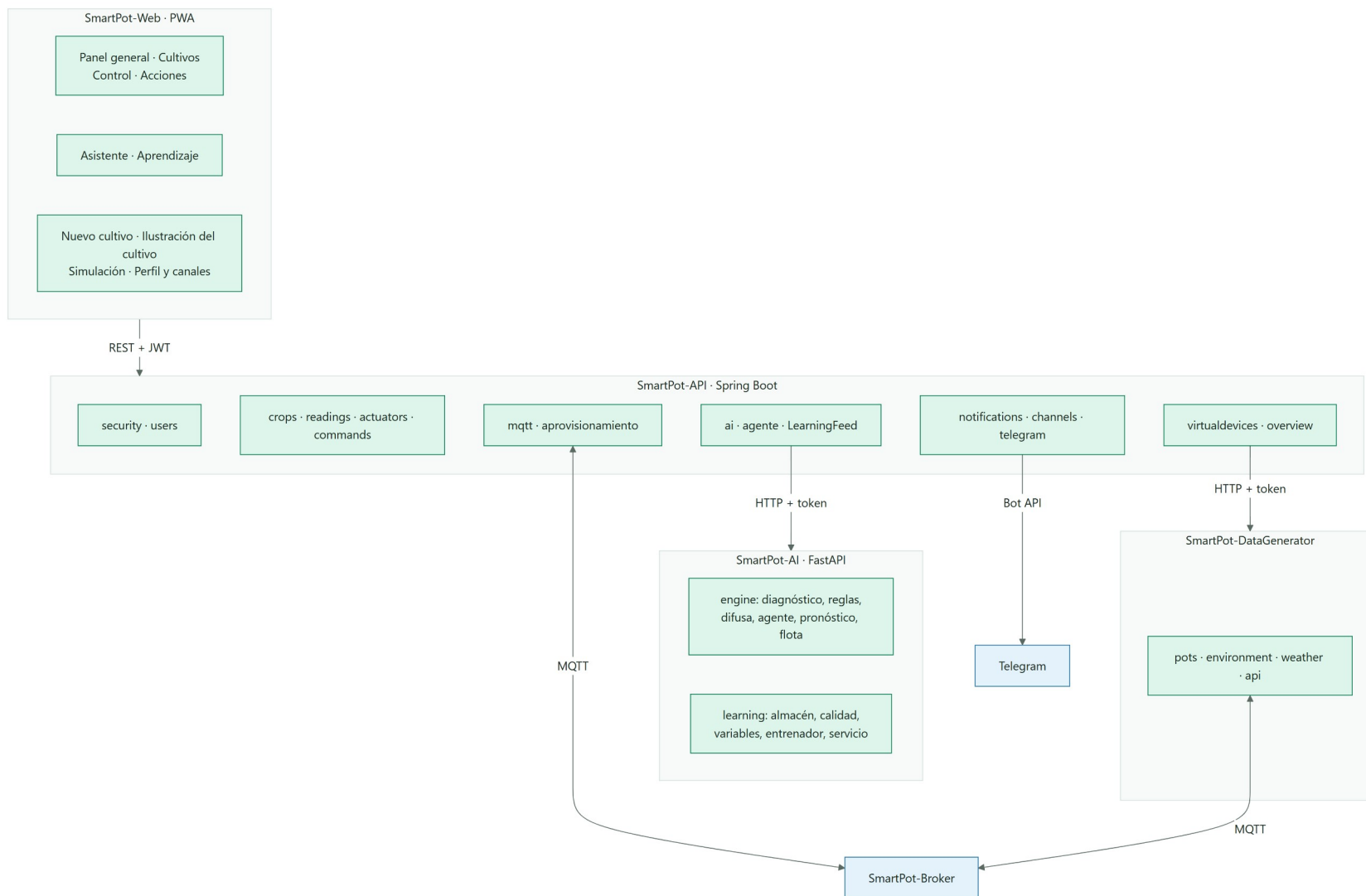


Figura 19 · Componentes y paquetes actuales



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}%%
flowchart TB
    subgraph web["SmartPot-Web · PWA"]
        w1["Panel general · Cultivos<br/>Control · Acciones"]
        w2["Asistente · Aprendizaje"]
        w3["Nuevo cultivo · Ilustración del cultivo<br/>Simulación · Perfil y canales"]
    end
    subgraph api["SmartPot-API · Spring Boot"]
        a1["security · users"]
        a2["crops · readings · actuators · commands"]
        a3["mqtt · aprovisionamiento"]
        a4["ai · agente · LearningFeed"]
        a5["notifications · channels · telegram"]
        a6["virtualdevices · overview"]
    end
    subgraph ai["SmartPot-AI · FastAPI"]
        i1["engine: diagnóstico, reglas,<br/>difusa, agente, pronóstico, flota"]
        i2["learning: almacén, calidad,<br/>variables, entrenador, servicio"]
    end
    subgraph sim["SmartPot-DataGenerator"]
        s1["pots · environment · weather · api"]
    end
    web -->|"REST + JWT"| api
    a3 <-->|"MQTT"| broker["SmartPot-Broker"]
    s1 <-->|"MQTT"| broker
    a4 -->|"HTTP + token"| ai
    a6 -->|"HTTP + token"| sim
    a5 -->|"Bot API"| tg["Telegram"]
    classDef box fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef ext fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    class w1,w2,w3,a1,a2,a3,a4,a5,a6,i1,i2,s1 box
    class broker,tg ext
```

El diseño original ya tenía un broker MQTT y dividía el ESP32 en dos núcleos (sensores y actuadores). Se conservó la idea del broker y se simplificó el firmware a un ciclo principal; lo que cambió fue el backend, que pasó de cuatro componentes genéricos a dominios con la misma organización por capas.

21.3 Comportamiento: enviar un comando

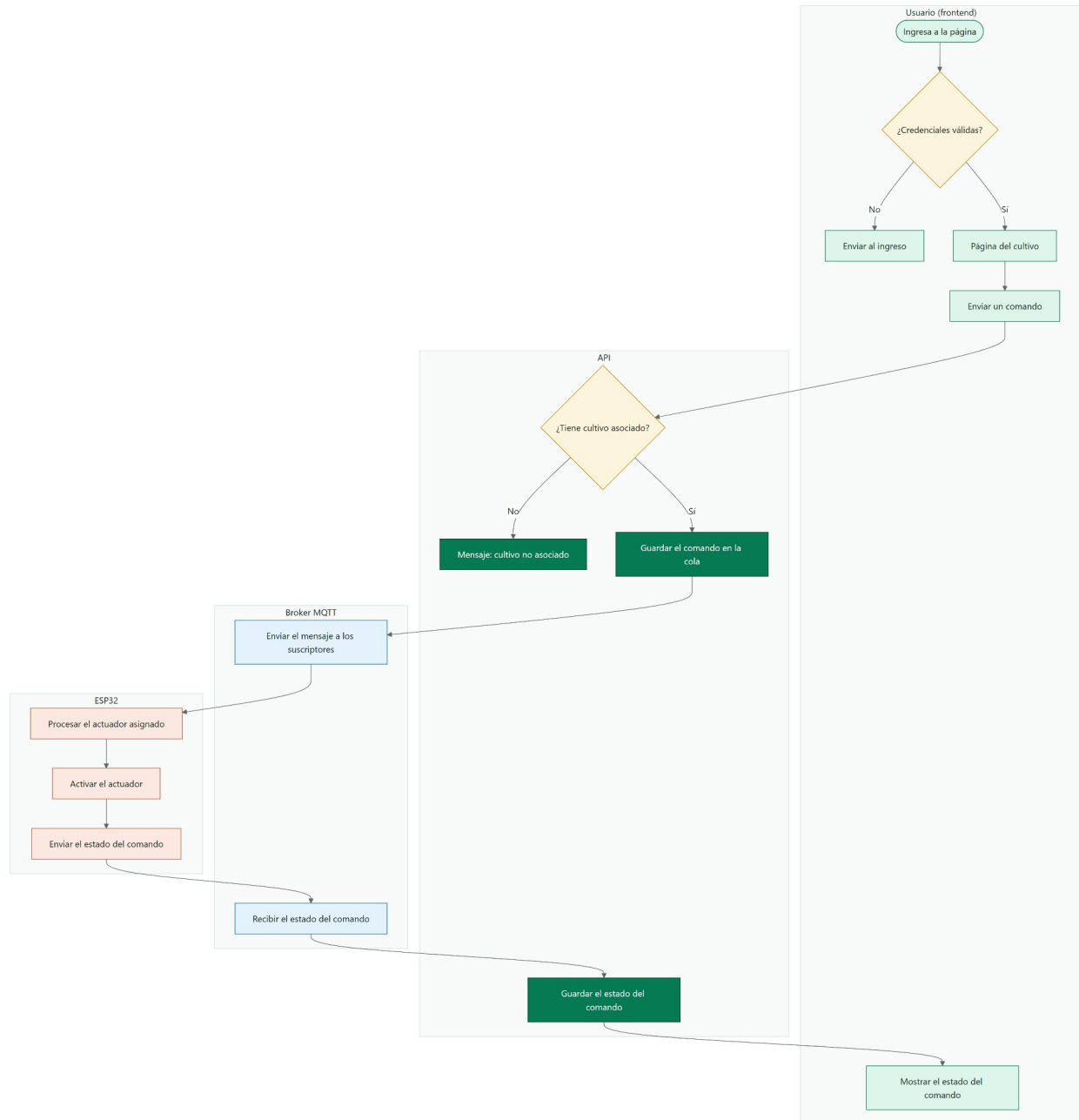
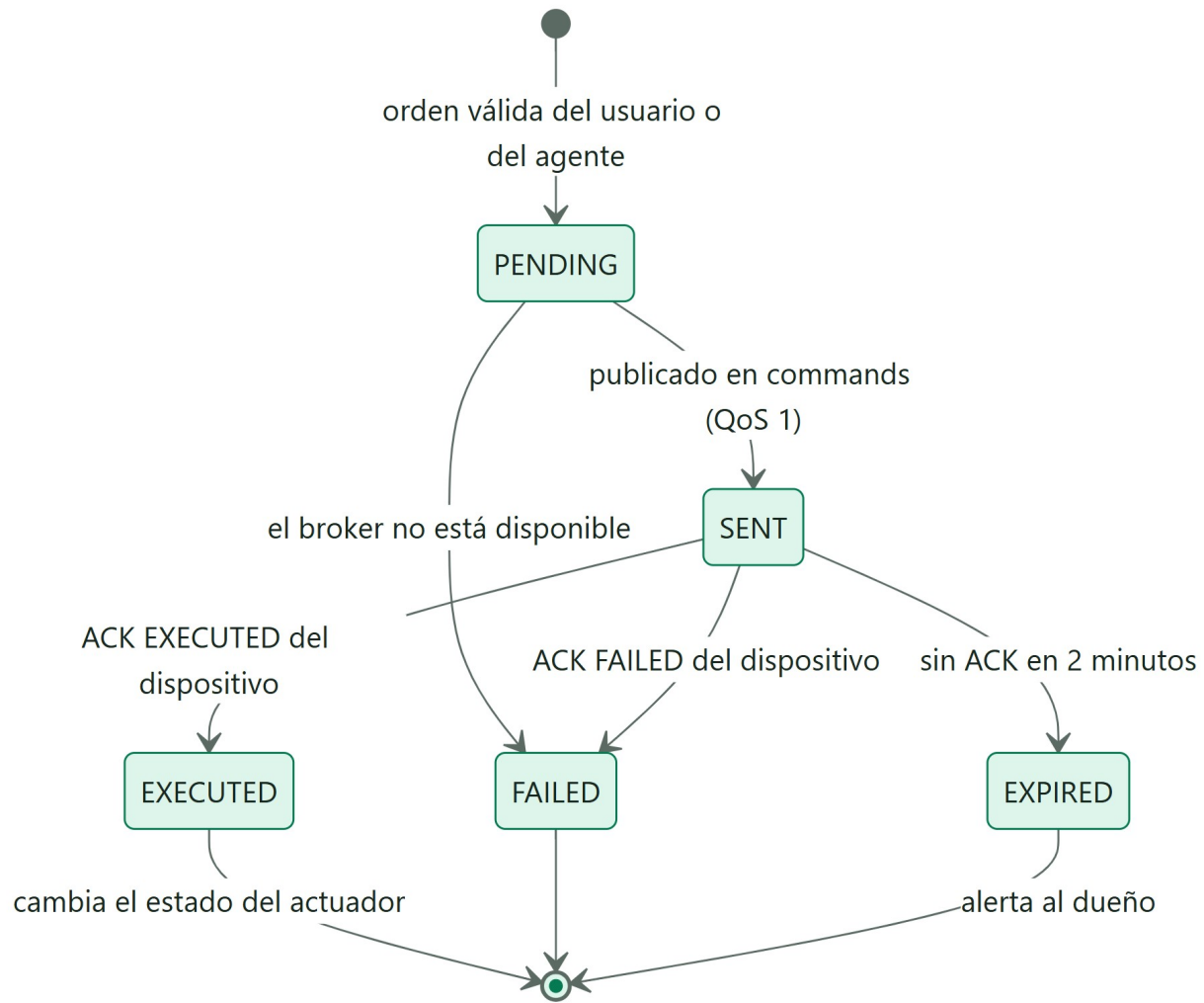


Figura 20 · Actividad del diseño original: enviar un comando



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph u["Usuario (frontend)"]
        u1["Ingresar a la página"] --> u2{"¿Credenciales válidas?"}
        u2 -->|No| u3["Enviar al ingreso"]
        u2 -->|Sí| u4["Página del cultivo"]
        u4 --> u5["Enviar un comando"]
        u9["Mostrar el estado del comando"]
    end
    subgraph a["API"]
        a1{"¿Tiene cultivo asociado?"}
        a1 -->|No| a2["Mensaje: cultivo no asociado"]
        a1 -->|Sí| a3["Guardar el comando en la cola"]
        a4["Guardar el estado del comando"]
    end
    subgraph b["Broker MQTT"]
        b1["Enviar el mensaje a los suscriptores"]
        b2["Recibir el estado del comando"]
    end
    subgraph e["ESP32"]
        e1["Procesar el actuador asignado"] --> e2["Activar el actuador"]
        e2 --> e3["Enviar el estado del comando"]
    end
    u5 --> a1
    a3 --> b1
    b1 --> e1
    e3 --> b2
    b2 --> a4
    a4 --> u9
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class u1,u3,u4,u5,u9 leaf
    class a1,u2 sun
    class a2,a3,a4 core
    class b1,b2 water
    class e1,e2,e3 clay
```

**Figura 21** · Estados de un comando hoy

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
stateDiagram-v2
    [*] --> PENDING: orden válida del usuario o del agente
    PENDING --> SENT: publicado en commands (QoS 1)
    PENDING --> FAILED: el broker no está disponible
    SENT --> EXECUTED: ACK EXECUTED del dispositivo
    SENT --> FAILED: ACK FAILED del dispositivo
    SENT --> EXPIRED: sin ACK en 2 minutos
    EXECUTED --> [*]: cambia el estado del actuador
    FAILED --> [*]
    EXPIRED --> [*]: alerta al dueño
```

El flujo del diseño se mantuvo casi igual: validar sesión, verificar el cultivo, encolar, publicar y esperar el estado. Lo nuevo es que el estado es explícito (PENDING, SENT, EXECUTED, FAILED, EXPIRED) y que un comando sin respuesta vence en dos minutos y avisa.

21.4 Comportamiento: datos históricos y una lectura

La secuencia original se centraba en consultar el historial; la actual muestra el camino completo de una lectura, desde el dispositivo hasta la acción automática, porque ese es hoy el flujo crítico del sistema.

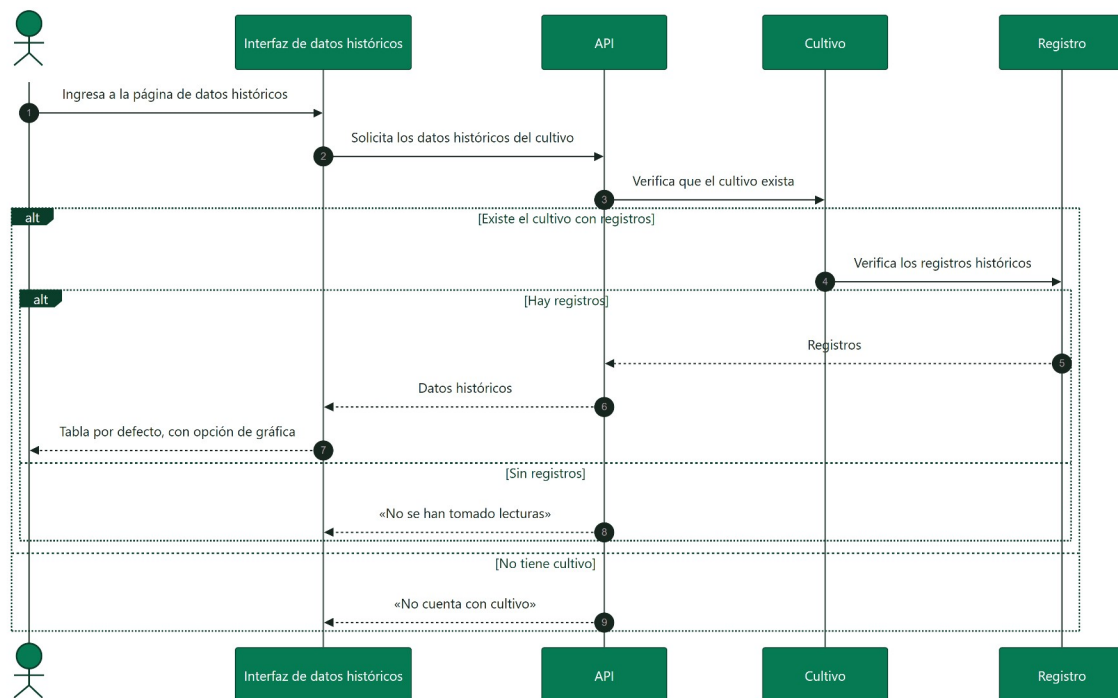


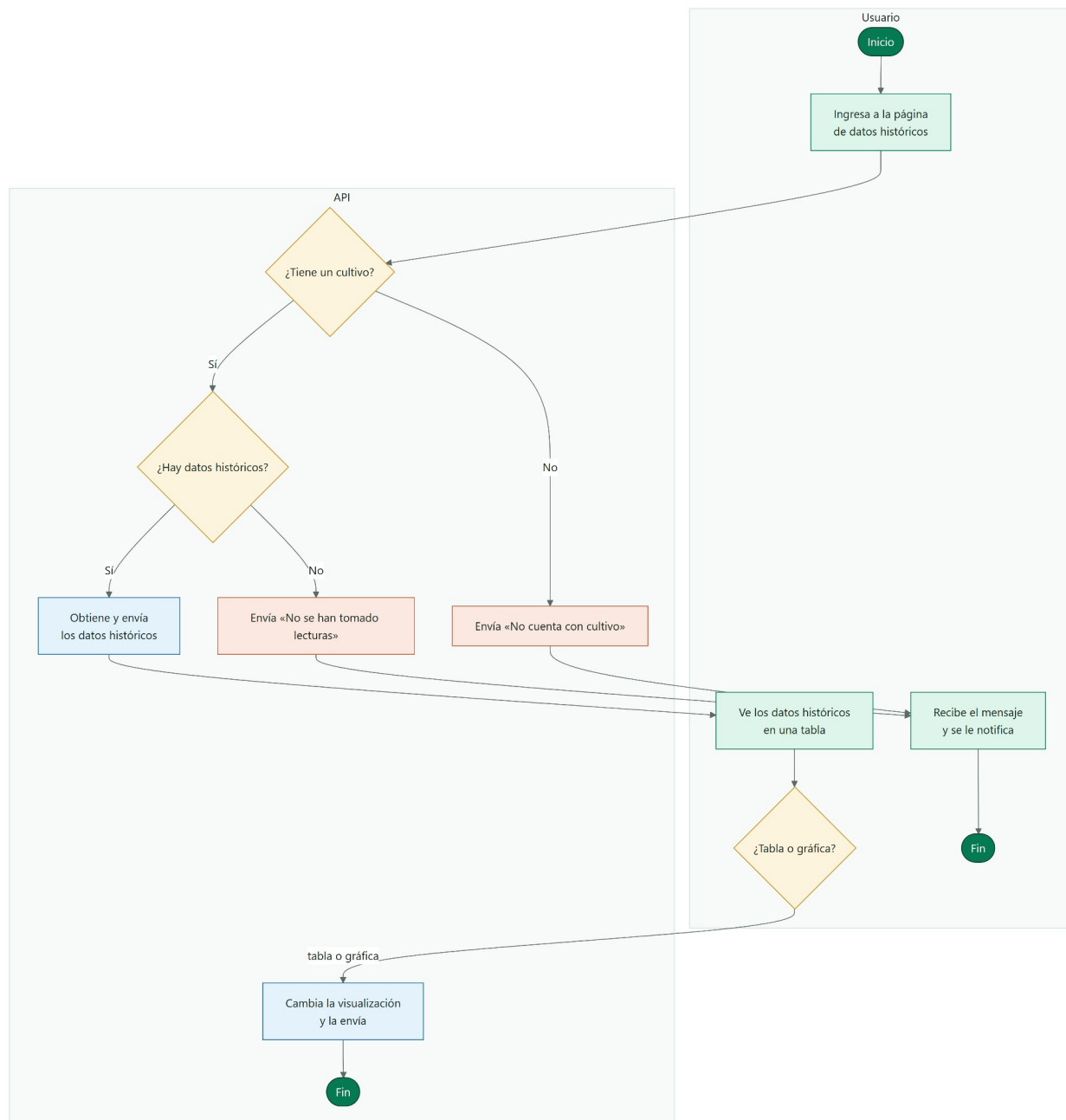
Figura 22 · Secuencia del diseño original: datos históricos

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    actor U as Usuario
    participant I as Interfaz de datos históricos
    participant A as API
    participant C as Cultivo
    participant R as Registro
    U->>I: Ingresa a la página de datos históricos
    I->>A: Solicita los datos históricos del cultivo
    A->>C: Verifica que el cultivo exista
    alt Existe el cultivo con registros
        C->>R: Verifica los registros históricos
        alt Hay registros
            R-->>A: Registros
            A-->>I: Datos históricos
            I-->>U: Tabla por defecto, con opción de gráfica
        else Sin registros
            A-->>I: «No se han tomado lecturas»
        end
    else No tiene cultivo
        A-->>I: «No cuenta con cultivo»
    end
end
```

La actividad original del mismo caso agregaba las validaciones: si la persona no tiene cultivo o si no hay lecturas, se le avisa; si hay datos, se muestran en una tabla que puede cambiar a gráfica. Hoy la gráfica es la vista por defecto y los estados vacíos reemplazan esos mensajes.

**Figura 23** · Actividad del diseño original: datos históricos

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph persona["Usuario"]
        direction TB
        inicio(["Inicio"]) --> entra["Ingresa a la página<br/>de datos históricos"]
        ve["Ve los datos históricos<br/>en una tabla"] --> elige{"¿Tabla o gráfica?"}
        recibe["Recibe el mensaje<br/>y se le notifica"] --> fin1(["Fin"])
    end
    subgraph api["API"]
        direction TB
        tiene{"¿Tiene un cultivo?"}
        hay{"¿Hay datos históricos?"}
        obtiene["Obtiene y envía<br/>los datos históricos"]
        sinCultivo["Envía «No cuenta con cultivo»"]
        sinDatos["Envía «No se han tomado lecturas»"]
        cambia["Cambia la visualización<br/>y la envía"] --> fin2(["Fin"])
    end
    entra --> tiene
    tiene -->|"Sí"| hay
    tiene -->|"No"| sinCultivo --> recibe
    hay -->|"Sí"| obtiene --> ve
    hay -->|"No"| sinDatos --> recibe
    elige -->|"tabla o gráfica"| cambia
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class inicio,fin1,fin2 core
    class entra,ve,recibe leaf
    class tiene,hay,elige sun
    class obtiene,cambia water
    class sinCultivo,sinDatos clay
```

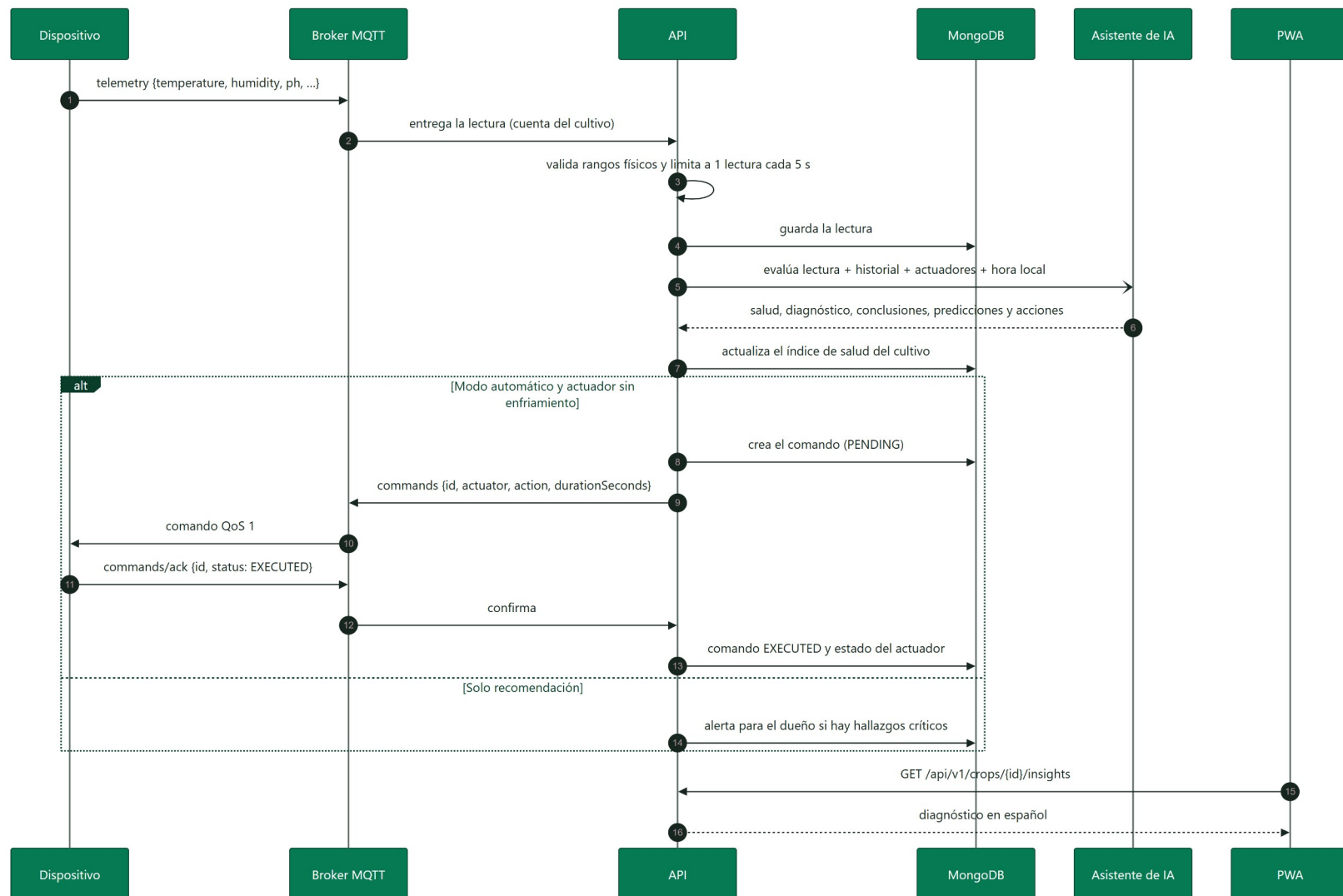


Figura 24 · Secuencia de una lectura con automatización



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    participant M as Dispositivo
    participant B as Broker MQTT
    participant A as API
    participant D as MongoDB
    participant I as Asistente de IA
    participant U as PWA
    M->>B: telemetry {temperature, humidity, ph, ...}
    B->>A: entrega la lectura (cuenta del cultivo)
    A->>A: valida rangos físicos y limita a 1 lectura cada 5 s
    A->>D: guarda la lectura
    A->>I: evalúa lectura + historial + actuadores + hora local
    I-->>A: salud, diagnóstico, conclusiones, predicciones y acciones
    A->>D: actualiza el índice de salud del cultivo
    alt Modo automático y actuador sin enfriamiento
        A->>D: crea el comando (PENDING)
        A->>B: commands {id, actuador, action, durationSeconds}
        B->>M: comando QoS 1
        M->>B: commands/ack {id, status: EXECUTED}
        B->>A: confirma
        A->>D: comando EXECUTED y estado del actuador
    else Solo recomendación
        A->>D: alerta para el dueño si hay hallazgos críticos
    end
    U->>A: GET /api/v1/crops/{id}/insights
    A-->>U: diagnóstico en español
```

21.5 Actividad: control automático

La actividad original ya tenía la idea correcta: el ESP32 lee y publica, la API guarda y, si una lectura pasa los límites, genera una alerta y el comando para el actuador de ese parámetro, que el dispositivo ejecuta y confirma.

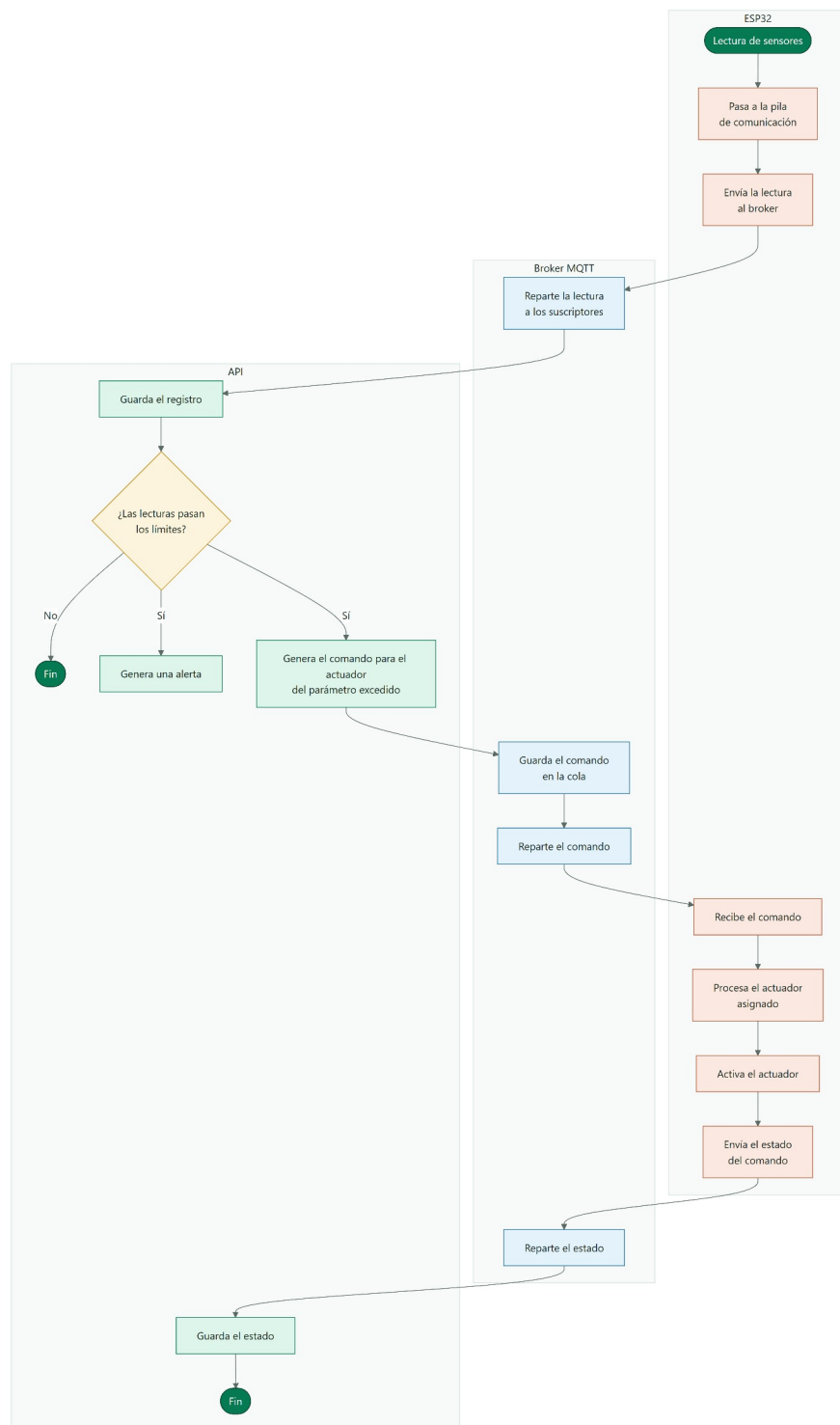


Figura 25 · Actividad del diseño original: control automático



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph esp["ESP32"]
        direction LR
        lee(["Lectura de sensores"]) --> pila["Pasa a la pila<br/>de comunicación"] -->
        publica["Envía la lectura<br/>al broker"]
        recibe["Recibe el comando"] --> procesa["Procesa el actuador<br/>asignado"] -->
        activa["Activa el actuador"] --> estado["Envía el estado<br/>del comando"]
    end
    subgraph broker["Broker MQTT"]
        direction LR
        reparte["Reparte la lectura<br/>a los suscriptores"]
        cola["Guarda el comando<br/>en la cola"] --> entrega["Reparte el comando"]
        reparteEstado["Reparte el estado"]
    end
    subgraph api["API"]
        direction LR
        guarda["Guarda el registro"] --> limites{"¿Las lecturas pasan<br/>los límites?"}
        limites -->|"No"| fin1(["Fin"])
        limites -->|"Sí"| alerta["Genera una alerta"]
        limites -->|"Sí"| genera["Genera el comando para el actuador<br/>del parámetro excedido"]
        guardaEstado["Guarda el estado"] --> fin2(["Fin"])
    end
    lee --> pila
    publica --> reparte
    genera --> cola
    entrega --> recibe
    estado --> reparteEstado
    reparteEstado --> guardaEstado
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class lee,fin1,fin2 core
    class pila,publica,recibe,procesa,activa,estado clay
    class reparte,cola,entrega,reparteEstado water
    class guarda,alerta,genera,guardaEstado leaf
    class limites sun
```



La actual agrega lo que la original no previó: la validación física de cada lectura, la evaluación asíncrona con el asistente, el modo automático que la persona activa, el enfriamiento por actuador, la confirmación con vencimiento y el aprendizaje con las lecturas reales.

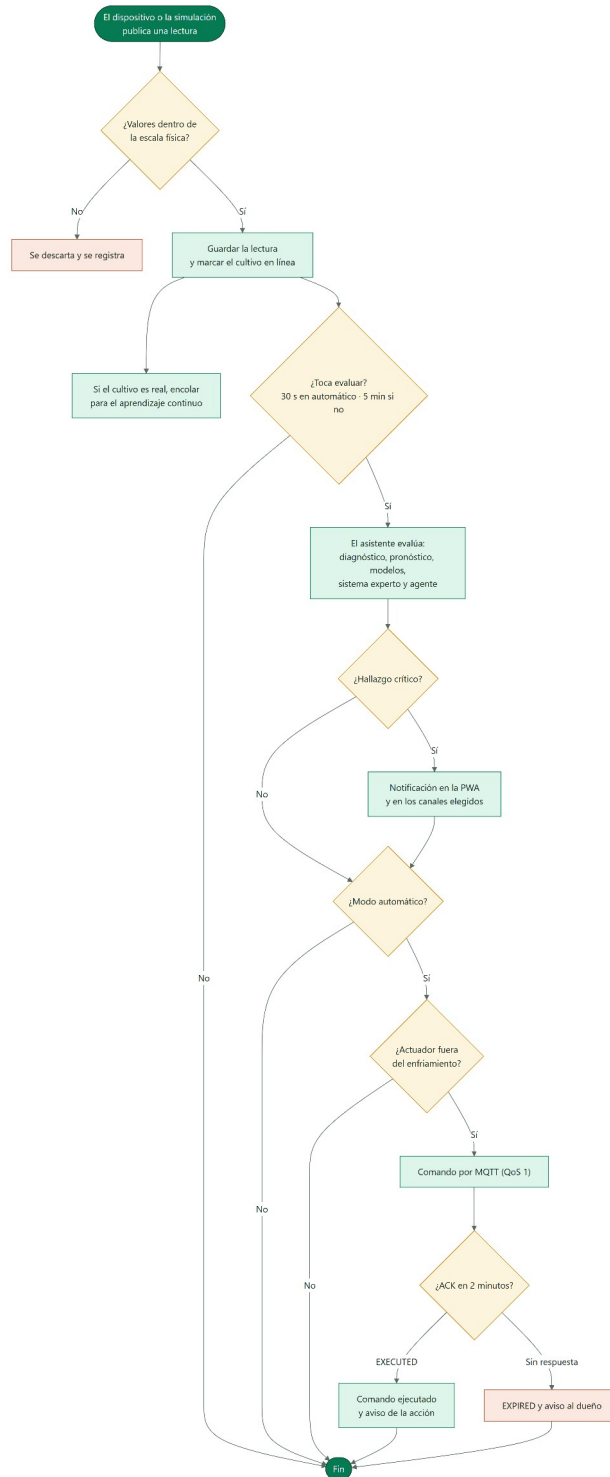


Figura 26 · Actividad del control automático



MERMAID

```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%

```

```

flowchart TB

```

```

    inicio(["El dispositivo o la simulación<br/>publica una lectura"]) → valida{"¿Valores
dentro de<br/>la escala física?"}
    valida →|"No"| descarta["Se descarta y se registra"]
    valida →|"Sí"| guarda["Guardar la lectura<br/>y marcar el cultivo en línea"]
    guarda → aprende["Si el cultivo es real, encolar<br/>para el aprendizaje continuo"]
    guarda → evalua{"¿Toca evaluar?<br/>30 s en automático · 5 min si no"}
    evalua →|"No"| fin(["Fin"])
    evalua →|"Sí"| ia["El asistente evalúa:<br/>diagnóstico, pronóstico, modelos,<br/>sistema
experto y agente"]
    ia → critico{"¿Hallazgo crítico?"}
    critico →|"Sí"| alerta["Notificación en la PWA<br/>y en los canales elegidos"]
    critico →|"No"| auto
    alerta → auto{"¿Modo automático?"}
    auto →|"No"| fin
    auto →|"Sí"| enfria{"¿Actuador fuera<br/>del enfriamiento?"}
    enfria →|"No"| fin
    enfria →|"Sí"| comando["Comando por MQTT (QoS 1)"]
    comando → ack{"¿ACK en 2 minutos?"}
    ack →|"EXECUTED"| ok["Comando ejecutado<br/>y aviso de la acción"]
    ack →|"Sin respuesta"| vence["EXPIRED y aviso al dueño"]
    ok → fin
    vence → fin
    classDef start fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef step fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef ask fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef bad fill:#FBE9E1,stroke:#B85A38,color:#17261F
    class inicio,fin start
    class guarda,aprende,ia,alerta,comando,ok step
    class valida,evalua,critico,auto,enfria,ack ask
    class descarta,vence bad

```

21.6 Datos

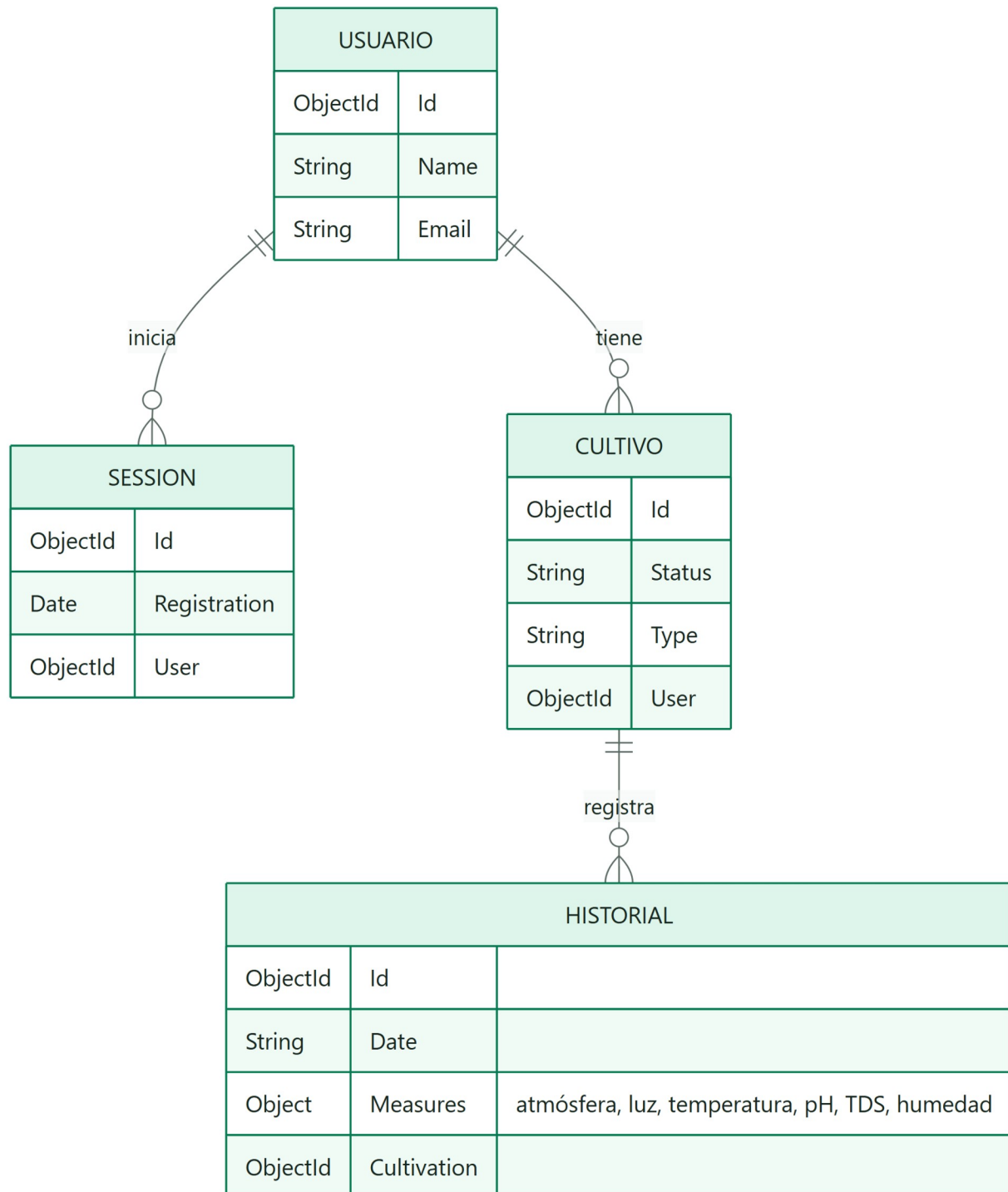


Figura 27 · Modelo de datos del diseño original



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
erDiagram
    USUARIO ||--o{ SESSION : "inicia"
    USUARIO ||--o{ CULTIVO : "tiene"
    CULTIVO ||--o{ HISTORIAL : "registra"
    USUARIO {
        ObjectId Id
        String Name
        String Email
    }
    SESSION {
        ObjectId Id
        Date Registration
        ObjectId User
    }
    CULTIVO {
        ObjectId Id
        String Status
        String Type
        ObjectId User
    }
    HISTORIAL {
        ObjectId Id
        String Date
        Object Measures "atmósfera, luz, temperatura, pH, TDS, humedad"
        ObjectId Cultivation
    }
```

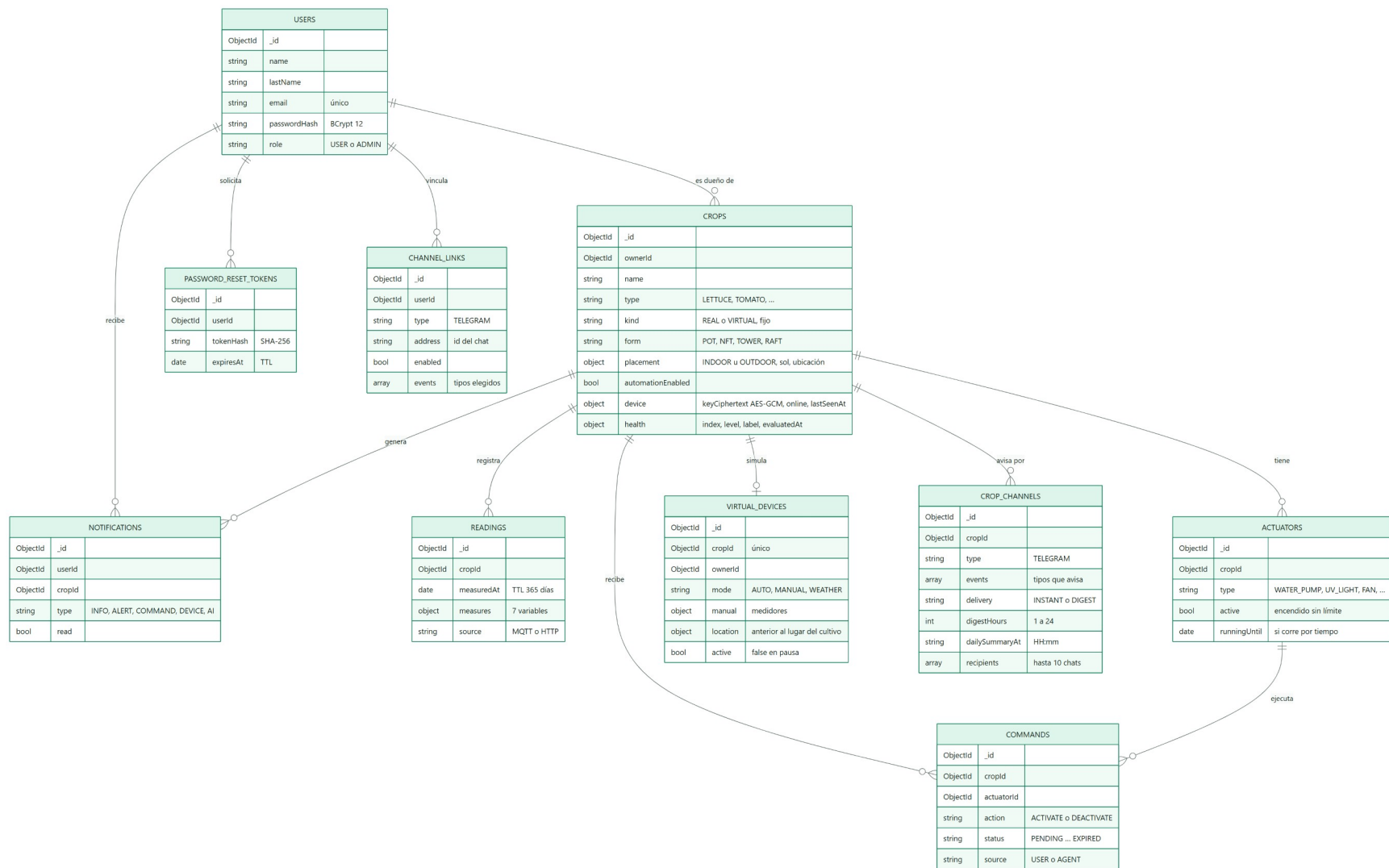


Figura 28 · Colecciones de MongoDB hoy



MERMAID



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%

```

```

erDiagram

```

```

    USERS ||--o{ CROPS : "es dueño de"
    USERS ||--o{ NOTIFICATIONS : "recibe"
    USERS ||--o{ PASSWORD_RESET_TOKENS : "solicita"
    USERS ||--o{ CHANNEL_LINKS : "vincula"
    CROPS ||--o{ READINGS : "registra"
    CROPS ||--o{ ACTUATORS : "tiene"
    CROPS ||--o{ COMMANDS : "recibe"
    CROPS ||--o{ NOTIFICATIONS : "genera"
    CROPS ||--o{ VIRTUAL_DEVICES : "simula"
    CROPS ||--o{ CROP_CHANNELS : "avisa por"
    ACTUATORS ||--o{ COMMANDS : "ejecuta"

    USERS {
        Objectid _id
        string name
        string lastName
        string email "único"
        string passwordHash "BCrypt 12"
        string role "USER o ADMIN"
    }

    CROPS {
        Objectid _id
        Objectid ownerId
        string name
        string type "LETTUCE, TOMATO, ..."
        string kind "REAL o VIRTUAL, fijo"
        string form "POT, NFT, TOWER, RAFT"
        object placement "INDOOR u OUTDOOR, sol, ubicación"
        bool automationEnabled
        object device "keyCiphertext AES-GCM, online, lastSeenAt"
        object health "index, level, label, evaluatedAt"
    }

    READINGS {
        Objectid _id
        Objectid cropId
        date measuredAt "TTL 365 días"
        object measures "7 variables"
        string source "MQTT o HTTP"
    }

    ACTUATORS {
        Objectid _id
        Objectid cropId
    }

```



```
    string type "WATER_PUMP, UV_LIGHT, FAN, ..."
    bool active "encendido sin límite"
    date runningUntil "si corre por tiempo"
}
COMMANDS {
    ObjectId _id
    ObjectId cropId
    ObjectId actuatorId
    string action "ACTIVATE o DEACTIVATE"
    string status "PENDING ... EXPIRED"
    string source "USER o AGENT"
}
NOTIFICATIONS {
    ObjectId _id
    ObjectId userId
    ObjectId cropId
    string type "INFO, ALERT, COMMAND, DEVICE, AI"
    bool read
}
CHANNEL_LINKS {
    ObjectId _id
    ObjectId userId
    string type "TELEGRAM"
    string address "id del chat"
    bool enabled
    array events "tipos elegidos"
}
CROP_CHANNELS {
    ObjectId _id
    ObjectId cropId
    string type "TELEGRAM"
    array events "tipos que avisa"
    string delivery "INSTANT o DIGEST"
    int digestHours "1 a 24"
    string dailySummaryAt "HH:mm"
    array recipients "hasta 10 chats"
}
VIRTUAL_DEVICES {
    ObjectId _id
    ObjectId cropId "único"
    ObjectId ownerId
    string mode "AUTO, MANUAL, WEATHER"
    object manual "medidores"
    object location "anterior al lugar del cultivo"
    bool active "false en pausa"
}
PASSWORD_RESET_TOKENS {
    ObjectId _id
    ObjectId userId
    string tokenHash "SHA-256"
    date expiresAt "TTL"
}
```



DISEÑO ORIGINAL	ACTUAL	LECTURA
Session guardada en la base	Sin colección de sesiones	JWT sin estado
Historial con Measures y referencia al cultivo	readings con índice por cultivo y fecha, y TTL de un año	La idea se conservó; se agregaron índices y retención
Cuatro entidades	Colecciones de usuarios, cultivos, lecturas, comandos, notificaciones, vínculos de canales y más	El modelo creció con el dominio, no por anticipado

21.7 Objetos y artefactos

El diseño incluía otros dos diagramas que no tienen un equivalente uno a uno. El de objetos se redibuja con datos de ejemplo neutros: el original tenía un nombre y un correo de broma y fechas imposibles.

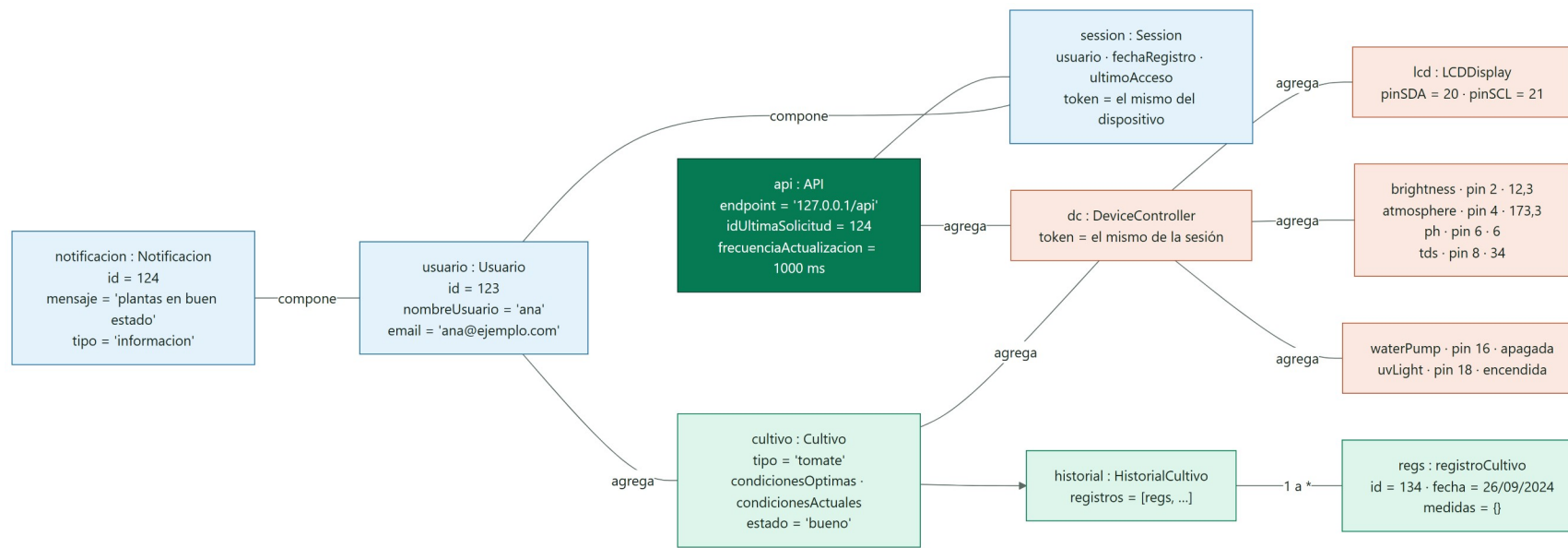


Figura 29 · Objetos del diseño original



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    usuario["usuario : Usuario<br/>id = 123<br/>nombreUsuario = 'ana'<br/>email =
'ana@ejemplo.com'"]
    session["session : Session<br/>usuario · fechaRegistro · ultimoAcceso<br/>token = el mismo
del dispositivo"]
    notificacion["notificacion : Notificacion<br/>id = 124<br/>mensaje = 'plantas en buen
estado'<br/>tipo = 'informacion'"]
    cultivo["cultivo : Cultivo<br/>tipo = 'tomate'<br/>condicionesOptimas ·
condicionesActuales<br/>estado = 'bueno'"]
    historial["historial : HistorialCultivo<br/>registros = [regs, ...]"]
    regs["regs : registroCultivo<br/>id = 134 · fecha = 26/09/2024<br/>medidas = {}"]
    dc["dc : DeviceController<br/>token = el mismo de la sesión"]
    api["api : API<br/>endpoint = '127.0.0.1/api'<br/>idUltimaSolicitud =
124<br/>frecuenciaActualizacion = 1000 ms"]
    lcd["lcd : LCDDisplay<br/>pinSDA = 20 · pinSCL = 21"]
    sensores["sensores : brightness · pin 2 · 12,3<br/>atmosphere · pin 4 · 173,3<br/>ph · pin 6 ·
6<br/>tds · pin 8 · 34"]
    actuadores["actuadores : waterPump · pin 16 · apagada<br/>uvLight · pin 18 · encendida"]

    usuario ---| "compone" | session
    notificacion ---| "compone" | usuario
    usuario ---| "agrega" | cultivo
    cultivo --> historial
    historial ---| "1 a *" | regs
    cultivo ---| "agrega" | dc
    api ---| "agrega" | dc
    api --- session
    dc ---| "agrega" | lcd
    dc ---| "agrega" | sensores
    dc ---| "agrega" | actuadores

    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F

    class usuario,session,notificacion water
    class cultivo,historial,regs leaf
    class dc,lcd,sensores,actuadores clay
    class api core
```



El de artefactos tenía dos versiones; esta es la detallada:

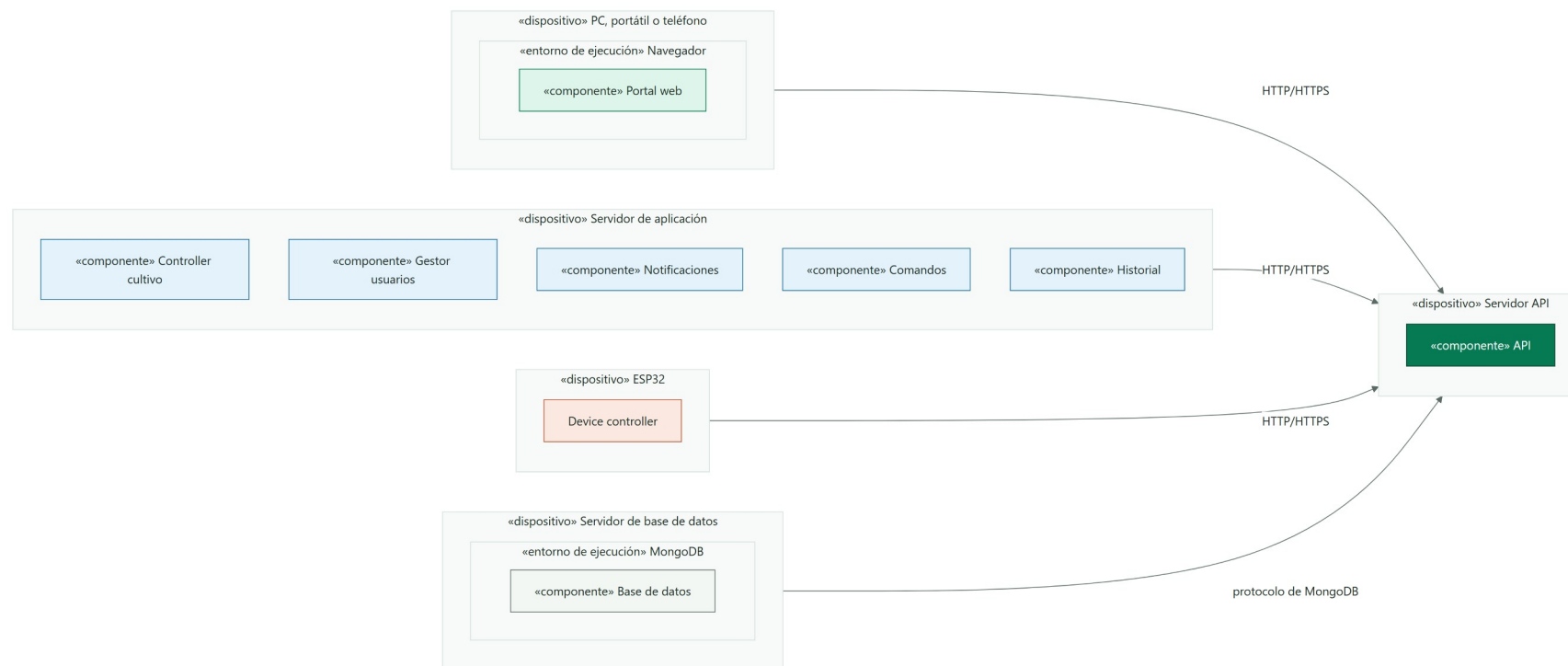


Figura 30 · Artefactos del diseño original



MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph cliente["«dispositivo» PC, portátil o teléfono"]
        subgraph navegador["«entorno de ejecución» Navegador"]
            portal["«componente» Portal web"]
        end
    end
    subgraph aplicacion["«dispositivo» Servidor de aplicación"]
        direction TB
        controller["«componente» Controller cultivo"]
        gestor["«componente» Gestor usuarios"]
        notificaciones["«componente» Notificaciones"]
        comandos["«componente» Comandos"]
        historial["«componente» Historial"]
    end
    subgraph servidorApi["«dispositivo» Servidor API"]
        api["«componente» API"]
    end
    subgraph esp["«dispositivo» ESP32"]
        device["Device controller"]
    end
    subgraph datos["«dispositivo» Servidor de base de datos"]
        subgraph mongo["«entorno de ejecución» MongoDB"]
            bd["«componente» Base de datos"]
        end
    end
    cliente -->|"HTTP/HTTPS"| servidorApi
    aplicacion -->|"HTTP/HTTPS"| servidorApi
    esp -->|"HTTP/HTTPS"| servidorApi
    datos -->|"protocolo de MongoDB"| servidorApi
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class portal leaf
    class controller,gestor,notificaciones,comandos,historial water
    class api core
    class device clay
    class bd muted
```



DIAGRAMA ORIGINAL	QUÉ MOSTRABA	HOY
Objetos	Instancias de <code>DeviceController</code> , <code>Cultivo</code> , <code>Session</code> , <code>Notificacion</code> y sensores con valores de ejemplo	Las pruebas de la API y de la IA construyen esos mismos objetos con datos válidos en cada ejecución
Artefactos	Navegador, servidor de aplicación, servidor de API, servidor de base de datos con MongoDB y el ESP32, unidos por HTTP	El despliegue continuo y las redes de producción (figuras de despliegue y redes)

El diagrama de objetos quedó a medio terminar: conserva los compartimentos de ejemplo de la herramienta y fechas imposibles como «34/34/2324». El de artefactos, en cambio, ya elegía MongoDB, aunque el acta posterior pedía H2 o MySQL con JPA: la decisión de datos se tomó en el diseño y el acta no la recogió.

LECTURA CRÍTICA

El diseño original acertó en lo que depende del problema (sensores, actuadores, cultivo, historial y un broker) y falló en lo que depende de la tecnología (dónde se decide, cómo se guarda una sesión, cómo se confirma un comando). Es la señal de un buen análisis del dominio con poca experiencia en sistemas distribuidos, y justifica haber rediseñado desde el contrato MQTT hacia afuera.



21.8 Paquetes

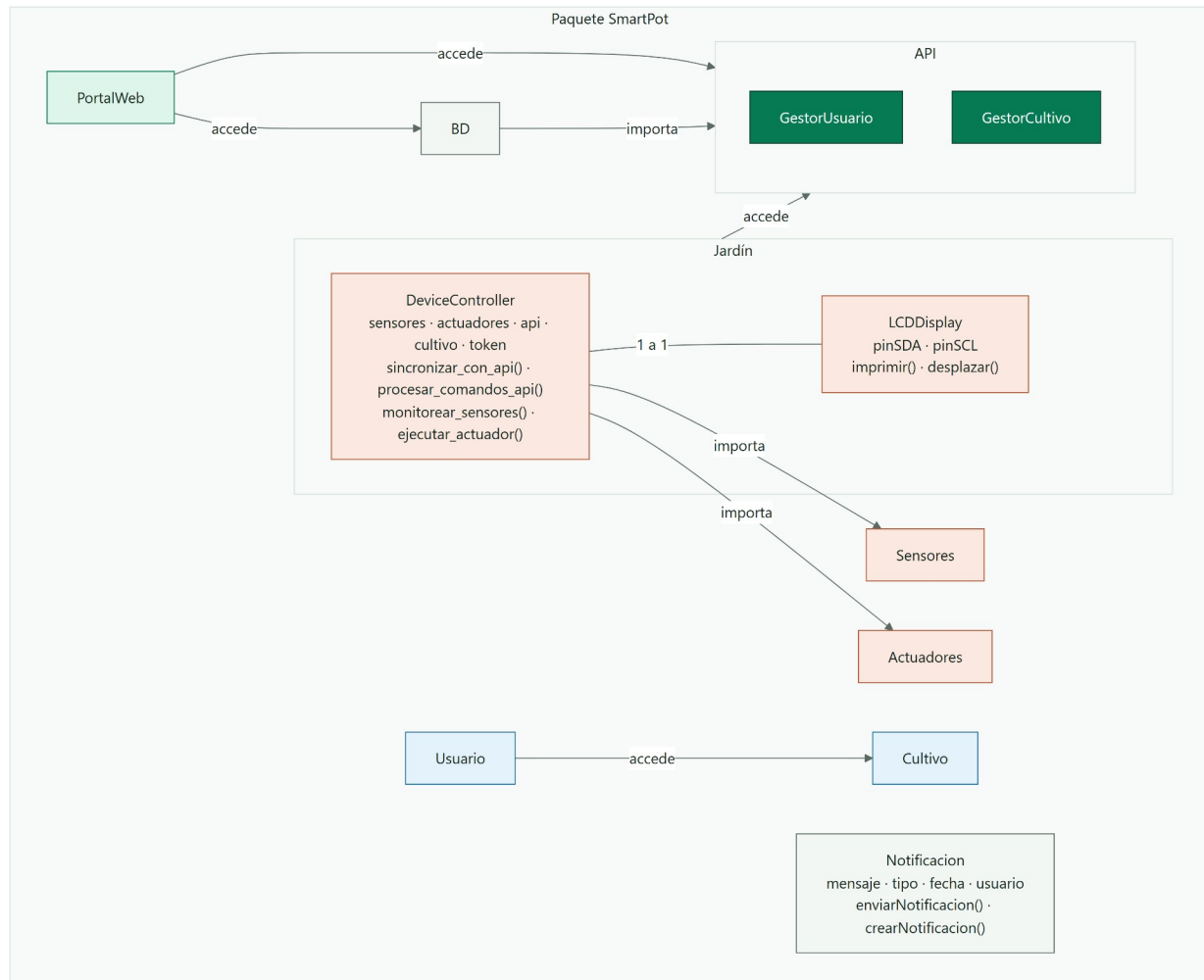


Figura 31 · Paquetes del diseño original

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFmly": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    subgraph smartpot["Paquete SmartPot"]
        direction LR
        subgraph jardin["Jardín"]
            direction TB
            dc["DeviceController<br/>sensores · actuadores · api · cultivo ·
token<br/>sincronizar_con_api() · procesar_comandos_api(<br/>monitorear_sensores() ·
ejecutar_actuador()"]
            lcd["LCDDisplay<br/>pinSDA · pinSCL<br/>imprimir() · desplazar()"]
        end
        sensores["Sensores"]
        actuadores["Actuadores"]
        subgraph api["API"]
            direction TB
            usuarios["GestorUsuario"]
            cultivos["GestorCultivo"]
        end
        portal["PortalWeb"]
        bd["BD"]
        usuario["Usuario"]
        cultivo["Cultivo"]
        notificacion["Notificacion<br/>mensaje · tipo · fecha · usuario<br/>enviarNotificacion()
· crearNotificacion()"]
    end
    dc -->|"importa"| sensores
    dc -->|"importa"| actuadores
    jardin -->|"accede"| api
    dc ---|"1 a 1"| lcd
    portal -->|"accede"| api
    portal -->|"accede"| bd
    bd -->|"importa"| api
    usuario -->|"accede"| cultivo
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class dc,lcd clay
    class sensores,actuadores clay
    class usuarios,cultivos core
```

```
class portal leaf
class bd,notificacion muted
class usuario,cultivo water
```

DISEÑO ORIGINAL	HOY	POR QUÉ
Un paquete SmartPot con Jardín, Sensores, Actuadores, API, PortalWeb, BD, Usuario y Cultivo	Diez repositorios, uno por componente, y dentro de la API un paquete por dominio (crops, readings, commands, actuators, channels, ai, overview, virtualdevices, users, security, notifications, mqtt, cache, mail, health)	Cada componente se publica, se prueba y se despliega por separado
PortalWeb accede a API y también a BD	La PWA solo habla con la API	Ninguna pantalla puede saltarse el control de dueño
BD importa API	La API depende de la base, nunca al revés	La dependencia original estaba invertida
DeviceController accede a API	El dispositivo solo habla con el broker, con una cuenta por cultivo	Un dispositivo comprometido no toca la API ni otros cultivos
GestorUsuario y GestorCultivo	users y security; crops, actuators y commands	El gestor de cultivos se dividió por responsabilidades

SECCIÓN 22

Interfaz

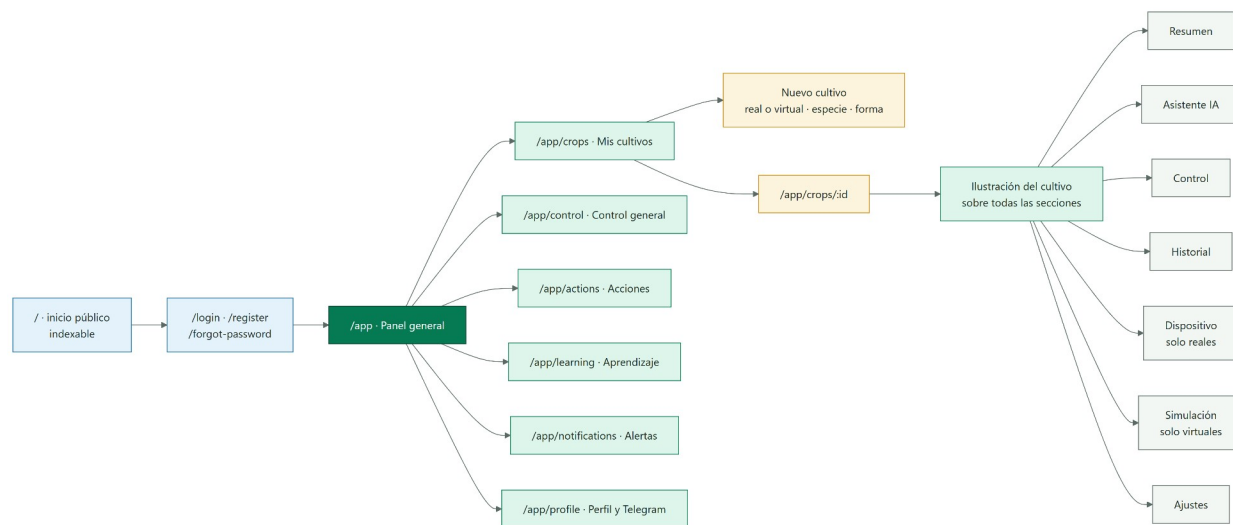


Figura 32 · Mapa de la PWA



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    inicio["/ · inicio público<br/>indexable"] --> ingreso["/login · /register<br/>/forgot-
password"]
    ingreso --> panel["/app · Panel general"]
    panel --> cultivos["/app/crops · Mis cultivos"]
    cultivos --> nuevo["Nuevo cultivo<br/>real o virtual · especie · forma"]
    cultivos --> detalle["/app/crops/:id"]
    detalle --> hero["Ilustración del cultivo<br/>sobre todas las secciones"]
    hero --> t1["Resumen"] & t2["Asistente IA"] & t3["Control"] & t4["Historial"] &
t5["Dispositivo<br/>solo reales"] & t6["Simulación<br/>solo virtuales"] & t7["Ajustes"]
    panel --> control["/app/control · Control general"]
    panel --> acciones["/app/actions · Acciones"]
    panel --> aprendizaje["/app/learning · Aprendizaje"]
    panel --> alertas["/app/notifications · Alertas"]
    panel --> perfil["/app/profile · Perfil y Telegram"]
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class inicio,ingreso water
    class panel core
    class cultivos,control,acciones,aprendizaje,alertas,perfil leaf
    class detalle,nuevo sun
    class hero leaf
    class t1,t2,t3,t4,t5,t6,t7 muted
```

PRINCIPIO	CÓMO SE APLICA
Primero el teléfono	Navegación inferior en el teléfono y lateral en el escritorio; la página de aprendizaje solo en escritorio
Lenguaje simple	Textos en español, sin jerga, con explicaciones en cada tarjeta
Accesibilidad	Colores de texto de alto contraste, tablas con encabezados y títulos accesibles, y movimiento reducido respetado en la escena del clima
Identidad	Paleta SmartPot (verdes hoja, azul agua, sol y arcilla) validada para daltonismo en las gráficas

PRINCIPIO	CÓMO SE APLICA
Descubrimiento	Página de inicio indexable con metadatos para buscadores y redes

SECCIÓN 23

Seguridad por diseño

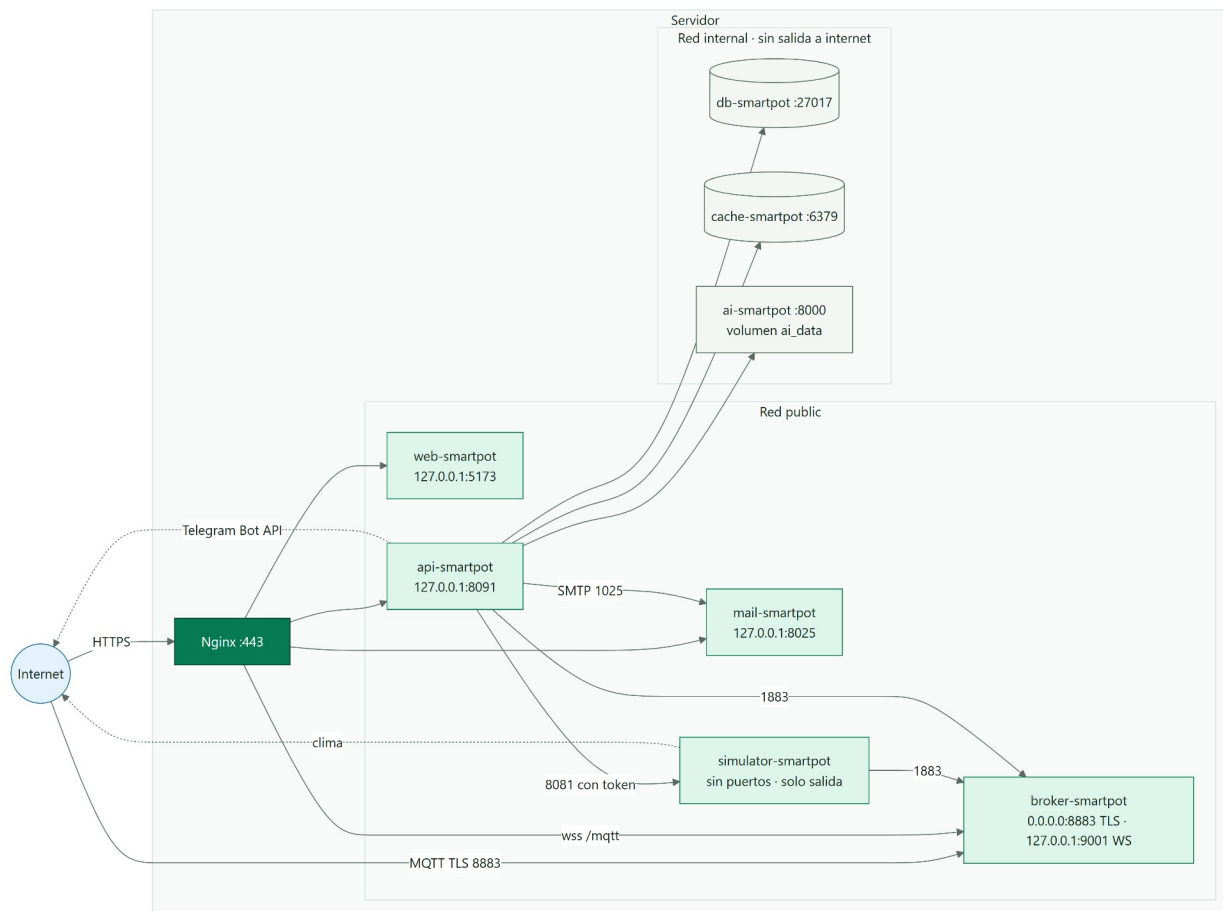


Figura 33 · Redes y puertos en producción

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    internet(("Internet"))
    subgraph host ["Servidor"]
        nginx["Nginx :443"]
        subgraph public ["Red public"]
            web["web-smartpot<br/>127.0.0.1:5173"]
            api["api-smartpot<br/>127.0.0.1:8091"]
            broker["broker-smartpot<br/>0.0.0.0:8883 TLS · 127.0.0.1:9001 WS"]
            mail["mail-smartpot<br/>127.0.0.1:8025"]
            sim["simulator-smartpot<br/>sin puertos · solo salida"]
        end
        end
        subgraph internal ["Red internal · sin salida a internet"]
            db["db-smartpot :27017"]
            cache["cache-smartpot :6379"]
            ai["ai-smartpot :8000<br/>volumen ai_data"]
        end
    end
    internet -->|"HTTPS"| nginx
    internet -->|"MQTT TLS 8883"| broker
    nginx --> web
    nginx --> api
    nginx -->|"wss /mqtt"| broker
    nginx --> mail
    api --> db
    api --> cache
    api --> ai
    api -->|"8081 con token"| sim
    sim -->|"1883"| broker
    sim -.→|"clima"| internet
    api -.→|"Telegram Bot API"| internet
    api -->|"1883"| broker
    api -->|"SMTP 1025"| mail
    classDef pub fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef priv fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    classDef gate fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef world fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    class web,api,broker,mail,sim pub
    class db,cache,ai priv
    class nginx gate
    class internet world
```



AMENAZA	CONTROL
Suplantación de un dispositivo	Usuario MQTT por cultivo con clave generada por la API, cifrada con AES-GCM y ACL por tópico
Acceso a cultivos ajenos	Control de dueño en cada servicio, probado con la cadena de seguridad real
Fuerza bruta	BCrypt de costo 12 y límite de peticiones por IP
Escucha en la red	TLS en la web y en la API; MQTT en el puerto 8883 con una autoridad certificadora propia
Exposición de servicios internos	Redes internas de Docker y solo Nginx publicado
Webhook falsificado	Secreto en la cabecera de Telegram
Dependencias vulnerables	Dependabot, CodeQL y SBOM en cada imagen



PARTE V

Construcción

SECCIÓN 24

Organización del trabajo

PRÁCTICA	DETALLE
Repositorios	Uno por componente, más <code>.github</code> para entornos, despliegue, QA y documentación
Idiomas	Código en inglés; interfaz, mensajes, registros y documentación en español
Herramientas	Python con uv, web con pnpm, Java con Maven Wrapper
Commits	En inglés, en imperativo, con un solo propósito y uno o dos archivos
Calidad continua	CI en cada repositorio, CodeQL, Dependabot y QA central
Entornos	Demo de un solo comando, desarrollo y producción con el mismo <code>compose</code> base



SECCIÓN 25

Infraestructura y despliegue

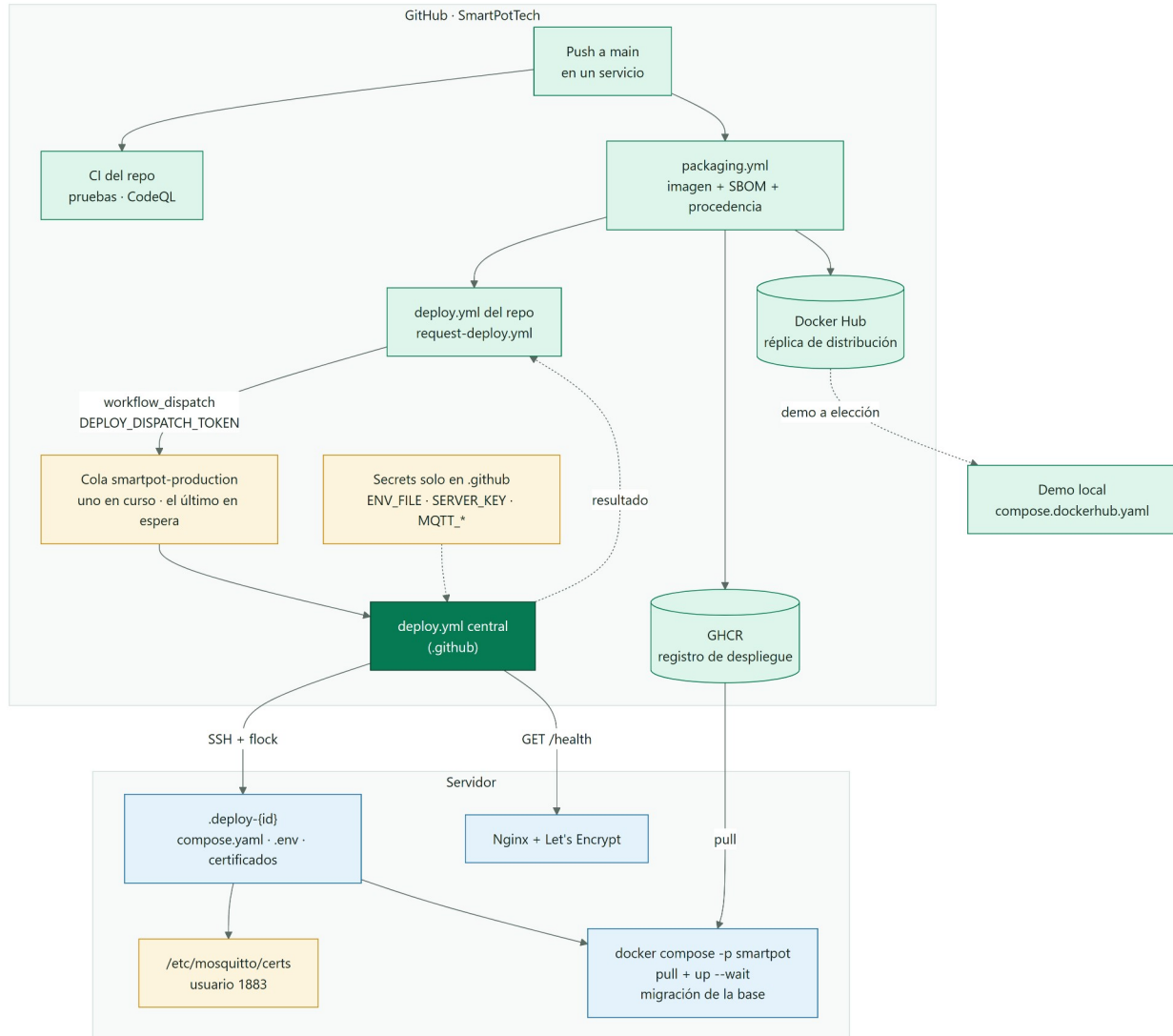


Figura 34 · Despliegue continuo

MERMAID



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph github["GitHub · SmartPotTech"]
        repo["Push a main<br/>en un servicio"] --> ci["CI del repo<br/>pruebas · CodeQL"]
        repo --> pkg["packaging.yml<br/>imagen + SBOM + procedencia"]
        pkg --> ghcr["GHCR<br/>registro de despliegue"]
        pkg --> hub["Docker Hub<br/>réplica de distribución"]
        pkg --> dep["deploy.yml del repo<br/>request-deploy.yml"]
        dep -->|"workflow_dispatch<br/>DEPLOY_DISPATCH_TOKEN"| queue["Cola smartpot-
production<br/>uno en curso · el último en espera"]
        queue --> central["deploy.yml central<br/>(.github)"]
        central -->|"resultado"| dep
        secrets["Secrets solo en .github<br/>ENV_FILE · SERVER_KEY · MQTT_*"] --> central
    end
    subgraph servidor["Servidor"]
        stage["deploy-{id}<br/>compose.yml · .env · certificados"]
        certs["/etc/mosquitto/certs<br/>usuario 1883"]
        compose["docker compose -p smartpot<br/>pull + up --wait<br/>migración de la base"]
        nginx["Nginx + Let's Encrypt"]
    end
    central -->|"SSH + flock"| stage
    stage --> certs
    stage --> compose
    ghcr -->|"pull"| compose
    hub -->|"demo a elección"| demo["Demo local<br/>compose.dockerhub.yml"]
    central -->|"GET /health"| nginx
    classDef gh fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef key fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef srv fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class repo,ci,pkg,dep,demo gh
    class central core
    class secrets,certs,queue key
    class stage,compose,nginx srv

```

Cada servicio publica su imagen en GHCR (y en Docker Hub como réplica) y pide el despliegue al workflow central, que corre de a uno: si llegan varios pedidos a la vez, solo el más reciente espera. El servidor descarga las imágenes, recrea solo lo que cambió y aplica los esquemas de la base con la migración de SmartPot-DB, idempotente, que evita depender de los scripts de inicio de MongoDB.



SECCIÓN 26

Asistente de IA

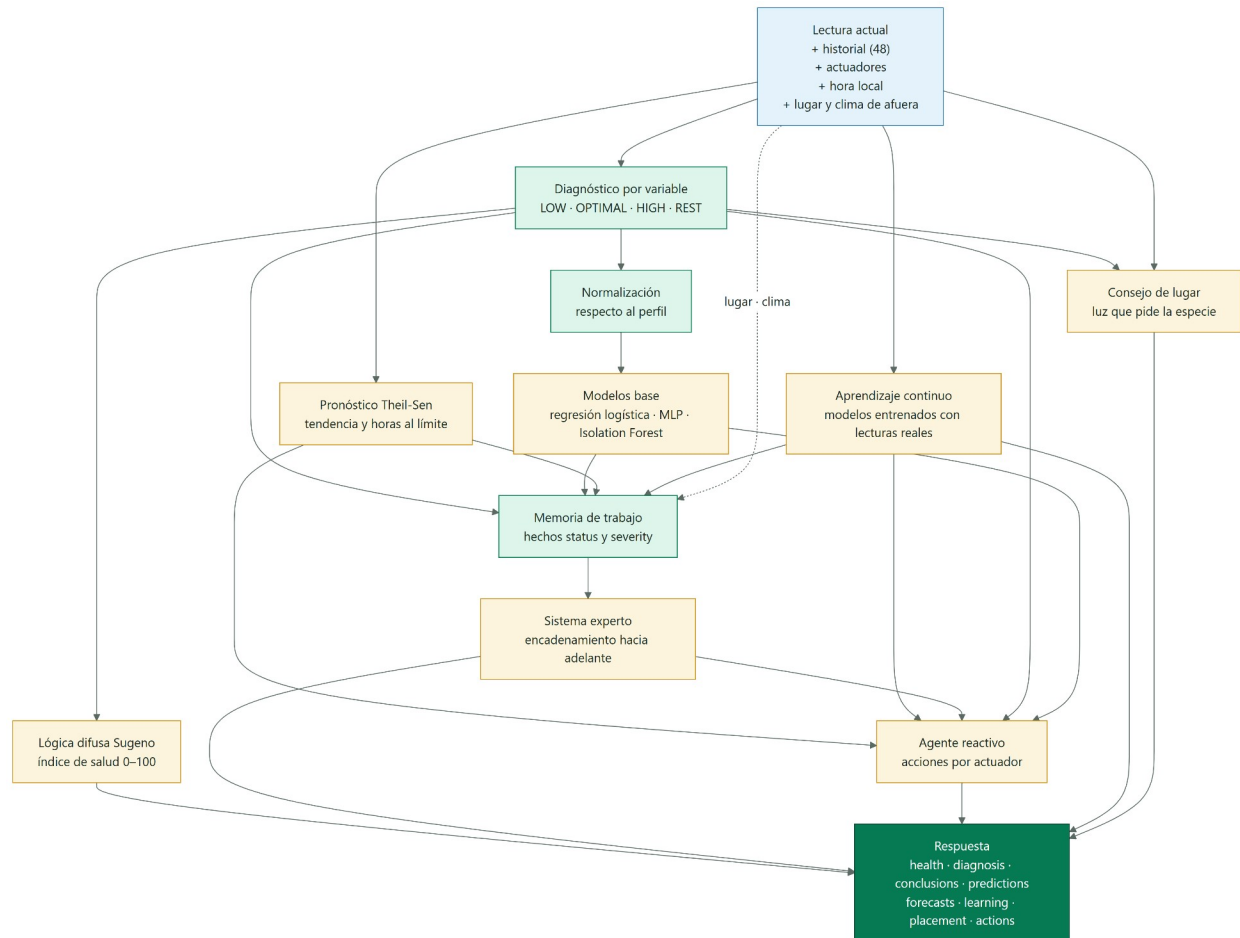


Figura 35 · Técnicas del asistente de IA

MERMAID



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    req["Lectura actual<br/>+ historial (48)<br/>+ actuadores<br/>+ hora local<br/>+ lugar y
clima de afuera"] --> diag["Diagnóstico por variable<br/>LOW · OPTIMAL · HIGH · REST"]
    req --> fc["Pronóstico Theil-Sen<br/>tendencia y horas al límite"]
    req --> lr["Aprendizaje continuo<br/>modelos entrenados con lecturas reales"]
    diag --> norm["Normalización<br/>respecto al perfil"]
    norm --> ml["Modelos base<br/>regresión logística · MLP · Isolation Forest"]
    diag --> mem["Memoria de trabajo<br/>hechos status y severity"]
    ml --> mem
    fc --> mem
    lr --> mem
    fc --> ag
    lr --> ag
    req -.→| "lugar · clima" | mem
    mem --> es["Sistema experto<br/>encadenamiento hacia adelante"]
    diag --> place["Consejo de lugar<br/>luz que pide la especie"]
    req --> place
    diag --> fz["Lógica difusa Sugeno<br/>índice de salud 0-100"]
    es --> ag["Agente reactivo<br/>acciones por actuador"]
    diag --> ag
    ml --> ag
    fz --> out["Respuesta<br/>health · diagnosis · conclusions · predictions<br/>forecasts ·
learning · placement · actions"]
    es --> out
    ag --> out
    lr --> out
    place --> out
    classDef input fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef step fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef brain fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef result fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class req input
    class diag,norm,mem step
    class ml,es,fz,ag,fc,lr,place brain
    class out result

```

La IA se construyó por capas, de lo explicable a lo aprendido: sistema experto con reglas encadenadas, lógica difusa para el índice de salud, pronóstico Theil-Sen de horas hasta el límite, análisis de flota con K-Means y, encima, modelos entrenados con las lecturas reales.

SECCIÓN 27

Integraciones

27.1 Telegram

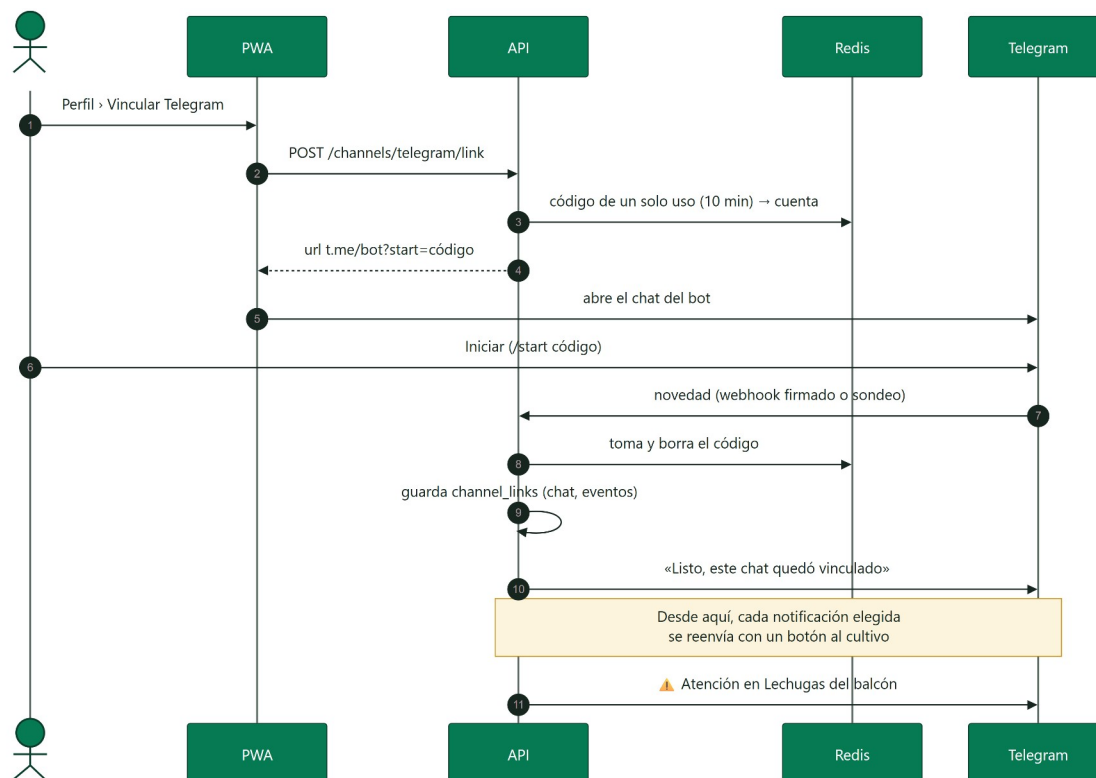


Figura 36 · Vinculación de Telegram

MERMAID



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    actor P as Persona
    participant W as PWA
    participant A as API
    participant C as Redis
    participant T as Telegram
    P->>W: Perfil > Vincular Telegram
    W->>A: POST /channels/telegram/link
    A->>C: código de un solo uso (10 min) → cuenta
    A-->>W: url t.me/bot?start=código
    W->>T: abre el chat del bot
    P->>T: Iniciar (/start código)
    T->>A: novedad (webhook firmado o sondeo)
    A->>C: toma y borra el código
    A->>A: guarda channel_links (chat, eventos)
    A->>T: «Listo, este chat quedó vinculado»
    Note over A,T: Desde aquí, cada notificación elegida<br/>se reenvía con un botón al cultivo
    A->>T: ▲ Atención en Lechugas del balcón

```

27.2 Cultivos reales y virtuales

La primera versión encendía una «maceta virtual» sobre cualquier cultivo, incluso uno con hardware, y dibujaba siempre una maceta. La revisión separó los dos mundos y los hizo fieles: el tipo se elige al crear el cultivo y no cambia (el API rechaza el cambio); un cultivo virtual no entrega credenciales, nace con los seis actuadores, se pausa sin perder su configuración y no alimenta el aprendizaje; Wokwi, al ejecutar el firmware real, cuenta como cultivo real. Cada cultivo tiene además su forma (maceta, tubos NFT, torre vertical o balsa flotante) y se dibuja sobre todas sus secciones con su especie y cada actuador encendido o apagado, sin botones; si no está conectado, no se ilustra. La simulación de los virtuales se configura en su propia pestaña.

27.3 Lugar, switches y avisos por cultivo

El último ciclo partió de la revisión de la interfaz con cultivos de verdad. Cada cultivo dice dónde está (bajo techo o al aire libre, cuánto sol recibe y en qué ciudad); con eso la PWA lo dibuja en su lugar, el simulador filtra el clima real y el asistente compara la luz que pide la especie con la que recibe, sin impedir



que la persona lo deje donde quiera. Los actuadores pasaron a ser switches que solo aceptan órdenes que cambian algo, y en los cultivos virtuales cada orden mueve las lecturas en los tres modos de simulación. Los avisos dejaron de ser iguales para todos: cada cultivo elige qué manda por Telegram, al instante o en resúmenes, y con quién se comparte. El detalle está en la documentación técnica (secciones 5, 6.9 y 9).

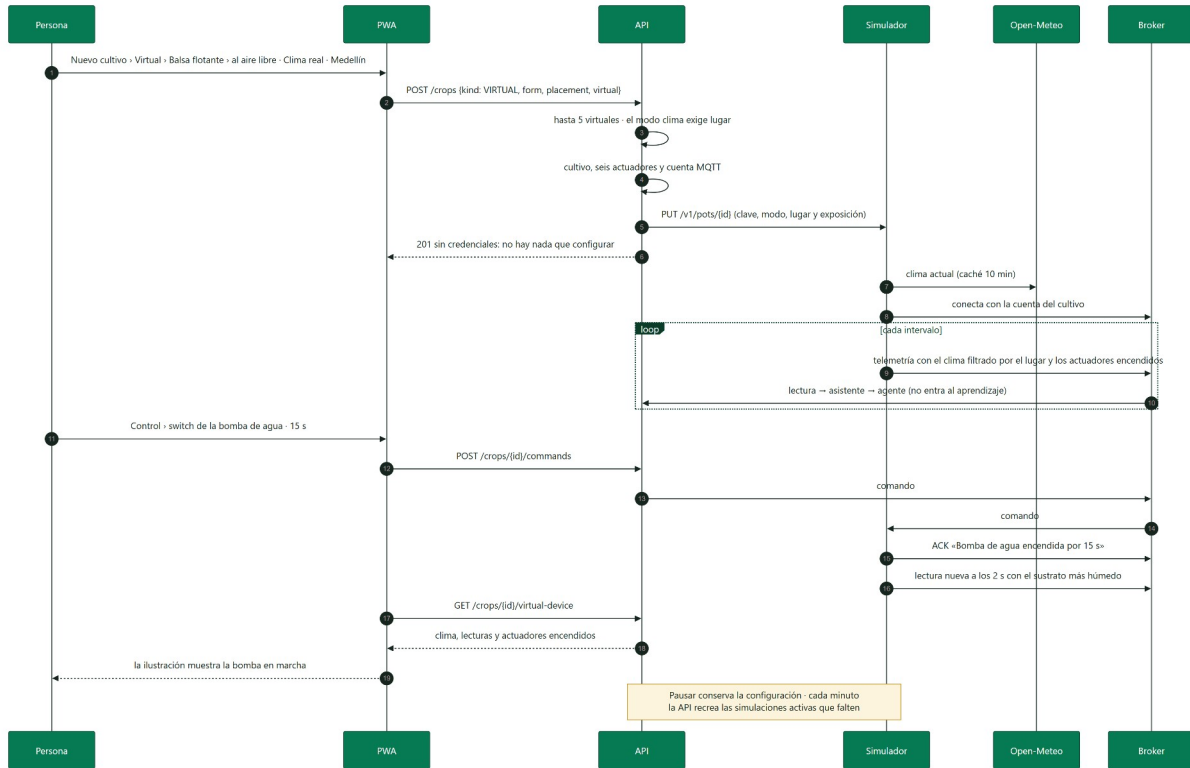


Figura 37 · Cultivo virtual con clima real



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    participant P as Persona
    participant W as PWA
    participant A as API
    participant S as Simulador
    participant O as Open-Meteo
    participant B as Broker
    P->>W: Nuevo cultivo > Virtual > Balsa flotante > al aire libre · Clima real · Medellín
    W->>A: POST /crops {kind: VIRTUAL, form, placement, virtual}
    A->>A: hasta 5 virtuales · el modo clima exige lugar
    A->>A: cultivo, seis actuadores y cuenta MQTT
    A->>S: PUT /v1/pots/{id} (clave, modo, lugar y exposición)
    A-->>W: 201 sin credenciales: no hay nada que configurar
    S->>O: clima actual (caché 10 min)
    S->>B: conecta con la cuenta del cultivo
    loop cada intervalo
        S->>B: telemetría con el clima filtrado por el lugar y los actuadores encendidos
        B->>A: lectura → asistente → agente (no entra al aprendizaje)
    end
    P->>W: Control > switch de la bomba de agua · 15 s
    W->>A: POST /crops/{id}/commands
    A->>B: comando
    B->>S: comando
    S->>B: ACK «Bomba de agua encendida por 15 s»
    S->>B: lectura nueva a los 2 s con el sustrato más húmedo
    W->>A: GET /crops/{id}/virtual-device
    A-->>W: clima, lecturas y actuadores encendidos
    W-->>P: la ilustración muestra la bomba en marcha
    Note over A,S: Pausar conserva la configuración · cada minuto<br/>la API recrea las
simulaciones activas que falten
```



SECCIÓN 28

Aprendizaje continuo

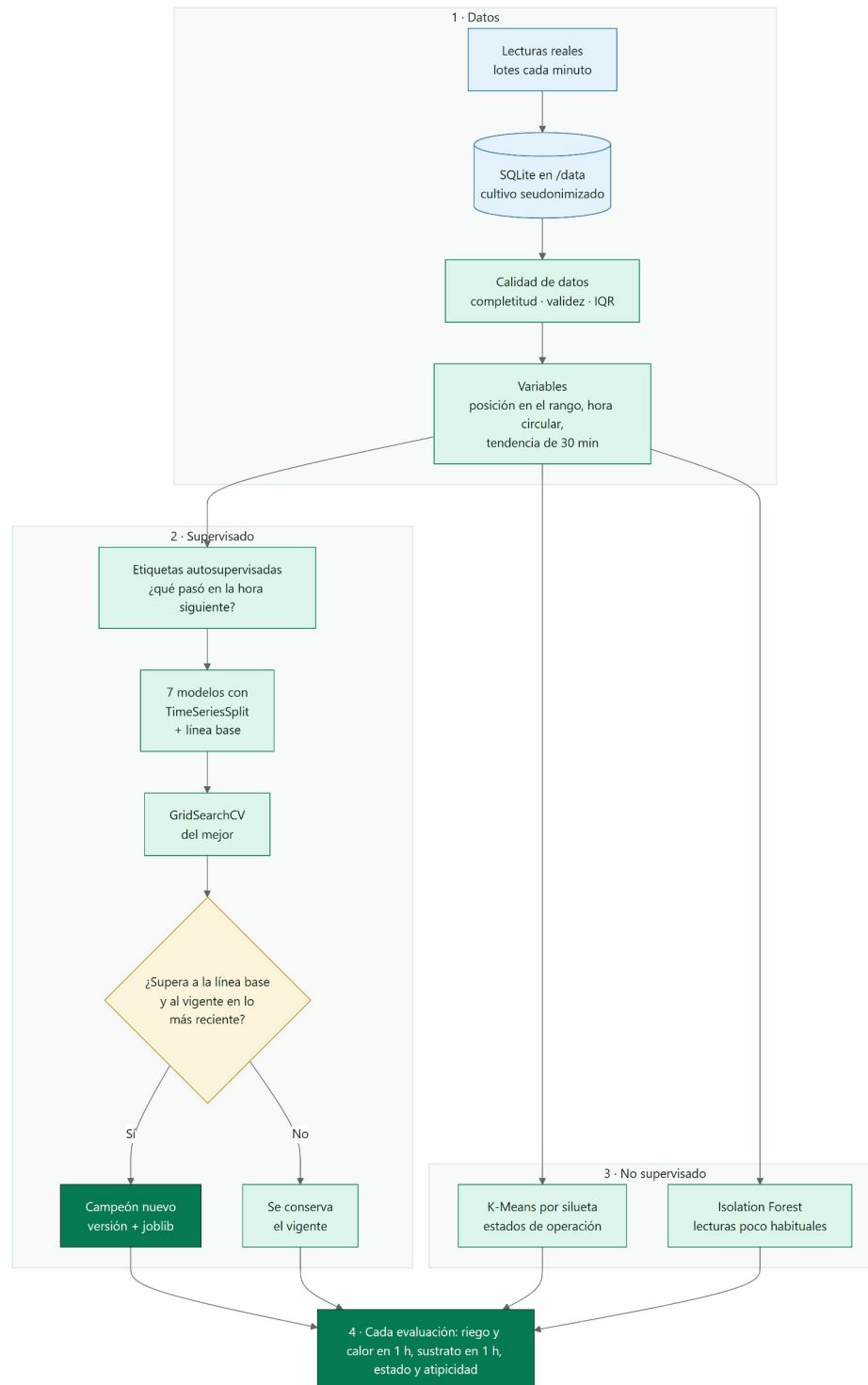


Figura 38 · Aprendizaje continuo con lecturas reales



MERMAID

```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    subgraph prep["1 · Datos"]
        direction LR
        lect["Lecturas reales<br/>lotos cada minuto"] --> alm["SQLite en /data<br/>cultivo
seudonimizado"]
        alm --> cal["Calidad de datos<br/>completitud · validez · IQR"]
        cal --> var["Variables<br/>posición en el rango, hora circular,<br/>tendencia de 30 min"]
    end
    subgraph sup["2 · Supervisado"]
        direction LR
        eti["Etiquetas autosupervisadas<br/>¿qué pasó en la hora siguiente?"] --> cv["7 modelos
con<br/>TimeSeriesSplit<br/>+ línea base"]
        cv --> gs["GridSearchCV<br/>del mejor"]
        gs --> cc["¿Supera a la línea base<br/>y al vigente en lo<br/>más reciente?"]
        cc --> si["Sí"]
        si --> nuevo["Campeón nuevo<br/>versión + joblib"]
        cc --> no["No"]
        no --> vig["Se conserva<br/>el vigente"]
    end
    subgraph nosup["3 · No supervisado"]
        direction LR
        km["K-Means por silueta<br/>estados de operación"]
        iso["Isolation Forest<br/>lecturas poco habituales"]
    end
    var --> eti
    var --> km
    var --> iso
    inf["4 · Cada evaluación: riego y calor en 1 h, sustrato en 1 h, estado y atipicidad"]
    nuevo --> inf
    vig --> inf
    km --> inf
    iso --> inf
    classDef data fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef step fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef ask fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef out fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    class lect,alm data
    class cal,var,eti,cv,gs,km,iso,vig step
    class cc ask
    class nuevo,inf out

```



Cada lectura real llega a la IA con el cultivo seudonimizado. La IA mide su calidad, etiqueta lo que pasó en la hora siguiente, compara siete modelos con validación cruzada temporal frente a una línea base, afina el mejor y solo lo pone en servicio si supera al vigente. Con aprendizaje no supervisado descubre los estados típicos de cada especie y reconoce lo poco habitual.

LECTURA CRÍTICA DE LA CONSTRUCCIÓN

Lo más difícil no fue programar funciones sino sostener la coherencia entre diez repositorios: un contrato que cambia en la API tiene que cambiar en el firmware, el simulador, la IA, la PWA y la documentación. Tres prácticas lo hicieron posible: contratos versionados, un QA que arma la plataforma completa en cada cambio y convenciones iguales en todos los repositorios.



PARTE VI

Pruebas

El acta pedía pruebas unitarias, de integración y de aceptación, con una cobertura mínima y un tiempo de respuesta máximo. La plataforma llegó a una pirámide completa que se ejecuta sola en cada cambio; esta parte muestra cómo está armada, qué criterios del acta cumple y cuáles siguen sin medirse.



SECCIÓN 29

Estrategia

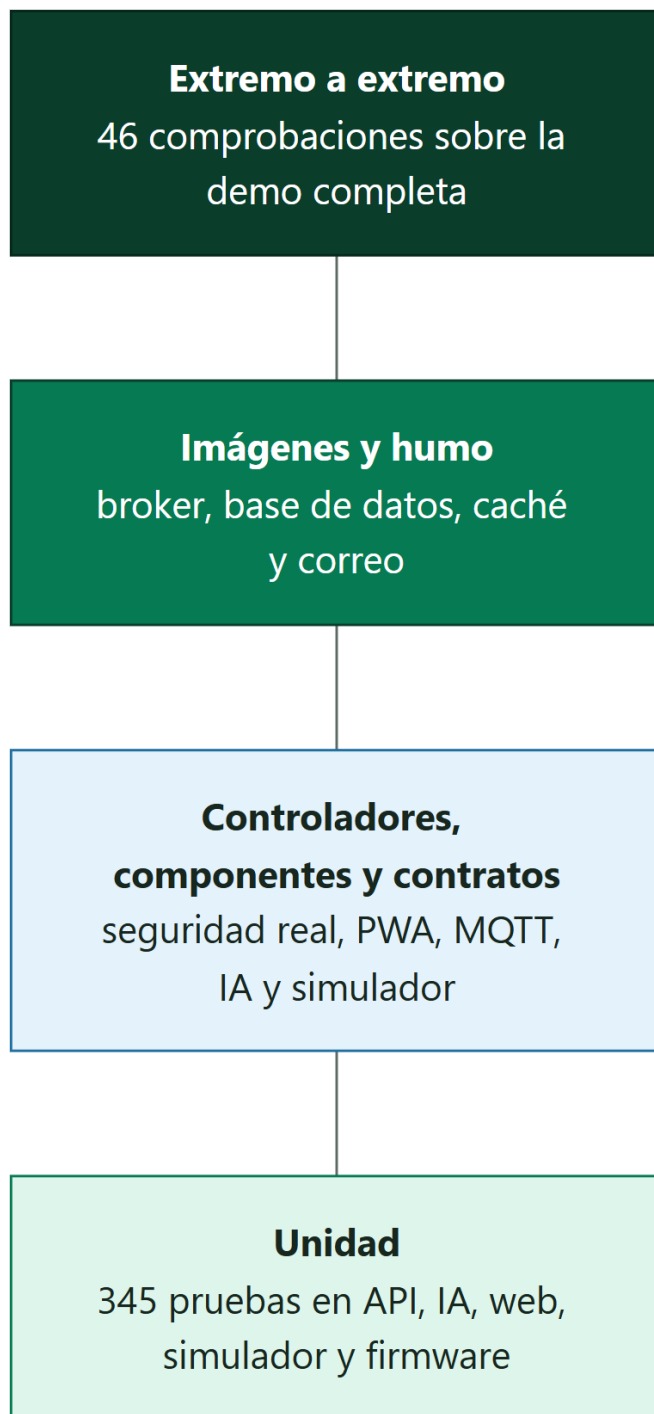


Figura 39 · Pirámide de pruebas



MERMAID

```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%

```

```

flowchart TB

```

```

    e2e["<b>Extremo a extremo</b><br>46 comprobaciones sobre la demo completa"]

```

```

    humo["<b>Imágenes y humo</b><br>broker, base de datos, caché y correo"]

```

```

    contrato["<b>Controladores, componentes y contratos</b><br>seguridad real, PWA, MQTT, IA y
simulador"]

```

```

    unidad["<b>Unidad</b><br>345 pruebas en API, IA, web, simulador y firmware"]

```

```

    e2e --- humo --- contrato --- unidad

```

```

classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F

```

```

classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F

```

```

classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F

```

```

classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F

```

```

classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF

```

```

classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF

```

```

classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F

```

```

class e2e deep

```

```

class humo core

```

```

class contrato water

```

```

class unidad leaf

```

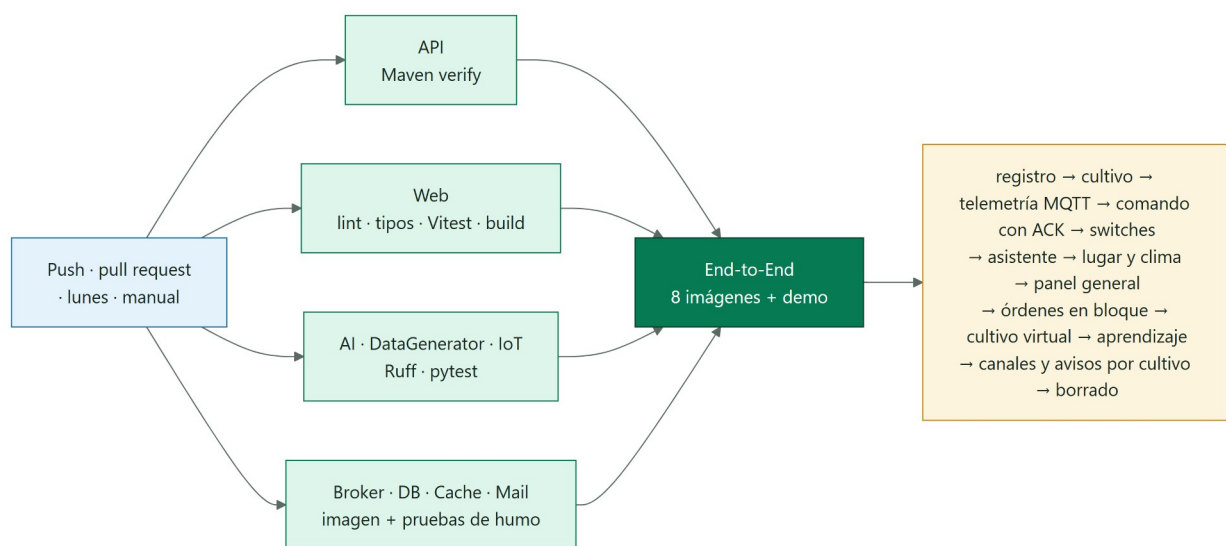


Figura 40 · Workflow de QA



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart LR
    trigger["Push · pull request<br/>· lunes · manual"] --> api["API<br/>Maven verify"]
    trigger --> web["Web<br/>lint · tipos · Vitest · build"]
    trigger --> py["AI · DataGenerator · IoT<br/>Ruff · pytest"]
    trigger --> ct["Broker · DB · Cache · Mail<br/>imagen + pruebas de humo"]
    api --> e2e["End-to-End<br/>8 imágenes + demo"]
    web --> e2e
    py --> e2e
    ct --> e2e
    e2e --> flow["registro → cultivo → telemetría MQTT → comando con ACK → switches<br/>→
asistente → lugar y clima → panel general<br/>→ órdenes en bloque → cultivo virtual →
aprendizaje<br/>→ canales y avisos por cultivo → borrado"]
    classDef start fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef job fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef final fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef detail fill:#FDF4DD,stroke:#C98D12,color:#17261F
    class trigger start
    class api,web,py,ct job
    class e2e final
    class flow detail
```

La base son 345 pruebas unitarias y de componentes (147 en la API, 84 en la IA, 65 en la PWA, 34 en el simulador y 15 en el firmware); encima, las pruebas de humo de cada imagen; y arriba, 46 comprobaciones de extremo a extremo sobre la demo completa. El QA corre en cada cambio de la organización, cada lunes y a mano.

SECCIÓN 30

Criterios de aprobación del acta

CRITERIO	ESTADO	CÓMO CERRARLO
Simulación IoT, API con JWT, portal en tiempo real y automatización	Cumplido	Probado por el E2E



CRITERIO	ESTADO	CÓMO CERRARLO
Cobertura de al menos 80 % en módulos críticos	Sin medir	Publicar la cobertura en el CI de API, IA y PWA y fijar el umbral por módulo
Latencia de la API menor a 500 ms	Sin medir formalmente	Prueba de carga con varios cultivos simulados y registro del percentil 95
TLS y JWT	Cumplido	Verificado en producción y en las pruebas del broker
Pruebas unitarias, de integración y de aceptación sin defectos críticos	Cumplido	La aceptación por caso de uso está en el recorrido del proyecto
Documentación: requisitos, manual, C4, modelo de datos y despliegue	Cumplido	Estos tres documentos, los READMEs y la guía de la demo

LECTURA CRÍTICA

Las pruebas encontraron lo que se buscaba y dejaron pasar lo que no: la caída de producción por las rutas públicas no la detectó ninguna prueba porque ninguna recorría la configuración real. El E2E actual, que se registra e ingresa sobre las imágenes publicadas, cerraría ese hueco. Los dos criterios numéricos del acta (cobertura y latencia) siguen sin medición y son la deuda de pruebas más clara.

SECCIÓN 31

Calidad de los modelos

Las pruebas de la IA entrenan con dos cultivos simulados durante tres días y exigen que cada modelo supere a su línea base en el 20 % de lecturas más recientes: F1 de 0,89 frente a 0,46 para anticipar el secado, F1 de 0,92 frente a 0,49 para anticipar el calor y un error de 1,2 % frente a 4,5 % al estimar la humedad del sustrato en una hora. Son series simuladas: en producción cada especie muestra sus puntajes reales en la página Aprendizaje.



PARTE VII

Mejora continua

SmartPot no terminó con una entrega: siguió mejorando en ciclos cortos que parten de un diagnóstico sobre la plataforma real. Esta parte describe ese ciclo, lo que cada vuelta cambió, los mecanismos que mejoran el sistema sin intervención, los indicadores con los que se medirá el próximo ciclo y las lecciones que deja el proyecto.

SECCIÓN 32

El ciclo de mejora

Cada ciclo empieza con un diagnóstico sobre la plataforma real, no sobre el plan: qué falla, qué está expuesto, qué pide la persona. Termina cuando el cambio está desplegado, probado de extremo a extremo y documentado.

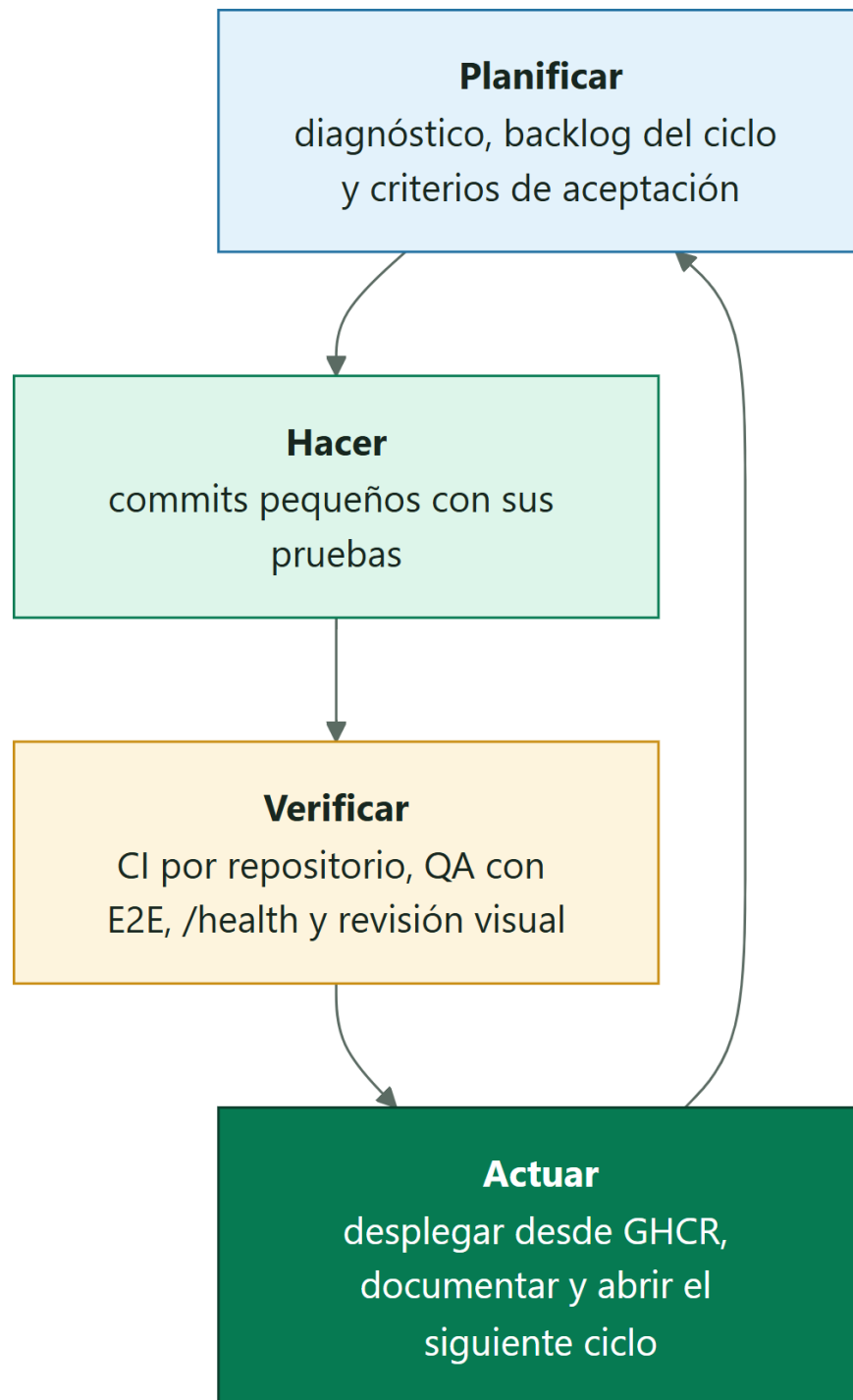


Figura 41 · Ciclo PDCA de SmartPot



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%%
flowchart TB
    plan["<b>Planificar</b><br/>diagnóstico, backlog del ciclo y criterios de aceptación"] -->
    hacer["<b>Hacer</b><br/>commits pequeños con sus pruebas"]
    hacer --> verificar["<b>Verificar</b><br/>CI por repositorio, QA con E2E, /health y
    revisión visual"]
    verificar --> actuar["<b>Actuar</b><br/>desplegar desde GHCR, documentar y abrir el
    siguiente ciclo"]
    actuar --> plan
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class plan water
    class hacer leaf
    class verificar sun
    class actuar core
```



SECCIÓN 33

Ciclos de mejora

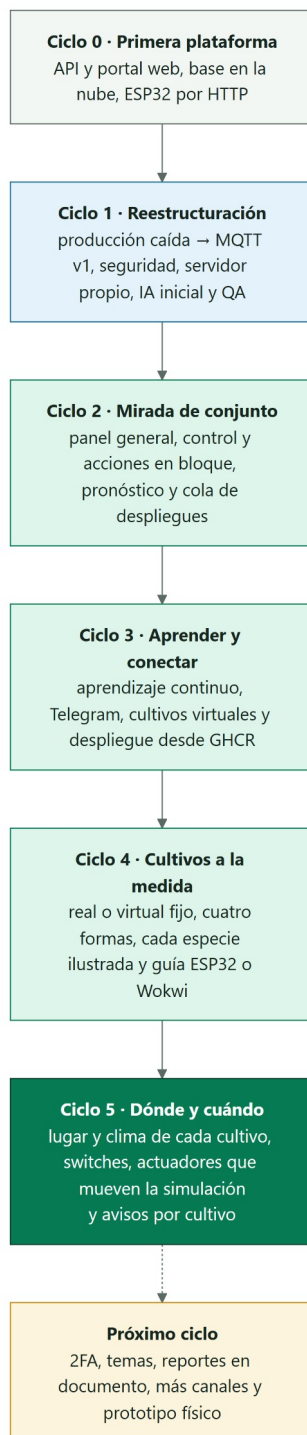


Figura 42 · Ciclos de mejora



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    c0["<b>Ciclo 0 · Primera plataforma</b><br/>API y portal web, base en la nube, ESP32 por HTTP"]
    c1["<b>Ciclo 1 · Reestructuración</b><br/>producción caída → MQTT v1, seguridad, servidor propio, IA inicial y QA"]
    c2["<b>Ciclo 2 · Mirada de conjunto</b><br/>panel general, control y acciones en bloque, pronóstico y cola de despliegues"]
    c3["<b>Ciclo 3 · Aprender y conectar</b><br/>aprendizaje continuo, Telegram, cultivos virtuales y despliegue desde GHCR"]
    c4["<b>Ciclo 4 · Cultivos a la medida</b><br/>real o virtual fijo, cuatro formas, cada especie ilustrada y guía ESP32 o Wokwi"]
    c5["<b>Ciclo 5 · Dónde y cuándo</b><br/>lugar y clima de cada cultivo, switches, actuadores que mueven la simulación<br/>y avisos por cultivo"]
    c6["<b>Próximo ciclo</b><br/>2FA, temas, reportes en documento, más canales y prototipo físico"]
    c0 --> c1 --> c2 --> c3 --> c4 --> c5 -.→ c6
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class c0 muted
    class c1 water
    class c2 leaf
    class c3 leaf
    class c4 leaf
    class c5 core
    class c6 sun
```

CICLO	DISPARADOR	CAMBIOS PRINCIPALES	EVIDENCIA
0 · Primera plataforma	Proyecto de diseño	API, portal web, base en la nube y ESP32	Repositorios desde 2024
1 · Reestructuración	Producción caída, servicios expuestos y rutas sin control de dueño	MQTT v1, servidor propio, seguridad, IA inicial, documentación y QA	Plataforma en smartpot.app; QA con E2E en verde



CICLO	DISPARADOR	CAMBIOS PRINCIPALES	EVIDENCIA
2 · Mirada de conjunto	Una persona con varios cultivos no tenía vista general	Panel general, control y acciones en bloque, pronóstico y cola de despliegues	Nuevos endpoints y 21 comprobaciones E2E
3 · Aprender y conectar	La IA no mejoraba con el uso y no había avisos fuera de la PWA	Aprendizaje continuo, Telegram, macetas virtuales con clima real y despliegue desde GHCR	260 pruebas y 30 comprobaciones E2E
4 · Cultivos a la medida	La maceta virtual mezclaba simulación y hardware, y no todo cultivo es una maceta	Cultivo real o virtual fijo al crearlo, cuatro formas, cada especie ilustrada con sus actuadores, guía ESP32 o Wokwi y aprendizaje solo con cultivos reales	281 pruebas y 35 comprobaciones E2E
5 · Dónde y cuándo	La ilustración no sabía dónde estaba el cultivo, los botones de los actuadores pedían dos clics y los avisos eran iguales para todos los cultivos	Lugar y clima de cada cultivo con el consejo del asistente, switches que solo aceptan órdenes que cambian algo, actuadores que mueven la simulación en los tres modos, avisos por cultivo con resúmenes y chats compartidos y la CA del broker dentro de <code>config.py</code>	345 pruebas y 46 comprobaciones E2E
Próximo	Pendientes de requisitos y de la investigación	Sección 36	—

SECCIÓN 34

Mejora que ocurre sola

MECANISMO	QUÉ MEJORA	FRECUENCIA
Aprendizaje continuo	Los modelos se reentrenan con lecturas reales y solo reemplazan al vigente si lo superan	Cada cierto número de lecturas nuevas por especie
Dependabot	Dependencias al día en todos los repositorios	Semanal en casi todos los ecosistemas; cada actualización pasa por el CI y se revisa antes de fusionar
CodeQL	Detección de vulnerabilidades en el código	En cada cambio
QA programado	La plataforma completa se arma y se prueba aunque nadie haya cambiado nada	Cada lunes

SECCIÓN 35

Indicadores

INDICADOR	DEFINICIÓN	FUENTE	META PROPUESTA
Tiempo en rango	Porcentaje de lecturas dentro del rango ideal de la especie	Lecturas en MongoDB	Subir semana a semana en cultivos con modo automático
Intervenciones manuales	Comandos de la persona frente a los del agente	Colección de comandos	Bajar a medida que el agente aprende
Ventaja de los modelos	Puntaje de cada modelo frente a su línea base	Página Aprendizaje	Siempre por encima de la línea base
Latencia de la API	Percentil 95 del tiempo de respuesta	Prueba de carga	Menos de 500 ms (criterio del acta)
Cobertura de pruebas	Porcentaje de líneas cubiertas en módulos críticos	CI de cada repositorio	80 % (criterio del acta)
Frecuencia de despliegue	Despliegues a producción por semana	Workflow central	Al menos uno por ciclo
Cambios fallidos	Despliegues que requieren revertir	Workflow central y <code>/health</code>	Cero
Adopción	Cuentas y cultivos activos, en forma agregada y anónima	API	Línea base en el piloto académico

SECCIÓN 36

Backlog del próximo ciclo

PRIORIDAD	TRABAJO	ORIGEN
Must	Medir cobertura y latencia en el CI	Criterios de aprobación del acta
Must	Tiempo en rango e intervenciones como indicadores visibles	Marco lógico
Must	Métricas de adopción agregadas y anónimas	Estudio de mercado
Should	Autenticación en dos pasos	RNF-001
Should	Umbrales propios por cultivo y calibración remota	RF-008 y RF-002
Should	Reportes descargables en documento	RF-012
Could	Actualización en vivo hacia la PWA	RNF-003
Could	Temas y ajustes de visualización	RNF-006
Could	Más canales de notificación	Arquitectura de canales
Later	Prototipo físico y dos ciclos de cultivo comparados	Propuesta de investigación



PRIORIDAD	TRABAJO	ORIGEN
Later	Redes recurrentes cuando haya meses de datos reales	Propuesta de investigación

SECCIÓN 37

Lecciones aprendidas

DISCIPLINA	LECCIÓN
Formulación	Formular el proyecto como flujo de datos, y no como aparato, fue lo que permitió que el software valiera sin hardware
Evaluación	Un estudio de mercado sin fuente primaria da un buen mapa y malos números; la variable que decide es la conversión y hay que medirla
Gestión	Un plan que no registra avance no detecta desvíos; la evidencia automática (pruebas, QA y despliegues) fue mejor termómetro que el porcentaje declarado
Gestión	Nivelar recursos y aceptar una fecha más tarde es una decisión de gestión, no un fracaso
Análisis	Un requisito que no se puede probar termina redefinido por quien lo construye; conviene escribirlo verificable desde el principio
Diseño	Los contratos primero (MQTT v1 y OpenAPI) desacoplan equipos y repositorios mejor que cualquier red de precedencias
Construcción	Diez repositorios solo son manejables con convenciones idénticas y un QA que los arme juntos
Pruebas	Las pruebas deben recorrer la configuración real, no solo el código
Mejora continua	Ciclos cortos con diagnóstico sobre la plataforma real avanzaron más en semanas que la planeación de un semestre